Cloud Messenger™
Learn • Build • Visualize • Troubleshoot • Grow

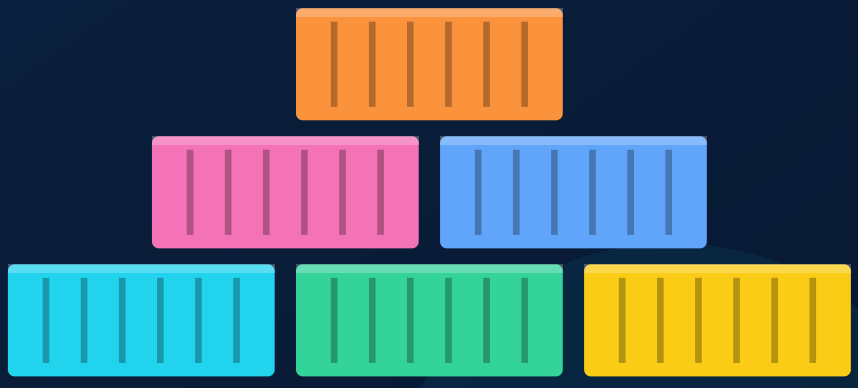
THE VISUAL HANDBOOK

Docker

From Virtualization to AWS ECS

Every diagram explained, in order, in plain words.
With the commands to get Docker running and links to every resource.

11 chapters | 34 annotated figures | click-through links



build once | ship anywhere | run everywhere

Created by: cloud-messenger™

Website cloudmessenger.in

LinkedIn www.linkedin.com/in/cloud-messenger

GitHub github.com/Cloud-Messenger

#buildwithCloud-Messenger

Learn • Build • Visualize • Troubleshoot • Grow | Made with care by cloud-messenger™

© 2026 cloudmessenger. The compilation, structure, layout, design and branding of this workbook are the original work of cloudmessenger.

How to read this handbook

WHO IS THIS FOR?

Complete beginners. You do not need to know what a container, a hypervisor or a Dockerfile is. Each chapter takes one idea, shows you the original diagram, then “decodes” it in everyday words with real commands you can type.

1 THE PICTURE

Every figure gets its own page, full size, so you can zoom in on the small print.

2 THE DECODED PAGE

A thumbnail, an “In plain words” explanation, three key points and the source links.

3 THE LINKS

Every link is clickable and the full address is printed under it, so it also works on paper.

ANATOMY OF A CHAPTER

Chapter opener (a story and a goal) → **Figure page** (the original image) → **Decoded page** (plain words, key points, links). Tap any line of the contents page or any chapter on the roadmap to jump straight there. The PDF outline panel in your reader lists every chapter too.

LEARNING TIPS

- Read in order: each chapter uses words introduced in the previous one.
- Type the commands yourself. A command you typed once is worth ten you only read.
- Stuck? Check the “Common mistakes” notes and the glossary at the back.
- Official docs change; if a link has moved, search the title shown above the address.

A NOTE ABOUT THE SOURCE IMAGES

This handbook is compiled from 35 diagrams, whiteboard sketches and screenshots, plus one technical guide that forms Chapter 11. They were made at different times, by different tools, so styles vary; their facts are explained here in one consistent voice. Where a figure contains something worth correcting or clarifying, a small amber note says so.

© 2026 cloudmessenger

Contents

Chapter 1 Virtualization 6

Fig 1 Virtualization for AWS Beginners 7
Fig 2 Virtualization & Hypervisors Explained: the master chart 9
Fig 3 Type-1 vs Type-2 in one glance 11
Fig 4 Hypervisor Types: why AWS runs Type-1 12
Fig 5 The Super Boss and the Guest 14

Chapter 2 Cloud Models 15

Fig 6 Cloud Deployment Models 16

Chapter 3 The Container Mindset 17

Fig 7 Virtualization vs Containerization: the mindset behind Docker 18
Fig 8 Docker is a zip file for applications 20
Fig 9 Where Docker fits among DevOps tools 21

Chapter 4 Docker Core Concepts 22

Fig 10 The golden rule: DF → I → C 23
Fig 11 Dockerfile, Image, Container: three words, three jobs 24
Fig 12 Docker architecture: Engine, Image, Container, Hub 25
Fig 13 Simple Docker flow: Dev, Test, Prod 26
Fig 14 Two stores: GitHub for code, Docker Hub for images 27

Chapter 5 Get Docker 28

Fig 15 Real terminal history: installing Docker on Amazon Linux 31
Install guide: three ways to get Docker 29

Chapter 6 Your First Container 32

Fig 16 The plan on one whiteboard: Docker Hub to EC2 33
Fig 17 From DockerHub to EC2: the 7 steps 34
Fig 18 Deploy Nginx on EC2: the complete step-by-step 36
Fig 19 Phases 5–7: from stock nginx to your own site 38

Chapter 7 The Dockerfile 39

Fig 20 Dockerfile Explained: five core instructions 40
Fig 21 Dockerfile line by line: all 15 instructions 42
Fig 22 How COPY index.html works 44
Fig 23 Cloud Messenger: build your own nginx website image 46
Fig 24 From code to container: a streaming-site landing page 48

Chapter 8 Docker Networking 50

Fig 25 Networking whiteboard, part 1: the machine's network cards 51
Fig 26 Whiteboard, part 2: several containers, several ports 52
Fig 27 Whiteboard, part 3: Bridge, Host, None 53
Fig 28 Docker Networking: six drivers compared 54
Fig 29 Networking in real life: analogies and production scenarios 56
Fig 30 Networking commands you will actually type 58

Chapter 9 Docker Compose 60

Fig 31 Why is there an Ubuntu container in Docker Compose? 61
Fig 32 Compose YAML explained line by line (racing_car + ubuntu_admin) 63
Fig 33 Every Compose component with diagrams (flipkart-web + ubuntu-admin) 65

Chapter 10 ECR & ECS on AWS 67

Fig 34 AWS ECR & ECS: complete guide 68

Chapter 11 Production Playbook 70

Executive summary and architecture 71
Visual project showcase 72
Phase 1: Server preparation (8 commands) 73
Phases 2 and 3: Workspace, image and build 74
Phase 4: Container launch 75
Production readiness checklist 76
Safer reference implementation 77
Command summary and conclusion 78
References 79

Command cheat sheet 80
Glossary 81
Resource index (all links) 82
Credits & notes 84

© 2026 cloudmessenger

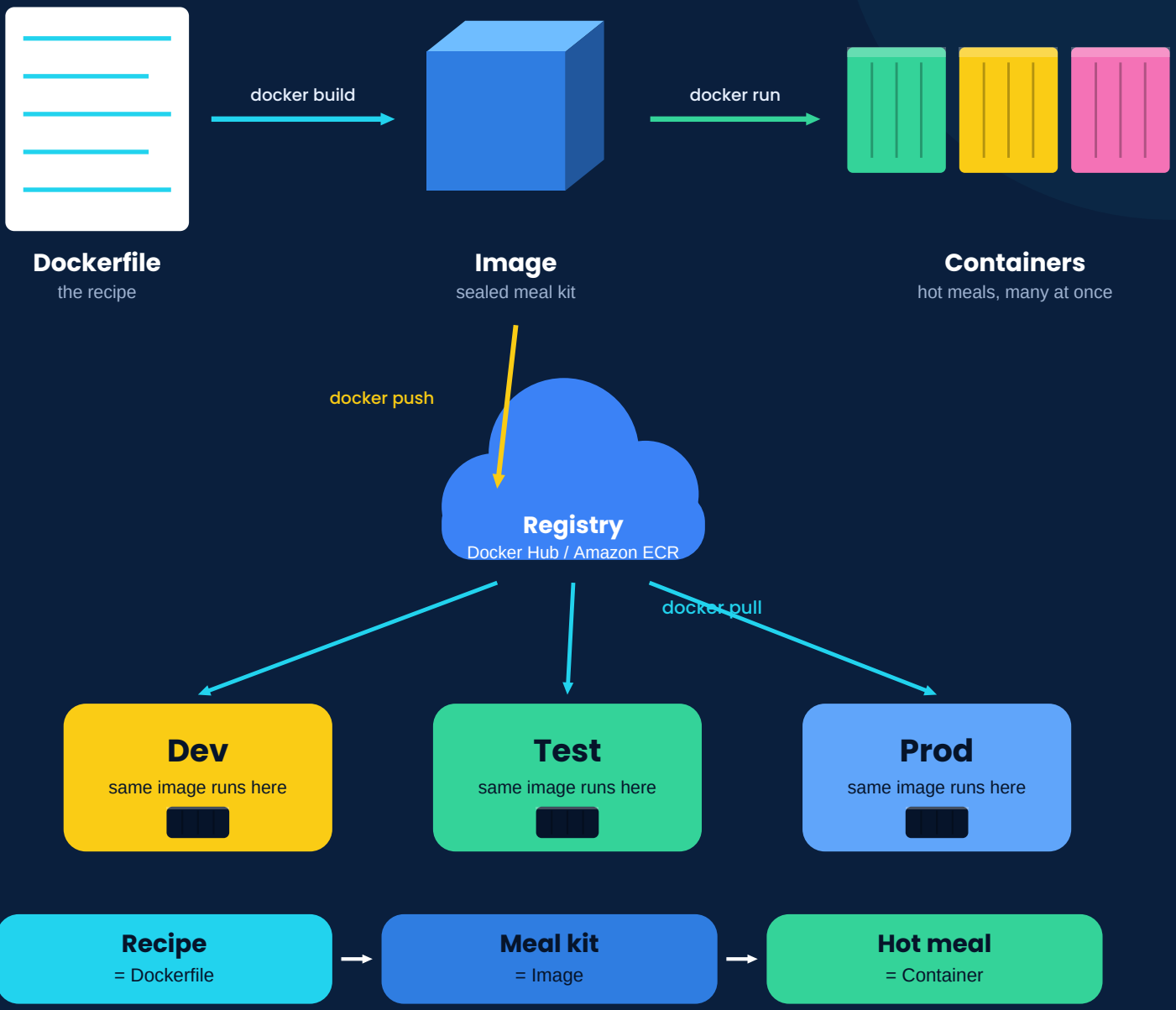
The Docker learning roadmap

1. **Virtualization**
   One computer, many computers — page 6
2. **Cloud Models**
   Who owns the building? — page 15
3. **The Container Mindset**
   Pack the app, not the whole computer — page 17
4. **Docker Core Concepts**
   Dockerfile → Image → Container — page 22
5. **Get Docker**
   Install it on your laptop or your cloud server — page 28
6. **Your First Container**
   Pull nginx, run it, open it in a browser — page 32
7. **The Dockerfile**
   Write your own recipe — page 39
8. **Docker Networking**
   How containers talk to each other and the world — page 50
9. **Docker Compose**
   Many containers, one file — page 60
10. **ECR & ECS on AWS**
    Ship containers to the cloud — page 67
11. **Production Playbook**
    From code to container, done properly — page 70

© 2026 cloudmessenger

Docker in one picture

Keep this page in mind. Every later figure is a zoom-in on one part of it.



IMAGE

A blueprint or recipe. It is not running. It can create many containers.

CONTAINER

A running instance. The application is alive. It was created from an image.

© 2026 cloudmessenger

CHAPTER 1 OF 11

Virtualization

One computer, many computers

THINK OF IT LIKE THIS

A big apartment building (the hardware) is split into separate flats (virtual machines). The building manager who decides who gets how much water and electricity is the hypervisor.

YOU WILL LEARN

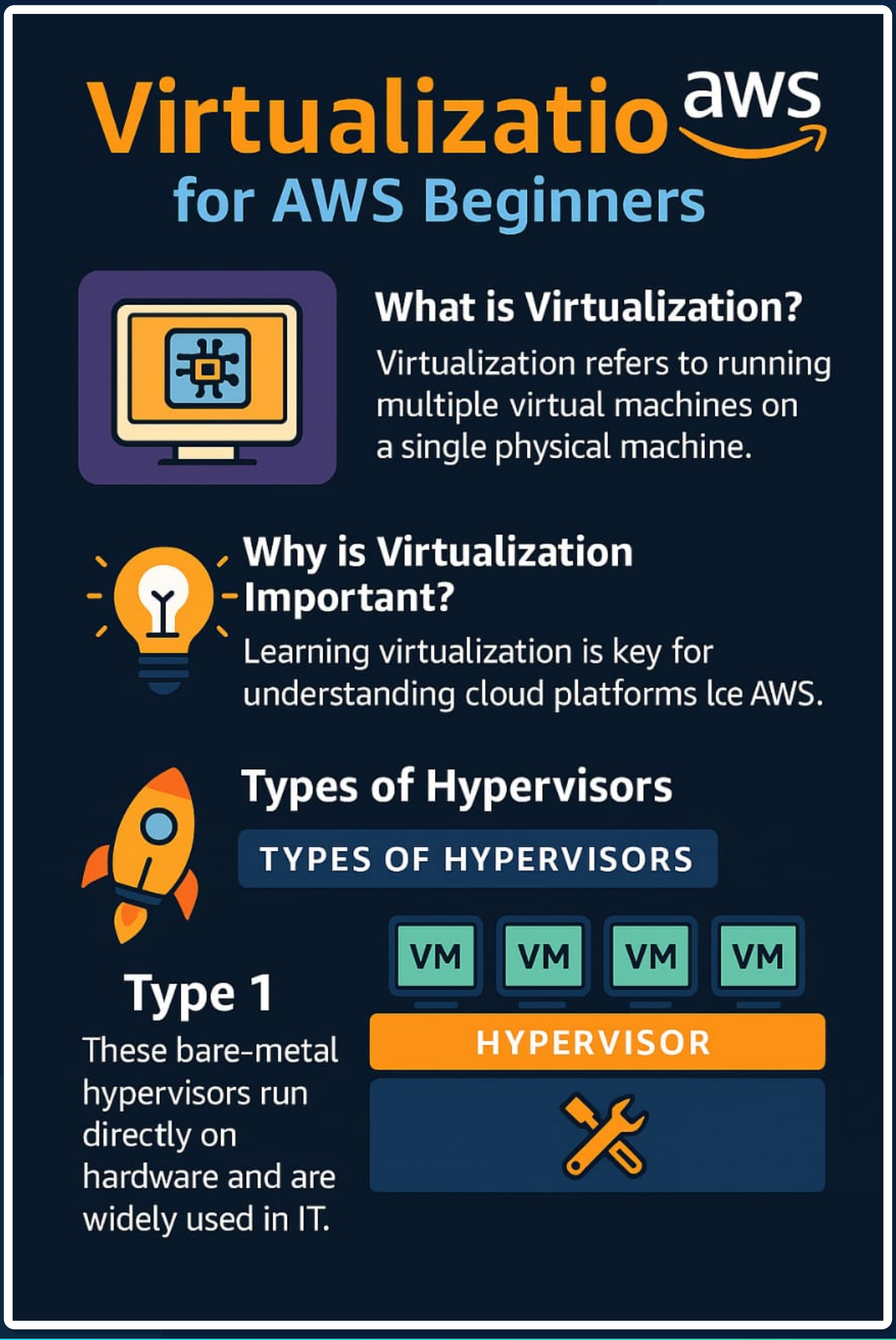
- What a virtual machine is and why clouds need them
- Type-1 (bare metal) vs Type-2 (hosted) hypervisors
- Why AWS chooses the Type-1 style

IN THIS CHAPTER

Figure 1 Virtualization for AWS Beginners (page 7)
Figure 2 Virtualization & Hypervisors Explained: the master chart (page 9)
Figure 3 Type-1 vs Type-2 in one glance (page 11)
Figure 4 Hypervisor Types: why AWS runs Type-1 (page 12)
Figure 5 The Super Boss and the Guest (page 14)

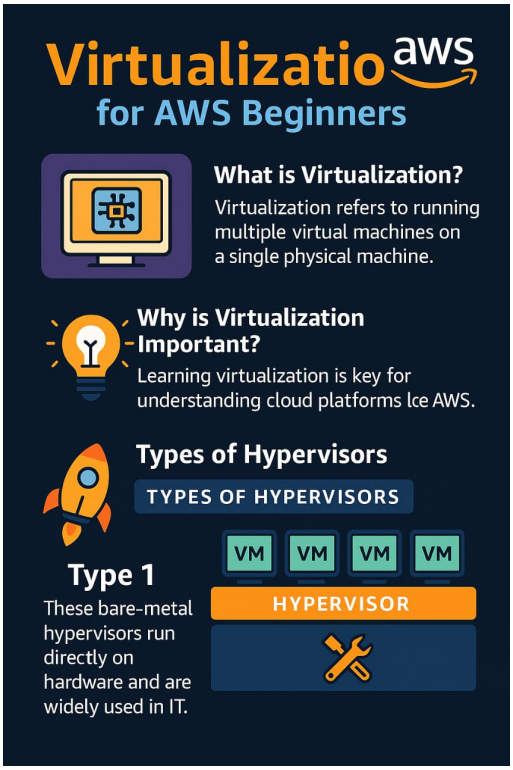
© 2026 cloudmessenger

Figure 1: Virtualization for AWS Beginners



© 2026 cloudmessenger

Figure 1: Virtualization for AWS Beginners



tap to see the full figure

IN PLAIN WORDS

Virtualization means one real computer pretends to be several smaller computers. Each pretend computer is a **virtual machine (VM)**. This card also introduces the manager that makes it possible, the **hypervisor**, and shows the Type 1 kind sitting directly on the hardware with VMs on top.

KEY POINTS

- One physical machine → many virtual machines
- The hypervisor shares CPU, memory and storage between VMs
- Cloud platforms such as AWS are built on this idea

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

AWS: What is virtualization?
https://aws.amazon.com/what-is/virtualization/

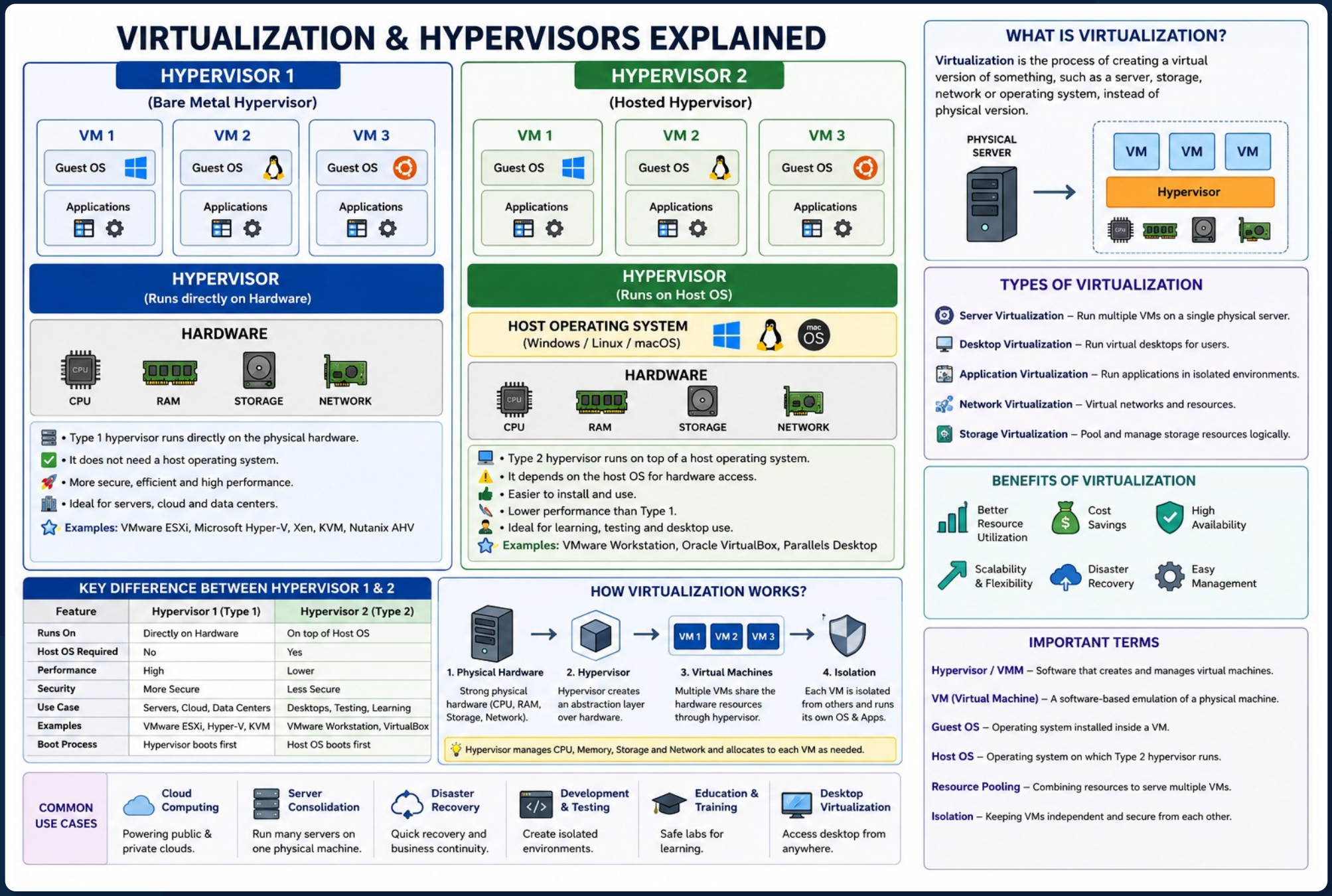
AWS: What is a hypervisor?
https://aws.amazon.com/what-is/hypervisor

CHAPTER IDEA

A big apartment building (the hardware) is split into separate flats (virtual machines). The building manager who decides who gets how much water and electricity is the hypervisor.

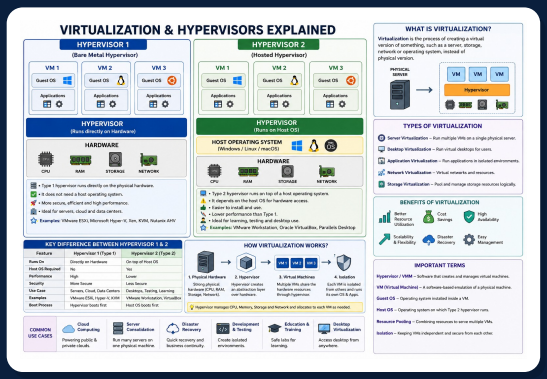
© 2026 cloudmessenger

Figure 2: Virtualization & Hypervisors Explained: the master chart



© 2026 cloudmessenger

Figure 2: Virtualization & Hypervisors Explained: the master chart



tap to see the full figure

IN PLAIN WORDS

The big picture on one page. On the left, **Type 1** runs straight on hardware (fast, used in data centres). In the middle, **Type 2** runs on a normal OS such as Windows or macOS (easy, good for learning). On the right are definitions, benefits and four steps: hardware, hypervisor, virtual machines, isolation.

KEY POINTS

- Type 1 examples: VMware ESXi, Hyper-V, Xen, KVM
- Type 2 examples: VirtualBox, VMware Workstation
- Isolation: each VM has its own OS and cannot disturb its neighbours

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

AWS: Type 1 vs Type 2 hypervisors
https://aws.amazon.com/compare/the-difference-between-type-1-and-type-2-hypervisors

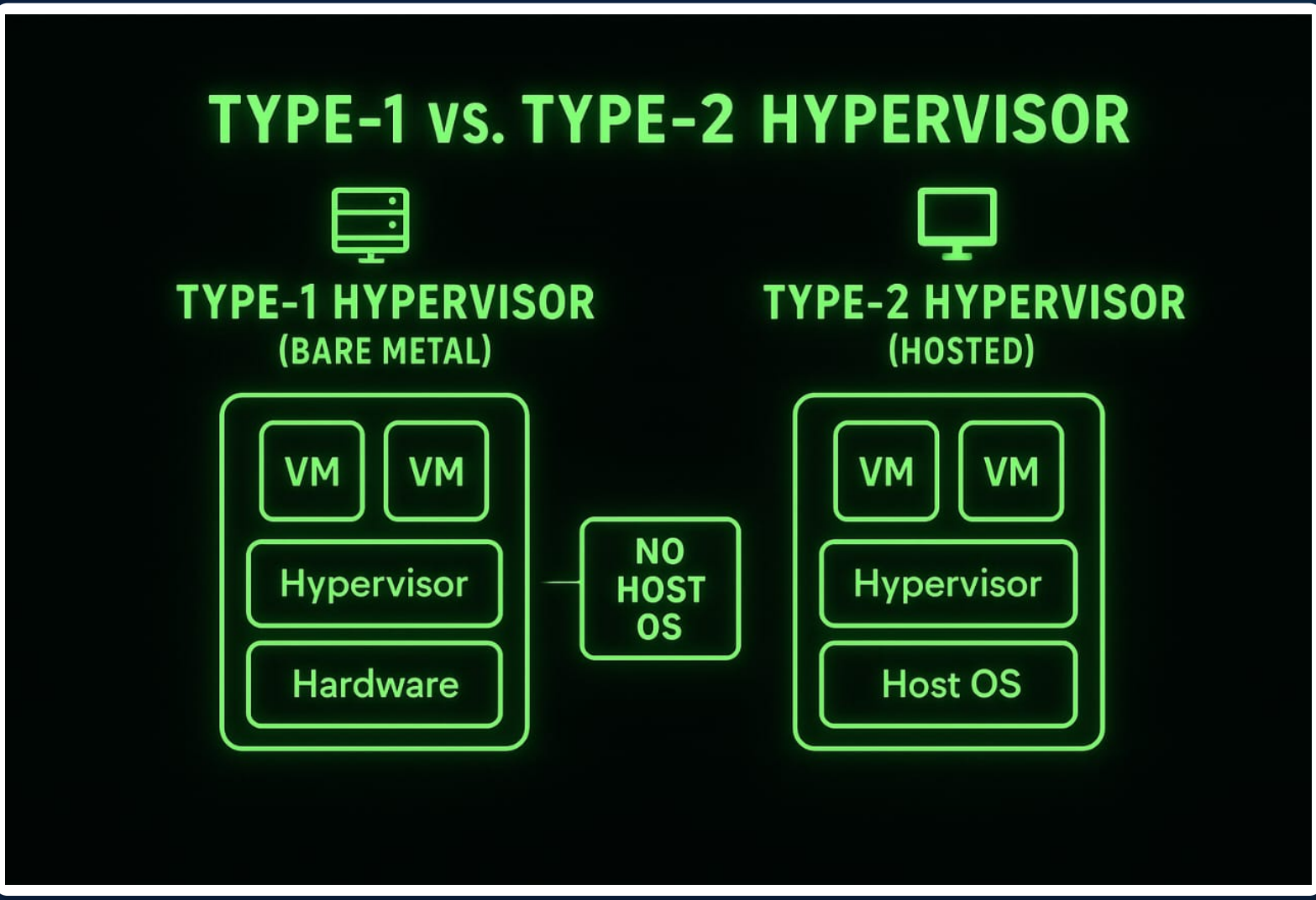
AWS whitepaper: Traditional virtualization primer
https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/traditional-virtualization-primer.html

CHAPTER IDEA

A big apartment building (the hardware) is split into separate flats (virtual machines). The building manager who decides who gets how much water and electricity is the hypervisor.

© 2026 cloudmessenger

Figure 3: Type-1 vs Type-2 in one glance



IN PLAIN WORDS

A neon sketch of the two stacks side by side. Type-1 goes Hardware, Hypervisor, VMs and is labelled **No Host OS**. Type-2 goes Host OS, Hypervisor, VMs. The only structural difference is that one extra operating-system layer.

KEY POINTS

- Remove the host OS layer and you get Type-1
- Fewer layers means less overhead
- Type-2 is just an application installed on your normal OS

RESOURCES (TAP TO OPEN)

AWS: Type 1 vs Type 2 hypervisors
https://aws.amazon.com/compare/the-difference-between-type-1-and-type-2-hypervisors

© 2026 cloudmessenger

Figure 4: Hypervisor Types: why AWS runs Type-1



© 2026 cloudmessenger

Figure 4: Hypervisor Types: why AWS runs Type-1



tap to see the full figure

IN PLAIN WORDS

A friendly cartoon summary. Type-1 is faster, more secure and enterprise-grade, so a giant like AWS, running at data-centre scale, uses this style. Type-2 is slower and meant for learning and testing on your own machine. Today AWS uses its own lightweight **Nitro** hypervisor with dedicated hardware cards, in the Type-1 spirit.

KEY POINTS

- Type-1: no Windows or Linux in between
- Type-2: runs on top of Windows or macOS, mainly for labs
- Scale and security are why clouds pick Type-1

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

AWS Nitro System
https://aws.amazon.com/ec2/nitro/

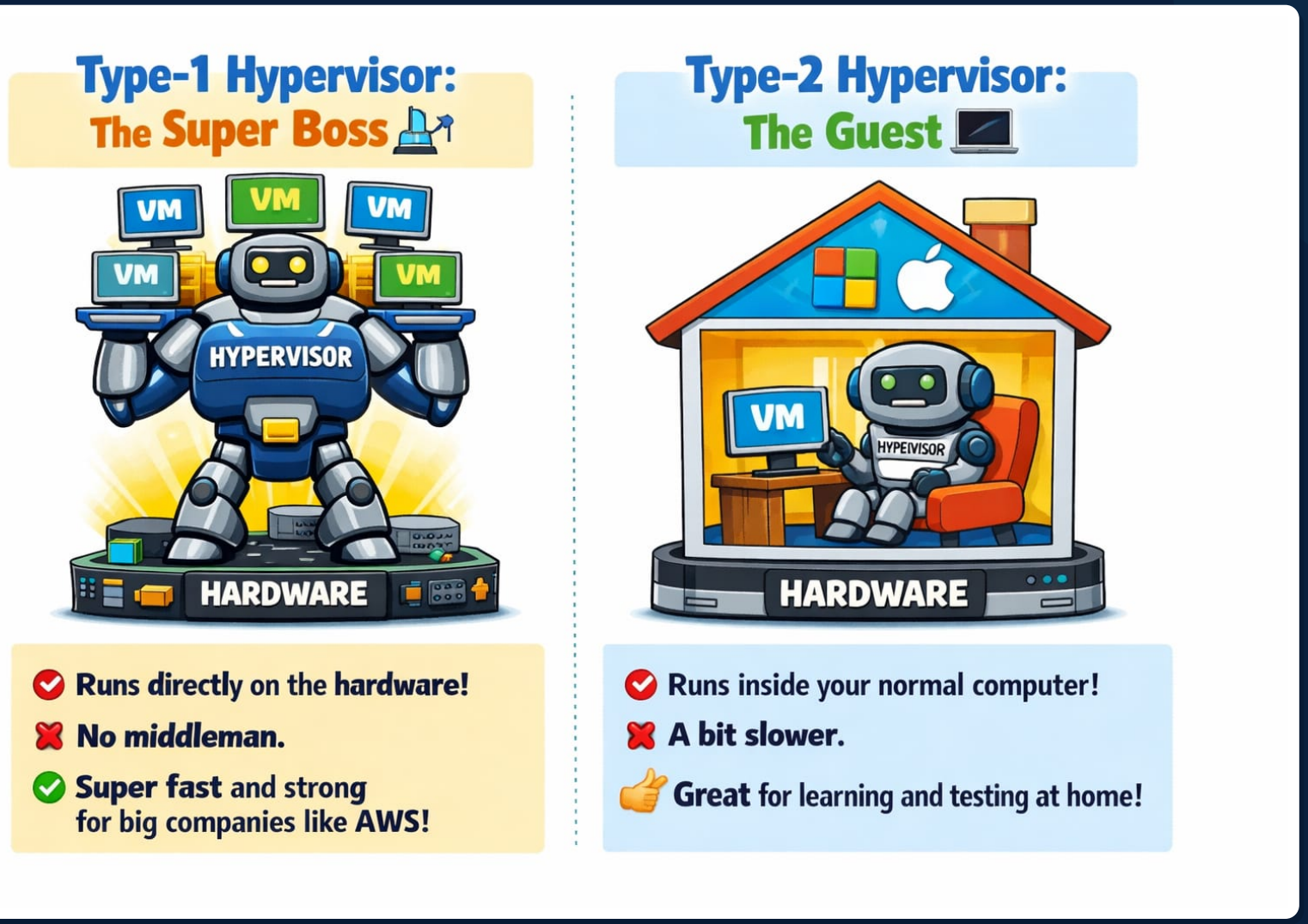
AWS: Type 1 vs Type 2 hypervisors
https://aws.amazon.com/compare/the-difference-between-type-1-and-type-2-hypervisors

CHAPTER IDEA

A big apartment building (the hardware) is split into separate flats (virtual machines). The building manager who decides who gets how much water and electricity is the hypervisor.

© 2026 cloudmessenger

Figure 5: The Super Boss and the Guest



IN PLAIN WORDS

A memory trick. The **Super Boss** robot stands directly on the hardware holding all the VMs, with no middleman. The **Guest** robot lives inside a house (Windows or macOS) that already sits on the hardware, so everything is a bit slower, but it is great for learning at home.

KEY POINTS

- Boss = Type-1: direct, fast, strong
- Guest = Type-2: lives inside your normal computer
- Use Type-2 to practise, Type-1 to run the world

RESOURCES (TAP TO OPEN)

AWS: What is a hypervisor?
https://aws.amazon.com/what-is/hypervisor

© 2026 cloudmessenger

CHAPTER 2 OF 11

Cloud Models

Who owns the building?

THINK OF IT LIKE THIS

Public cloud is a shared co-working space, private cloud is your own office tower, hybrid is owning an office but renting overflow desks, and multi-cloud is renting desks from several landlords.

YOU WILL LEARN

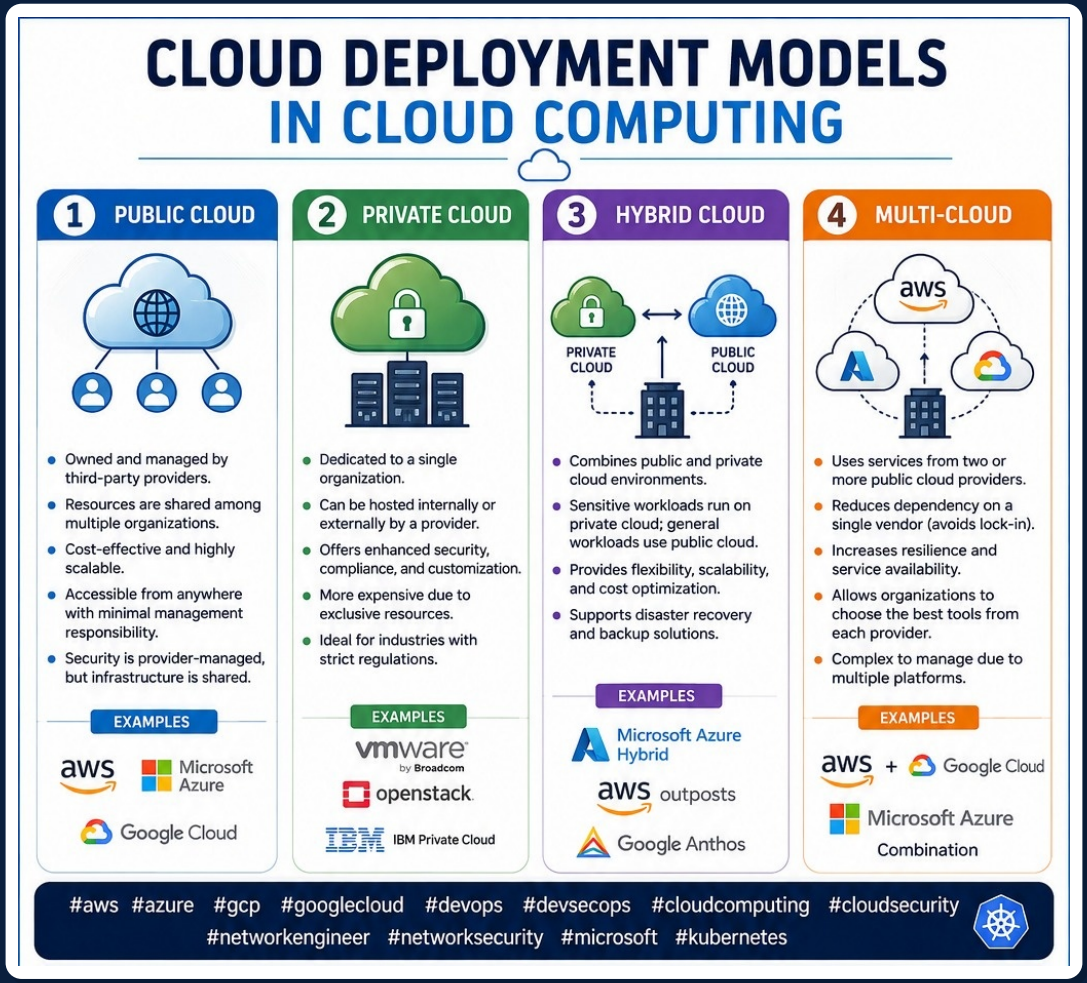
- Public, private, hybrid and multi-cloud
- Real examples of each
- Why companies mix providers

IN THIS CHAPTER

Figure 6 Cloud Deployment Models (page 16)

© 2026 cloudmessenger

Figure 6: Cloud Deployment Models



A promotional banner from another brand at the foot of the original image has been cropped out.

IN PLAIN WORDS

Where does your cloud live and who owns it? **Public**: shared and run by AWS, Azure or Google Cloud. **Private**: dedicated to one organisation (VMware, OpenStack, IBM). **Hybrid**: a mix, with sensitive workloads kept private. **Multi-cloud**: several providers at once to avoid being locked in.

KEY POINTS

- Public is cheap and scalable; private is secure but costly
- Hybrid balances flexibility with control
- Multi-cloud avoids vendor lock-in but is harder to manage

RESOURCES (TAP TO OPEN)

AWS: Types of cloud computing
https://aws.amazon.com/types-of-cloud-computing/

AWS: What is virtualization?
https://aws.amazon.com/what-is/virtualization/

© 2026 cloudmessenger

CHAPTER 3 OF 11

The Container Mindset

Pack the app, not the whole computer

THINK OF IT LIKE THIS

A VM is moving house with the entire building. A container is a tiffin box: just the meal and the spoon, and it opens the same way on any table.

YOU WILL LEARN

- Virtualization vs containerization
- Why Docker feels like a zip file for apps
- Where Docker sits next to Terraform, Ansible, CI/CD and Kubernetes

IN THIS CHAPTER

Figure 7 Virtualization vs Containerization: the mindset behind Docker (page 18)
Figure 8 Docker is a zip file for applications (page 20)
Figure 9 Where Docker fits among DevOps tools (page 21)

© 2026 cloudmessenger

Figure 7: Virtualization vs Containerization: the mindset behind Docker

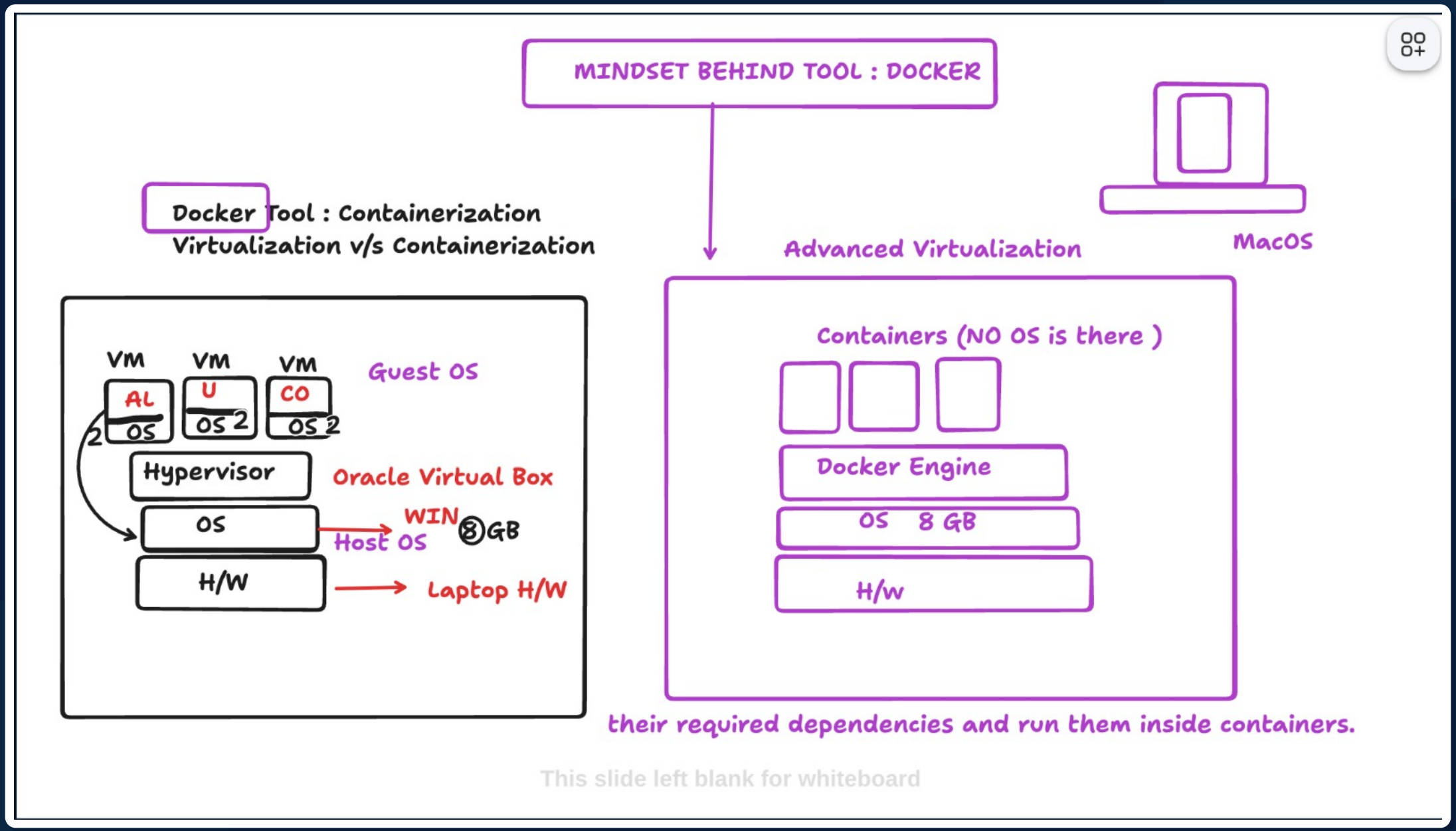
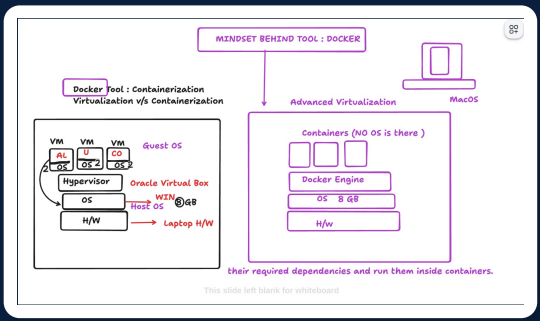


Figure 7: Virtualization vs Containerization: the mindset behind Docker



tap to see the full figure

IN PLAIN WORDS

Left: Hardware, host OS, hypervisor (Oracle VirtualBox here), then VMs that each carry their *own* guest OS (Alpine, Ubuntu, CentOS). Heavy, because every VM duplicates an operating system. **Right:** Hardware, one OS, the **Docker Engine**, then containers that share it. The 8 GB notes show how limited a laptop is.

KEY POINTS

- VMs bundle a full OS each; containers share the host's kernel
- Containers start in seconds and use far less memory
- Docker packages an app plus its dependencies and runs it inside a container

GOOD TO KNOW

The sketch says "no OS" for containers. Precisely, a container has no separate kernel; it still carries a few minimal OS files.

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: What is Docker?
https://docs.docker.com/get-started/docker-overview/

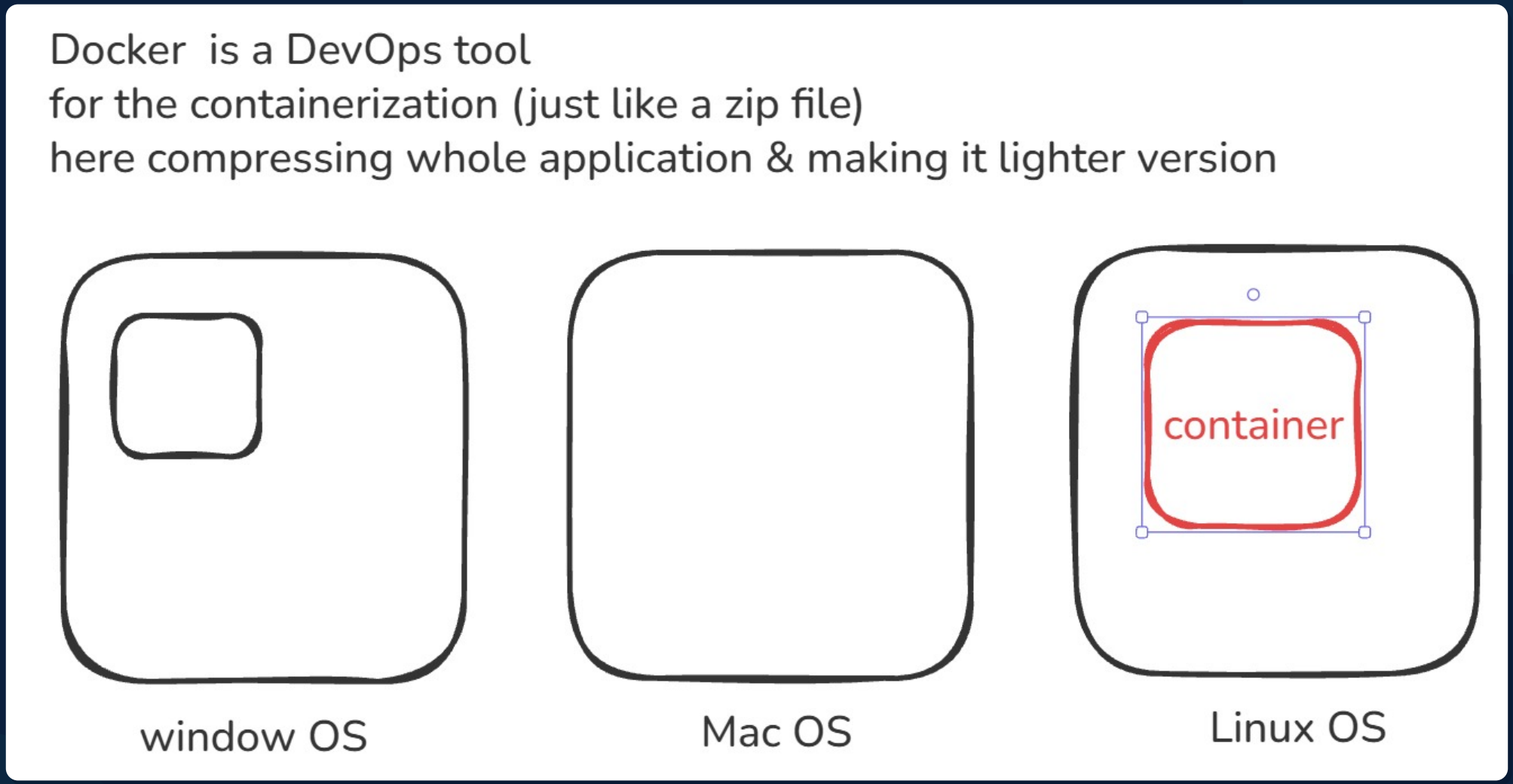
AWS: What is virtualization?
https://aws.amazon.com/what-is/virtualization/

CHAPTER IDEA

A VM is moving house with the entire building. A container is a tiffin box: just the meal and the spoon, and it opens the same way on any table.

© 2026 cloudmessenger

Figure 8: Docker is a zip file for applications



IN PLAIN WORDS

Docker is a DevOps tool for **containerization**, and it behaves like a zip file: it squeezes a whole application, with everything it needs, into a lighter package. The sketch draws the same container sitting inside Windows, macOS and Linux boxes: pack once, open anywhere.

KEY POINTS

- One package, many operating systems
- Lighter than shipping a whole virtual machine
- On Windows and macOS, Docker Desktop quietly provides a small Linux layer underneath

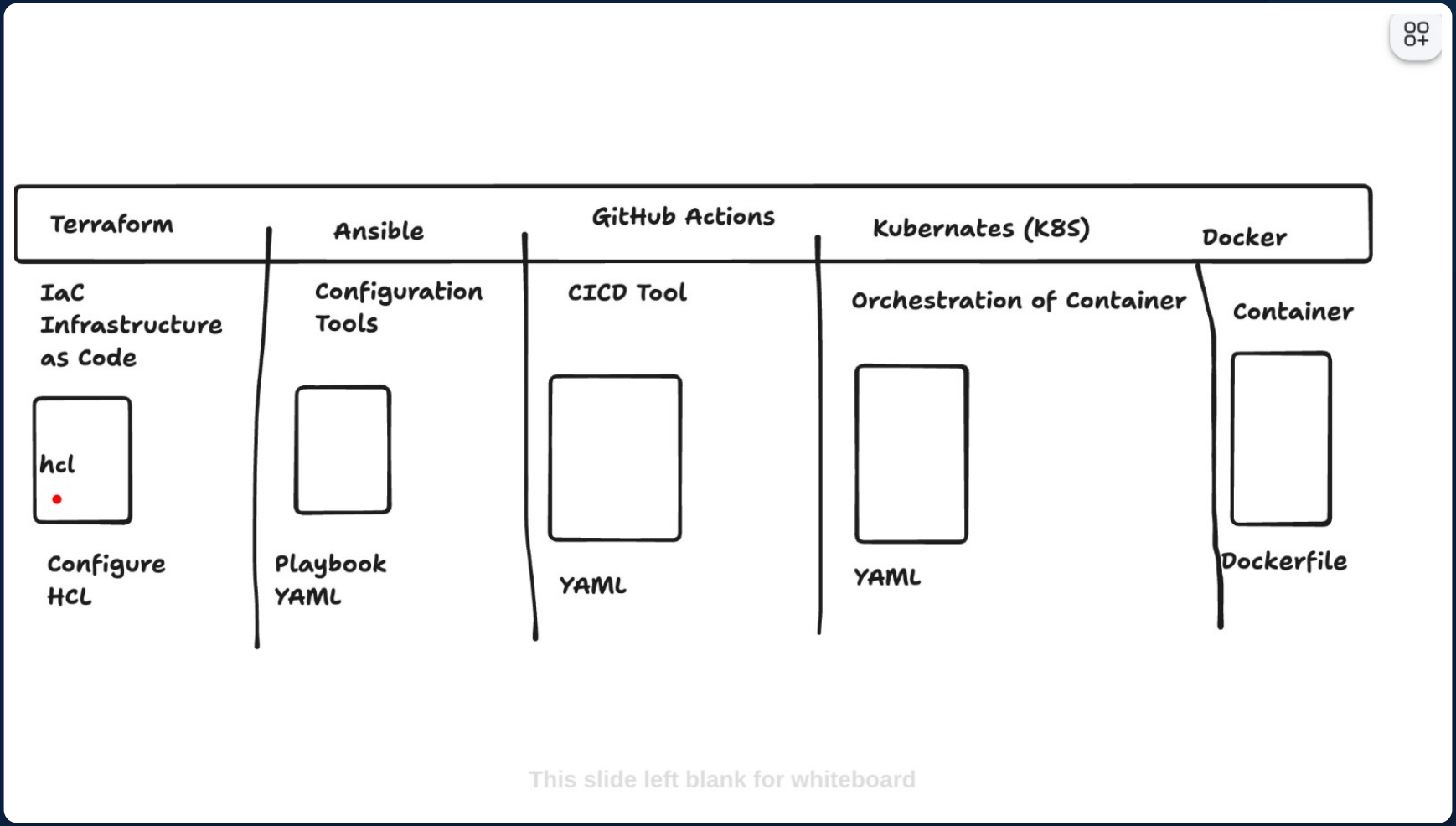
RESOURCES (TAP TO OPEN)

Docker docs: What is Docker?
https://docs.docker.com/get-started/docker-overview/

Docker Desktop documentation
https://docs.docker.com/desktop/

© 2026 cloudmessenger

Figure 9: Where Docker fits among DevOps tools



IN PLAIN WORDS

Five tools, five jobs. **Terraform** builds infrastructure from code (HCL). **Ansible** configures servers with YAML playbooks. **GitHub Actions** automates CI/CD with YAML. **Kubernetes** orchestrates many containers. **Docker** creates the containers themselves from a Dockerfile.

KEY POINTS

- Terraform: provision. Ansible: configure
- GitHub Actions: build and ship automatically
- Docker makes containers; Kubernetes manages them at scale

RESOURCES (TAP TO OPEN)

Terraform documentation
https://developer.hashicorp.com/terraform/docs

Ansible documentation
https://docs.ansible.com/

GitHub Actions documentation
https://docs.github.com/en/actions

Kubernetes documentation
https://kubernetes.io/docs/

© 2026 cloudmessenger

CHAPTER 4 OF 11

Docker Core Concepts

Dockerfile Image Container

THINK OF IT LIKE THIS

A Dockerfile is a recipe, an image is the sealed meal kit made from it, and a container is the hot meal on the table. One kit can serve many identical meals.

YOU WILL LEARN

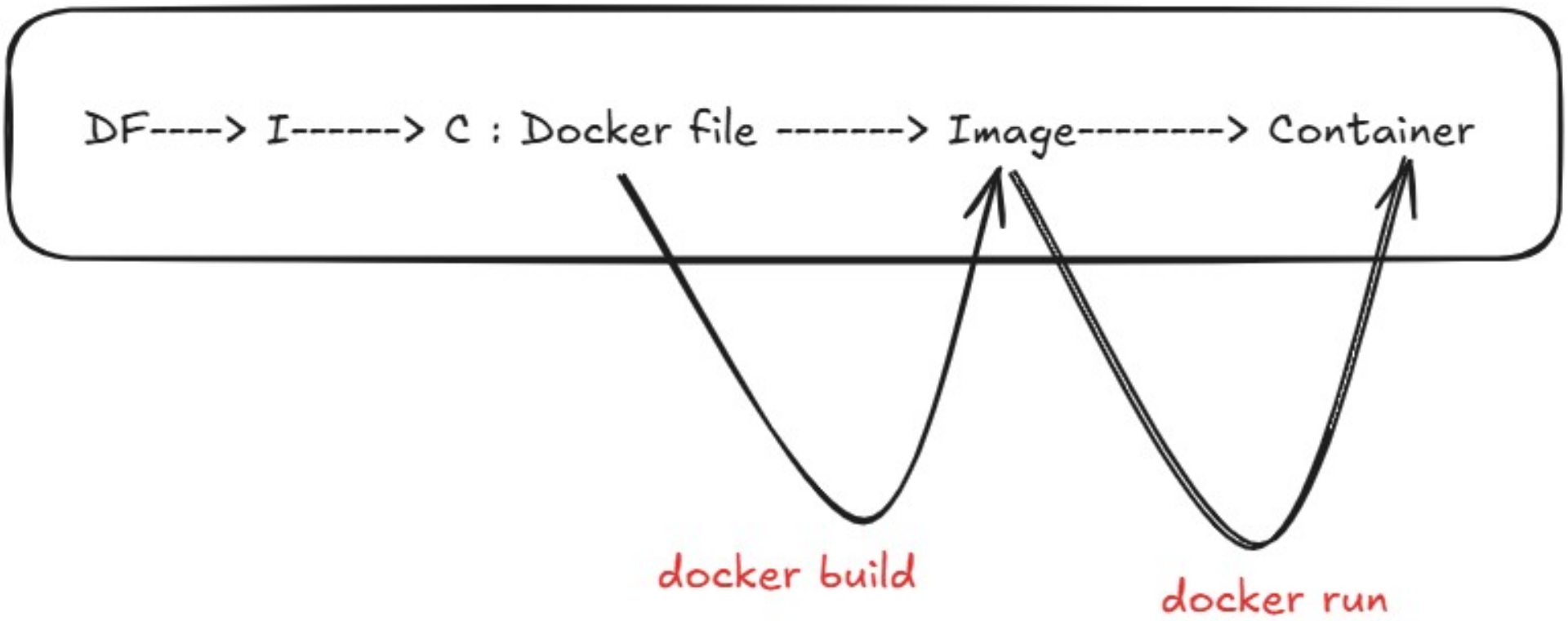
- The golden rule DF → I → C
- Docker Engine, registry and Docker Hub
- How one image travels from Dev to Test to Prod

IN THIS CHAPTER

Figure 10 The golden rule: DF → I → C (page 23)
Figure 11 Dockerfile, Image, Container: three words, three jobs (page 24)
Figure 12 Docker architecture: Engine, Image, Container, Hub (page 25)
Figure 13 Simple Docker flow: Dev, Test, Prod (page 26)
Figure 14 Two stores: GitHub for code, Docker Hub for images (page 27)

© 2026 cloudmessenger

Figure 10: The golden rule: DF I C



IN PLAIN WORDS

Dockerfile → Image → Container. The command **docker build** turns a Dockerfile into an image and **docker run** turns an image into a container. Memorise this chain and every other Docker command will find its place on it.

KEY POINTS

- DF = Dockerfile (the recipe)
- I = Image (the sealed meal kit)
- C = Container (the running meal)

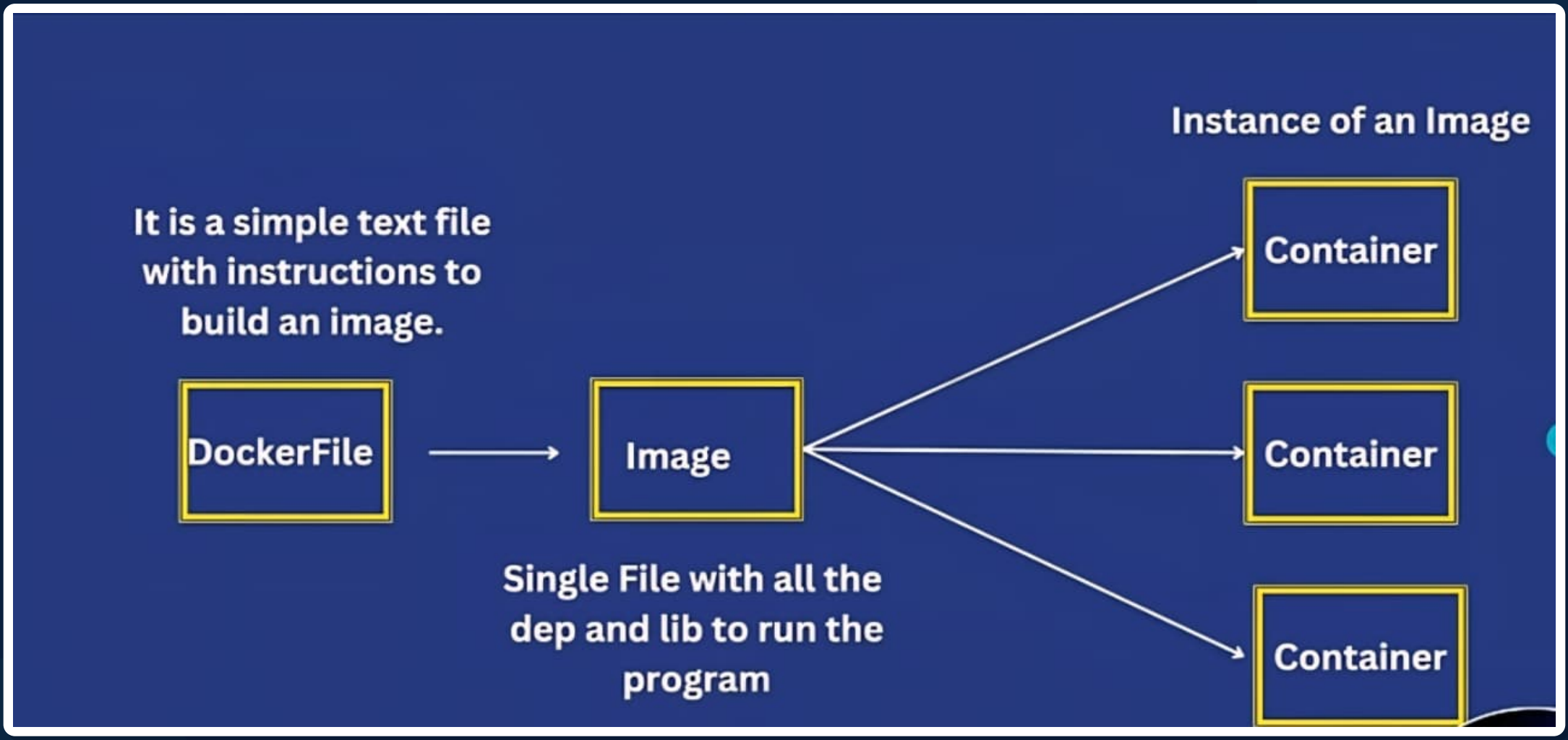
RESOURCES (TAP TO OPEN)

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

Docker docs: CLI reference
https://docs.docker.com/reference/cli/docker/

© 2026 cloudmessenger

Figure 11: Dockerfile, Image, Container: three words, three jobs



IN PLAIN WORDS

A **Dockerfile** is a simple text file of instructions. Build it once and you get an **Image**: a single file holding the program plus every library it needs. From one image you can start many identical **Containers** (instances).

KEY POINTS

- Dockerfile: instructions
- Image: the packaged blueprint
- Container: a live instance of an image

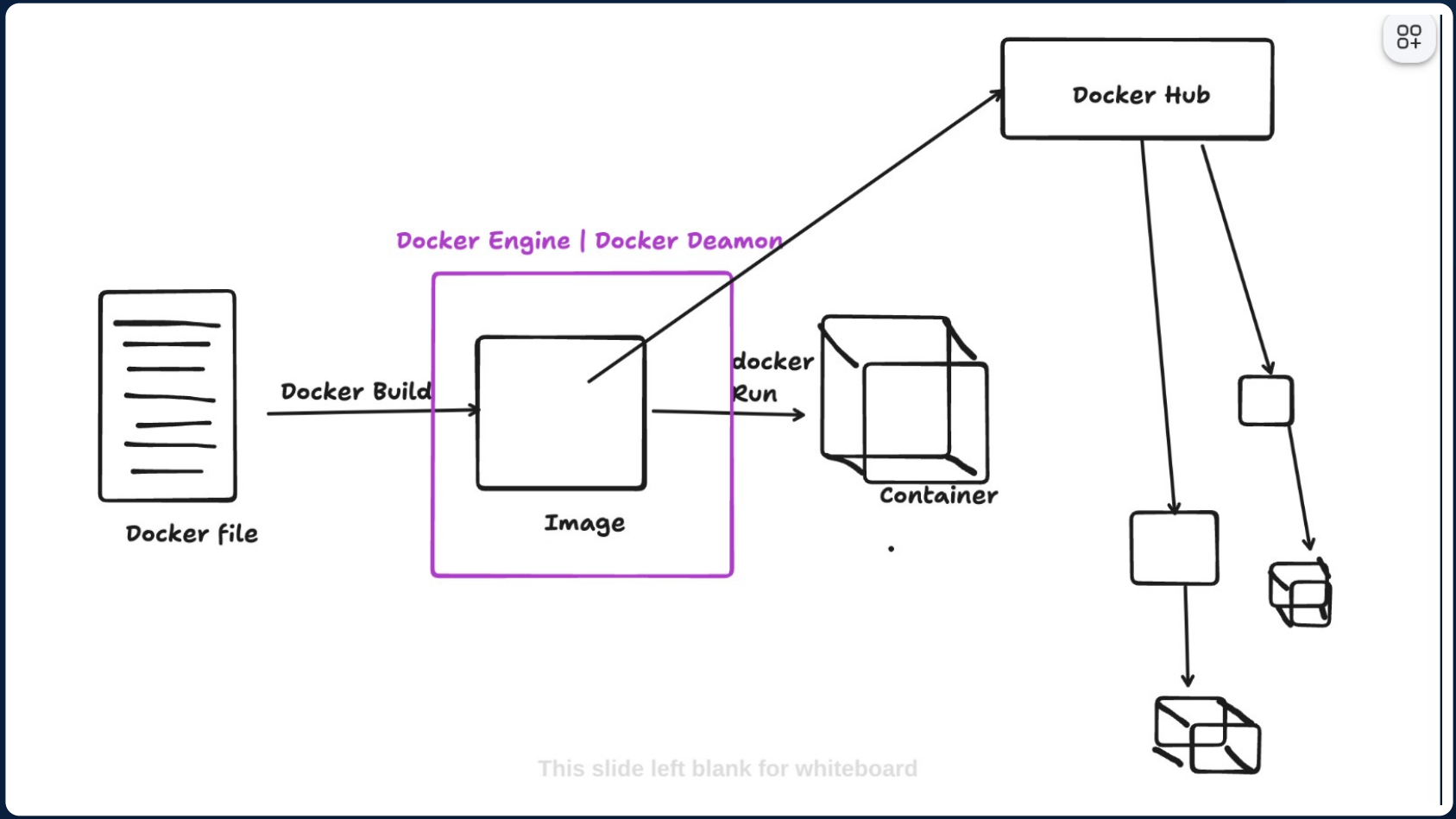
RESOURCES (TAP TO OPEN)

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

Docker docs: What is Docker?
https://docs.docker.com/get-started/docker-overview/

© 2026 cloudmessenger

Figure 12: Docker architecture: Engine, Image, Container, Hub



A second copy of this diagram (docker-architecture1.jpeg) was identical and is used only once.

IN PLAIN WORDS

The purple box is the **Docker Engine** (also called the Docker daemon, the background worker). docker build feeds a Dockerfile into it and creates an image; docker run starts a container from that image. The image can also go up to **Docker Hub**, and others pull it down to run their own containers (right side).

KEY POINTS

- The daemon does the real work; your commands are requests
- Images can be pushed to and pulled from a registry
- Many containers can come from one image

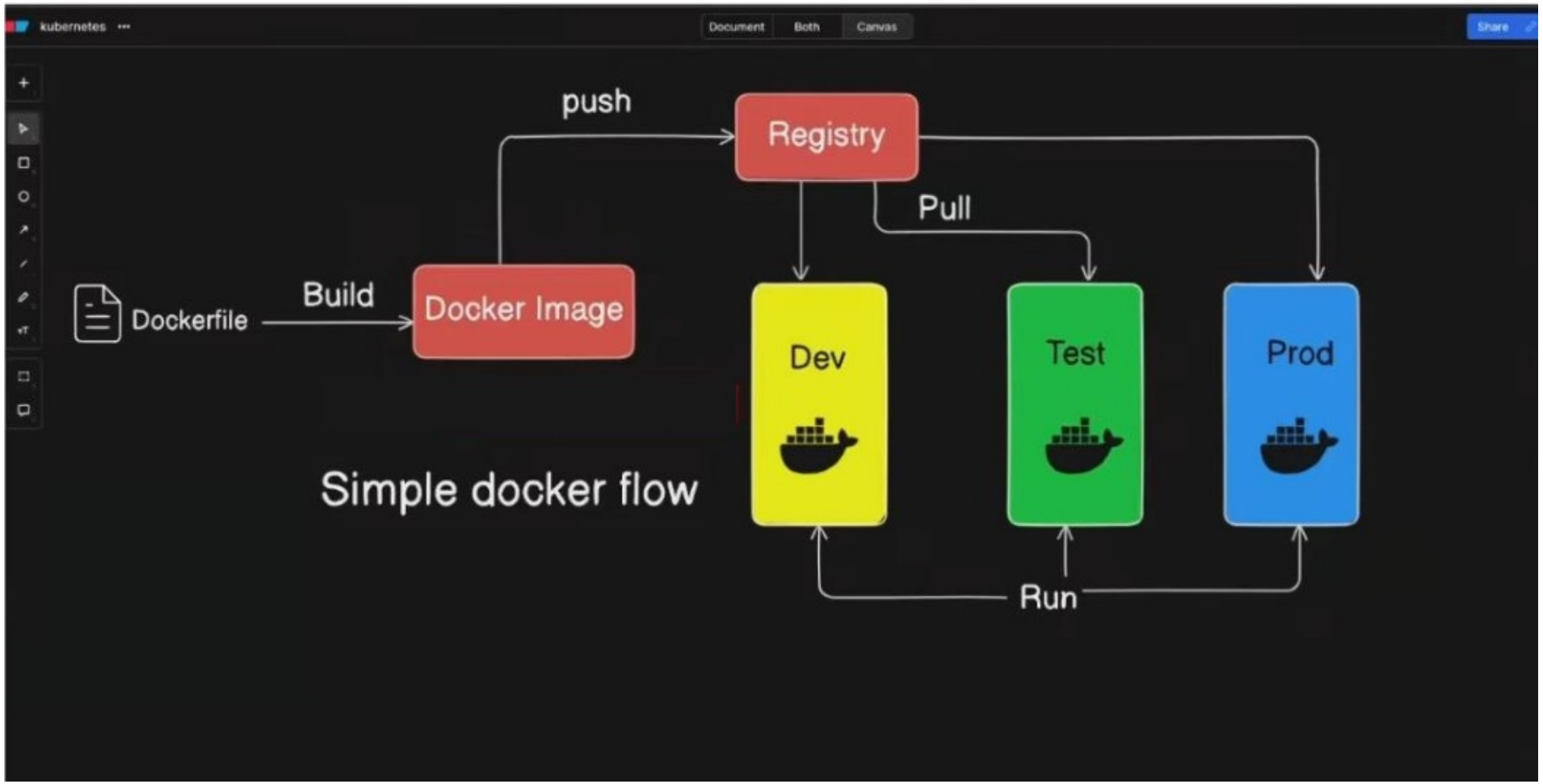
RESOURCES (TAP TO OPEN)

Docker docs: What is Docker?
https://docs.docker.com/get-started/docker-overview/

Docker Hub
https://hub.docker.com/

© 2026 cloudmessenger

Figure 13: Simple Docker flow: Dev, Test, Prod



IN PLAIN WORDS

Build once, run everywhere. Dockerfile → build → Docker Image → push to a **Registry** → pull on the Dev, Test and Prod machines and run. The very same image reaches all three environments, which ends the classic “it worked on my laptop” problem.

KEY POINTS

- Push uploads an image; pull downloads it
- Same image in Dev, Test and Prod means the same behaviour
- A registry can be Docker Hub or a private one such as Amazon ECR

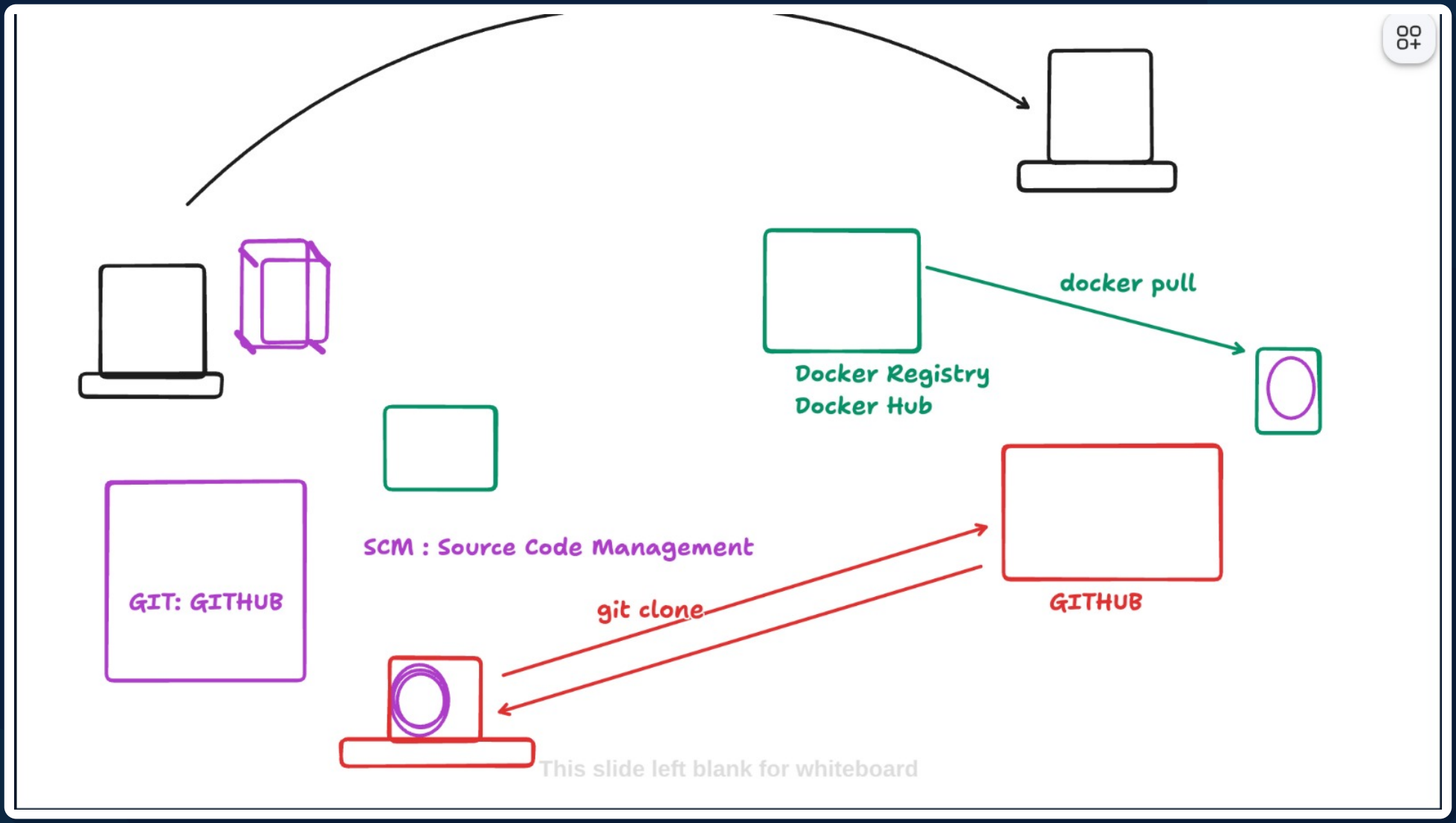
RESOURCES (TAP TO OPEN)

Docker Hub
https://hub.docker.com/

AWS: What is Amazon ECR?
https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html

© 2026 cloudmessenger

Figure 14: Two stores: GitHub for code, Docker Hub for images



IN PLAIN WORDS

GitHub (source code management) stores your code and **git clone** copies it to your machine. **Docker Registry / Docker Hub** stores ready-built images and **docker pull** copies one to your machine. Different stores, same idea: share once, download anywhere.

KEY POINTS

- git clone → get code
- docker pull → get a ready-to-run image
- SCM = Source Code Management

RESOURCES (TAP TO OPEN)

GitHub docs: Cloning a repository
https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

Docker Hub
https://hub.docker.com/

© 2026 cloudmessenger

CHAPTER 5 OF 11

Get Docker

Install it on your laptop or your cloud server

THINK OF IT LIKE THIS

Before you can cook, you need a kitchen. Docker Engine is the kitchen: install it once, then every recipe in this book works.

YOU WILL LEARN

- Docker Desktop for Windows, macOS, Linux
- Docker on Ubuntu EC2 with apt
- Docker on Amazon Linux EC2 with yum, from a real terminal history

IN THIS CHAPTER

Install guide three ways to get Docker (page 29)

Figure 15 Real terminal history: installing Docker on Amazon Linux (page 31)

© 2026 cloudmessenger

Install guide: three ways to get Docker

A. Your own laptop: Docker Desktop

Windows • macOS • Linux

STEPS

1. Download the installer for your system from the official page.
2. Run it. Windows normally uses WSL 2 and needs virtualization enabled; macOS asks you to drag Docker to Applications.
3. Open Docker Desktop and wait until it says it is running.
4. Open a terminal and check it with the commands below.

terminal

```
$ docker --version
$ docker run hello-world
```

OFFICIAL LINKS

Docker Desktop download page
https://www.docker.com/products/docker-desktop/

Docker Desktop for Windows
https://docs.docker.com/desktop/setup/install/windows-install/

Docker Desktop for Mac
https://docs.docker.com/desktop/setup/install/mac-install/

Docker Desktop for Linux
https://docs.docker.com/desktop/setup/install/linux/

B. Ubuntu server (EC2)

As in the “DockerHub to EC2” figures

STEPS

1. Connect to your instance over SSH.
2. Run the commands below, one line at a time.
3. For the newest version, follow Docker’s own repository steps instead of docker.io.

terminal

```
$ sudo apt update
$ sudo apt install docker.io -y
$ sudo systemctl start docker
$ sudo systemctl enable docker
$ docker --version
```

OFFICIAL LINKS

AWS: Connect to your EC2 instance
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect.html

Docker docs: Install on Ubuntu
https://docs.docker.com/engine/install/ubuntu/

Docker docs: Linux post-install steps
https://docs.docker.com/engine/install/linux-postinstall/

SEE IT IN THE FIGURES

Look at **Figure 18 (page 36)** for the Ubuntu command list with analogies, and **Figure 15 (page 31)** for the Amazon Linux session as typed in a real terminal.

WHICH ONE SHOULD I PICK?

- Learning on your own computer: choose A (Docker Desktop).
- Following the nginx-on-EC2 chapters: choose B if your server is Ubuntu, C if it is Amazon Linux.
- Not sure which Linux you have? Run: cat /etc/os-release

© 2026 cloudmessenger

Install guide: three ways to get Docker (continued)

C. Amazon Linux server (EC2)

As in the real terminal history

STEPS

1. Connect to your instance; the screenshot is logged in as root, so it has no sudo.
2. Install and start Docker with yum and systemctl.
3. Optional: add ec2-user to the docker group, then log out and in, to skip sudo.

terminal

```
$ sudo yum update -y
$ sudo yum install docker -y
$ sudo systemctl start docker
$ sudo systemctl status docker
$ sudo systemctl enable docker
$ docker -v
$ sudo usermod -a -G docker ec2-user
```

OFFICIAL LINKS

AWS: Install Docker on Amazon Linux 2023
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html

AWS: Connect to your EC2 instance
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect.html

CHECK THAT IT WORKED

Run the two commands below. **docker --version** prints a version number; **docker run hello-world** downloads a tiny test image and prints a welcome message.

```
$ docker --version
$ docker run hello-world
$ docker ps -a
```

COMMON MISTAKES AND FIXES

- permission denied: put sudo in front, or add your user to the docker group (see the post-install link) and log in again.
- Cannot connect to the Docker daemon: the service is not running. Start it with: sudo systemctl start docker
- Command not found: Docker did not install; re-run the install commands and read the first error line.
- Website not loading on EC2 later: open the port in the security group (links in Chapter 6).

NEXT STEP

You now have the kitchen. In **Chapter 6** you will pull the nginx image from Docker Hub and open it in your browser.

© 2026 cloudmessenger

Figure 15: Real terminal history: installing Docker on Amazon Linux

```
[root@ip-172-31-34-36 docker-demo]# history
    1  yum update -y
    2  yum update -y
    3  yum install docker -y
    4  which docker
    5  docker -v
    6  systemctl start docker
    7  systemctl status docker
    8  systemctl enable docker
    9  mkdir docker-demo
   10  cd docker-demo
   11  docker pull nginx
   12  docker run -d --name nginx-web -p 8080:80 nginx
   13  touch index.html
   14  vi index.html
   15  touch Dockerfile
   16  vi Dockerfile
   17  ls
   18  history
   19  docker build -t netflix:v1 .
   20  docker images
   21  docker run -d --name netflix-web 9090:80 netflix:v1
   22  docker run -d --name netflix-web -p 9090:80 netflix:v1
   23  history
[root@ip-172-31-34-36 docker-demo]#
```

IN PLAIN WORDS

A genuine EC2 session on Amazon Linux. Lines 1–3 update packages and install Docker with **yum**. Lines 4–8 check it, start the service and enable it on boot. Lines 9–12 make a folder, pull nginx and run it. Lines 13–18 create **index.html** and a **Dockerfile**. Lines 19–22 build image netflix:v1 and run it on port 9090. Line 21 forgot **-p**, so Docker read *9090:80* as an image name and refused; line 22 fixes it.

KEY POINTS

- systemctl enable docker → start Docker at every reboot
- Classic mistake: forgetting -p before host:container
- docker images proves the build worked

RESOURCES (TAP TO OPEN)

AWS: Install Docker on Amazon Linux 2023
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html

Docker docs: Install Docker Engine
https://docs.docker.com/engine/install/

AWS: Connect to your EC2 instance
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect.html

© 2026 cloudmessenger

CHAPTER 6 OF 11

Your First Container

Pull nginx, run it, open it in a browser

THINK OF IT LIKE THIS

Docker Hub is the Play Store for server software. docker pull is Install, docker run is Open.

YOU WILL LEARN

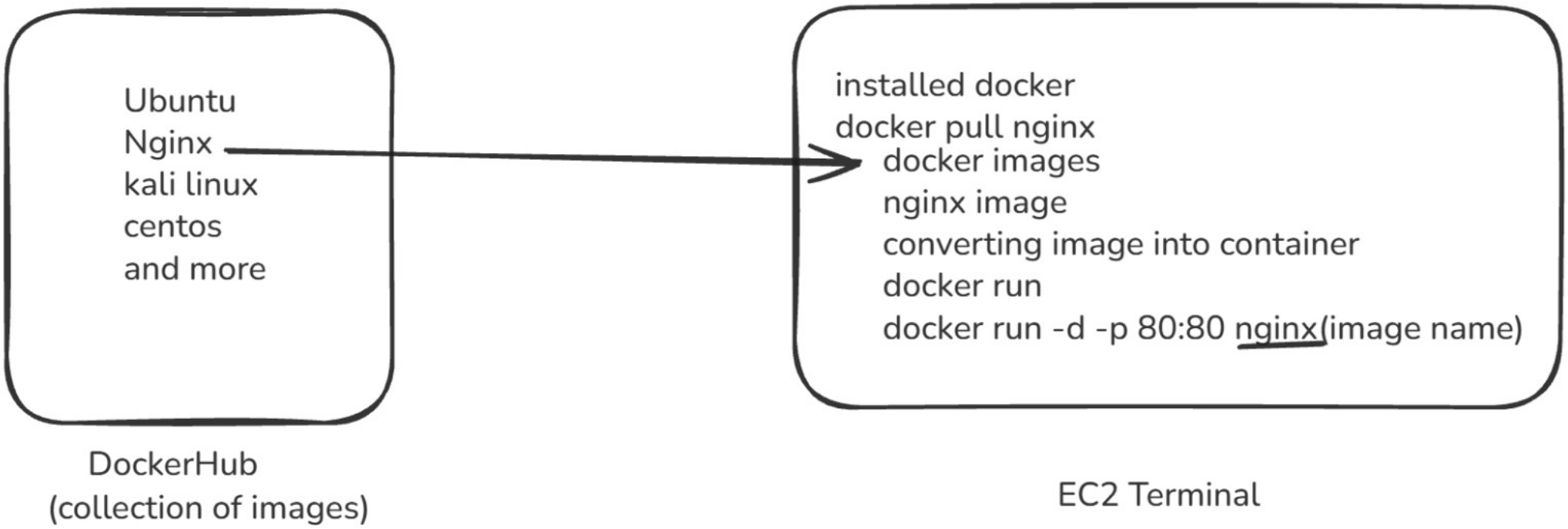
- docker pull, docker images, docker run
- What -d and -p 80:80 really do
- Opening the port in the AWS security group

IN THIS CHAPTER

Figure 16 The plan on one whiteboard: Docker Hub to EC2 (page 33)
Figure 17 From DockerHub to EC2: the 7 steps (page 34)
Figure 18 Deploy Nginx on EC2: the complete step-by-step (page 36)
Figure 19 Phases 5–7: from stock nginx to your own site (page 38)

© 2026 cloudmessenger

Figure 16: The plan on one whiteboard: Docker Hub to EC2



IN PLAIN WORDS

The left box is **Docker Hub**, a collection of images (Ubuntu, nginx, Kali Linux, CentOS and more). The right box is your EC2 terminal. The arrow shows nginx travelling across with **docker pull**. Then you list it, convert it to a container with docker run, and publish it with **docker run -d -p 80:80 nginx**.

KEY POINTS

- Order: install → pull → images → run → publish
- nginx is the image name at the end of the command
- Image first, container second

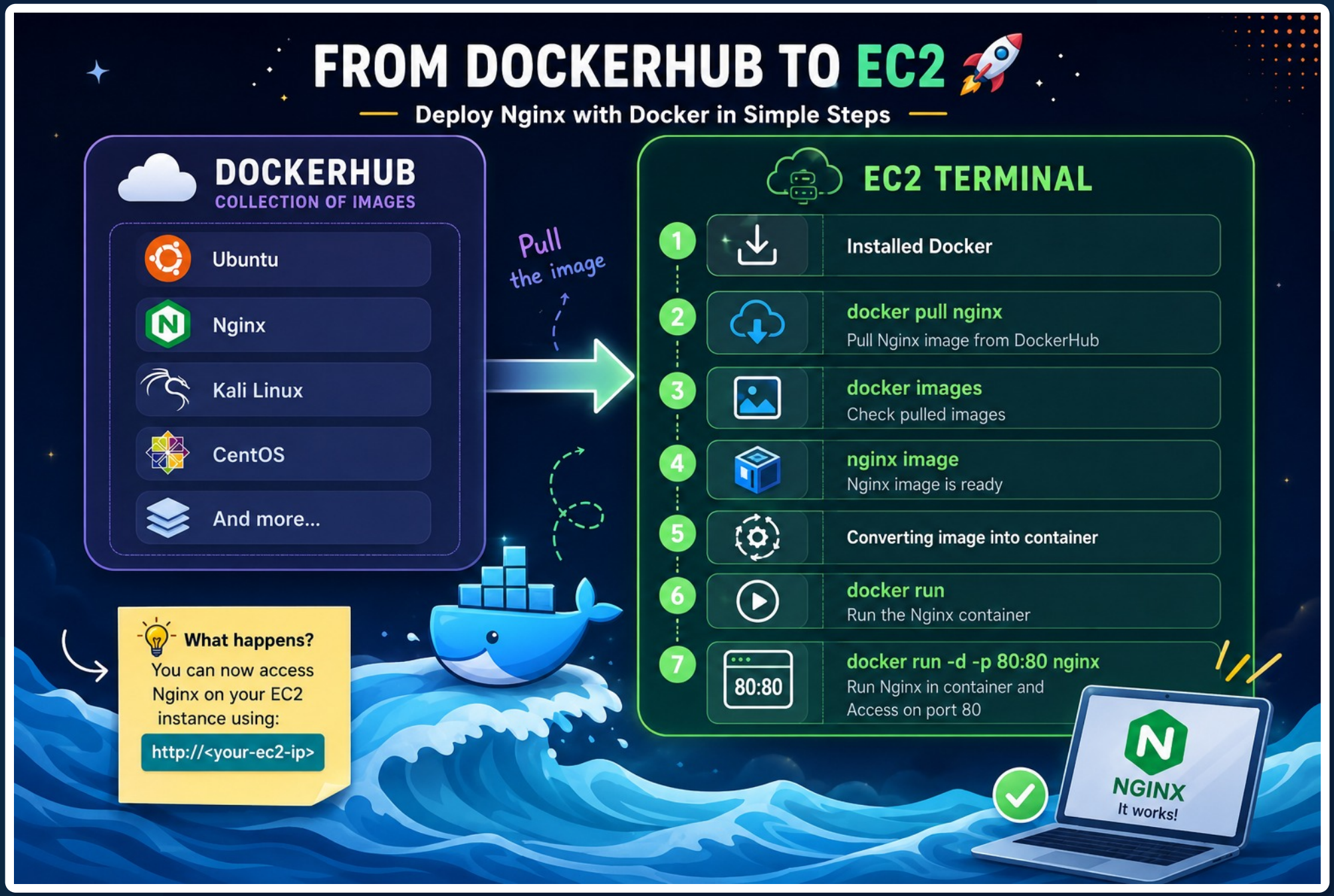
RESOURCES (TAP TO OPEN)

Docker Hub
https://hub.docker.com/

Docker Hub: official nginx image
https://hub.docker.com/_/nginx

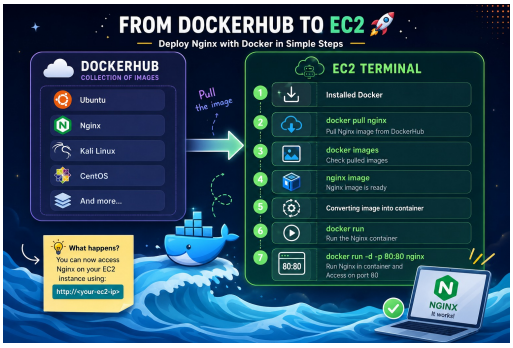
© 2026 cloudmessenger

Figure 17: From DockerHub to EC2: the 7 steps



© 2026 cloudmessenger

Figure 17: From DockerHub to EC2: the 7 steps



tap to see the full figure

IN PLAIN WORDS

The same journey as seven numbered steps: (1) install Docker, (2) **docker pull nginx**, (3) **docker images** to check, (4) the nginx image is ready, (5) Docker converts the image into a container, (6) **docker run**, (7) **docker run -d -p 80:80 nginx**. Open *http://<your-ec2-ip>* and you see “It works!”.

KEY POINTS

- -d runs it in the background
- -p 80:80 connects EC2 port 80 to container port 80
- Steps 4 and 5 describe what happens; they are not commands

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

AWS: Install Docker on Amazon Linux 2023
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html

Docker Hub: official nginx image
https://hub.docker.com/_/nginx

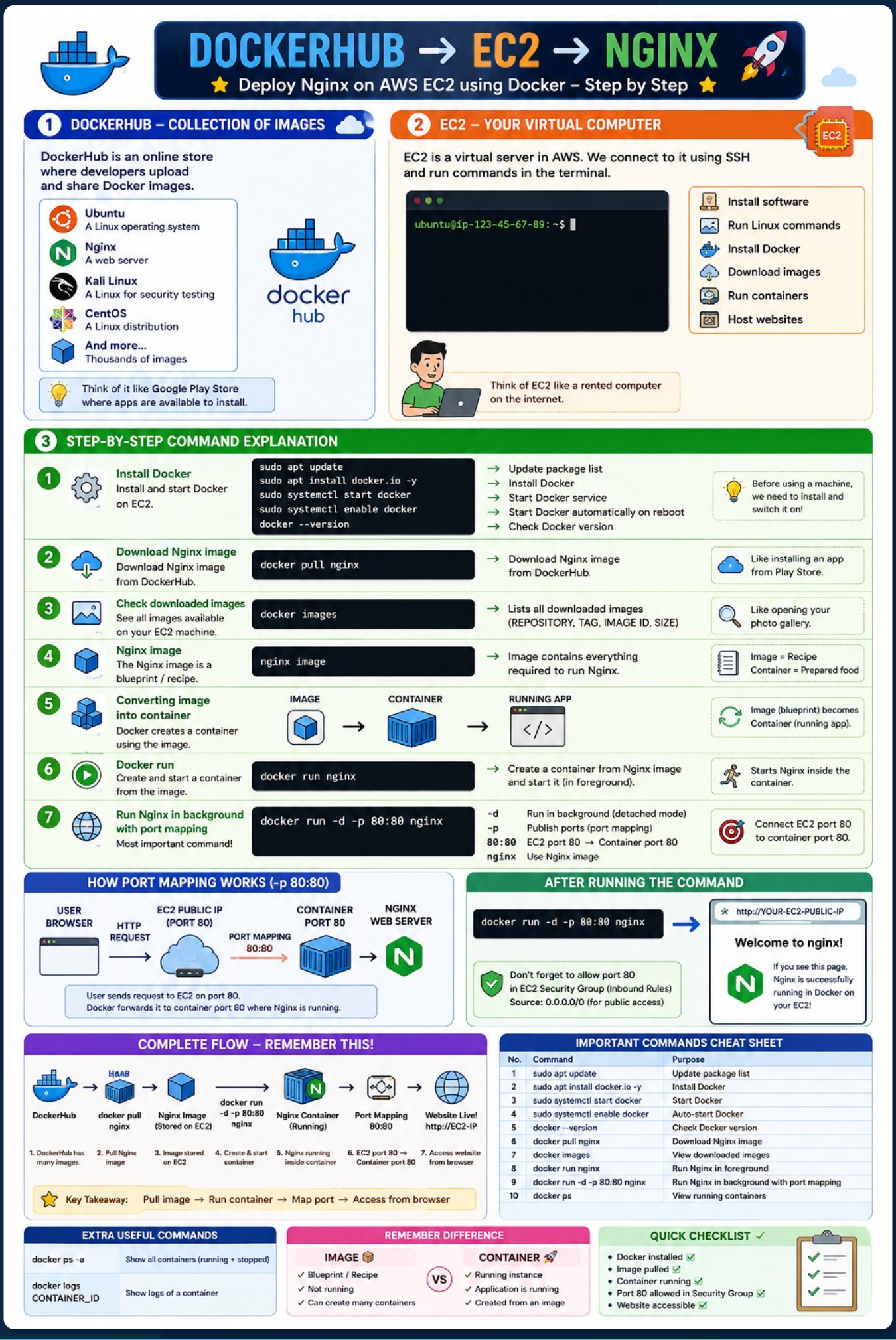
AWS: Security group rules (open port 80)
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-rules-reference.html

CHAPTER IDEA

Docker Hub is the Play Store for server software. docker pull is Install, docker run is Open.

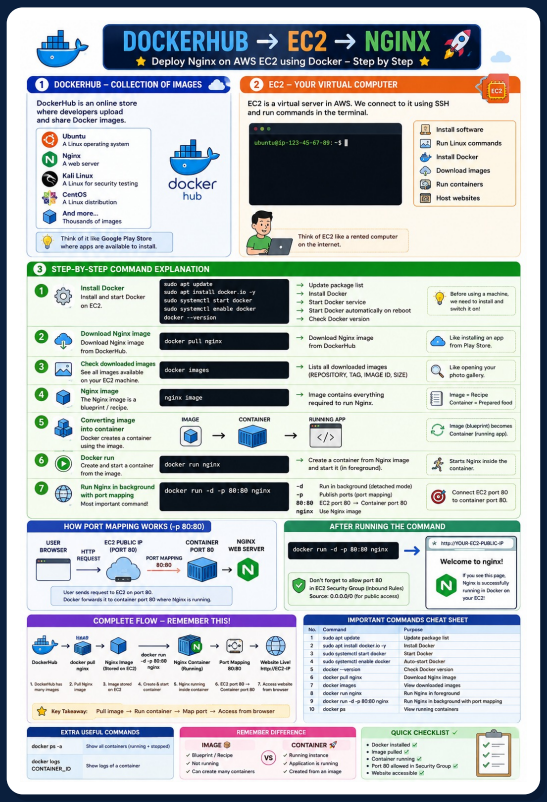
© 2026 cloudmessenger

Figure 18: Deploy Nginx on EC2: the complete step-by-step



© 2026 cloudmessenger

Figure 18: Deploy Nginx on EC2: the complete step-by-step



tap to see the full figure

IN PLAIN WORDS

The deluxe version, with analogies: Docker Hub is the Play Store, EC2 is a rented computer, an image is a recipe and a container is the cooked food. It gives copy-ready Ubuntu commands, explains how **port mapping** carries a browser request from EC2 port 80 to container port 80, and ends with a checklist and cheat sheet. Do not forget to allow **port 80** in the EC2 Security Group.

KEY POINTS

- Install: sudo apt update, then sudo apt install docker.io -y
- Remember: pull image → run container → map port → open browser
- No page loading? Check the security group inbound rules first

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Install on Ubuntu
https://docs.docker.com/engine/install/ubuntu/

AWS: Security group rules (open port 80)
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-rules-reference.html

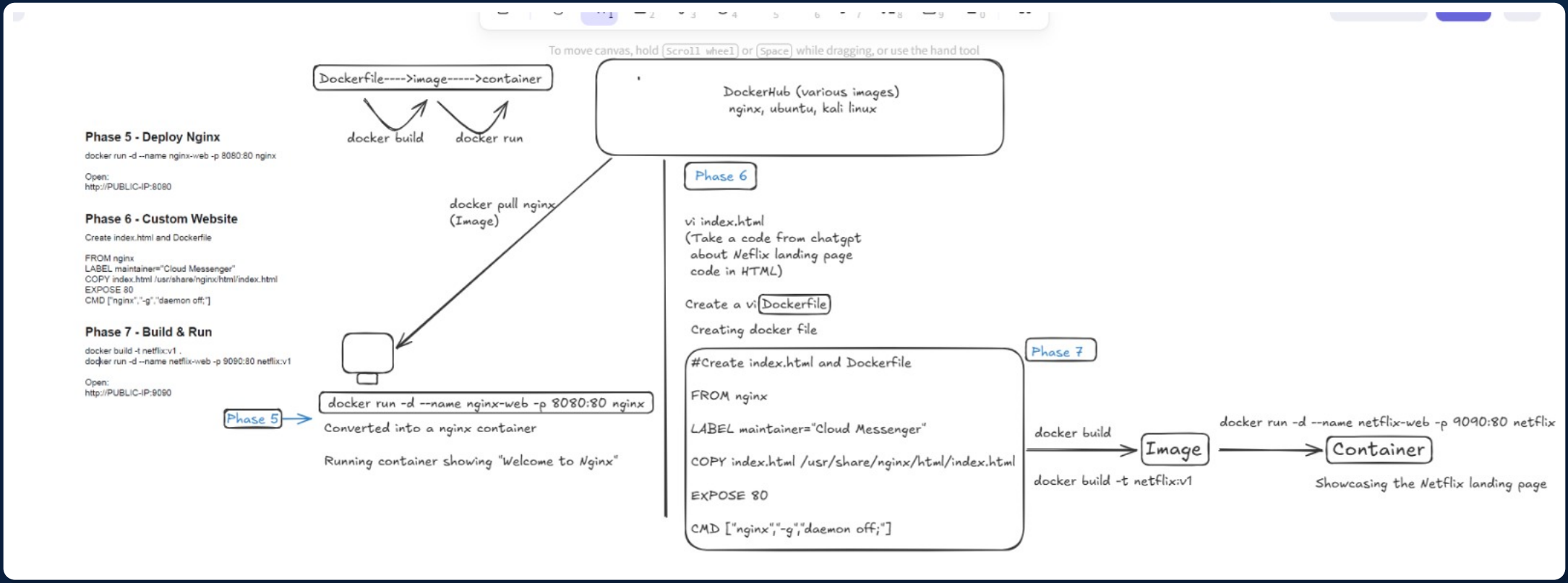
Docker Hub: official nginx image
https://hub.docker.com/_/nginx

CHAPTER IDEA

Docker Hub is the Play Store for server software. docker pull is Install, docker run is Open.

© 2026 cloudmessenger

Figure 19: Phases 5–7: from stock nginx to your own site



IN PLAIN WORDS

Phase 5: run stock nginx on port 8080 with **docker run -d --name nginx-web -p 8080:80 nginx** and open http://PUBLIC-IP:8080 to see “Welcome to nginx”. **Phase 6:** write your own index.html (a Netflix-style landing page) and a Dockerfile. **Phase 7: docker build -t netflix:v1 .** then **docker run -d --name netflix-web -p 9090:80 netflix:v1** and open :9090.

KEY POINTS

- --name gives the container a friendly name
- Different host ports (8080, 9090) let several sites run side by side
- The trailing dot in docker build means “use this folder”

RESOURCES (TAP TO OPEN)

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

Docker Hub: official nginx image
https://hub.docker.com/_/nginx

AWS: Security group rules (open port 80)
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-rules-reference.html

© 2026 cloudmessenger

CHAPTER 7 OF 11

The Dockerfile

Write your own recipe

THINK OF IT LIKE THIS

Instead of using the stock meal kit, you add your own sauce: your own index.html baked into the nginx image.

YOU WILL LEARN

- The five core instructions and all fifteen
- How COPY places your file inside the image
- Building and running your own website image

IN THIS CHAPTER

Figure 20 Dockerfile Explained: five core instructions (page 40)
Figure 21 Dockerfile line by line: all 15 instructions (page 42)
Figure 22 How COPY index.html works (page 44)
Figure 23 Cloud Messenger: build your own nginx website image (page 46)
Figure 24 From code to container: a streaming-site landing page (page 48)

© 2026 cloudmessenger

Figure 20: Dockerfile Explained: five core instructions

DOCKERFILE EXPLAINED

Understand Every Instruction with Real Examples

Example Dockerfile

```
FROM nginx
LABEL maintainer="Cloud Messenger"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx","-g","daemon off;"]
```

WHAT ISA DOCKERFILE?

- A Dockerfile is a text file that contains instructions to build a Docker image.
- It is like a recipe 🤯 for creating a Docker image.

DOCKERFILE WORKFLOW

Dockerfile → docker build → Docker Image → docker run → Running Container

INSTRUCTION	PURPOSE	EXAMPLE	VISUAL EXPLANATION	KEY POINTS
1 FROM	Sets the base image to start building on.	`FROM nginx`	Pull base image from Docker Hub: Docker Hub → nginx Base Image	• Mandatory first instruction • Can be any official image like nginx, ubuntu, python • All subsequent instructions build on top of it
2 LABEL	Adds metadata information about the image.	`LABEL maintainer="Cloud Messenger"`	Stores information in image: Maintainer: Cloud Messenger, Other info: Version, Project, etc.	• Helps in identification • Visible in inspection • Better than MAINTAINER (older version)
3 COPY	Copies files or directories from local system into the image.	`COPY index.html /usr/share/nginx/html/index.html`	Copies file from local machine to image: Local Machine, index.html → Image Filesystem /usr/share/nginx/html/index.html	• Source is local machine • Destination is inside image • Used for website files, configs, apps, etc.
4 EXPOSE	Informs Docker that the container listens on the specified network port.	`EXPOSE 80`	Documents the port used by application: Container Port 80	• Only documentation • Does NOT publish the port • Use -p while running container to publish
5 CMD	Provides default command to run when container starts.	`CMD ["nginx","-g", "daemon off;"]`	Runs command when container starts: Container Starts → `nginx -g "daemon off;`	• Only one CMD in Dockerfile • Can be overridden while running container • Keeps main process in foreground

BUILD PROCESS

1. Write Dockerfile
2. `docker build -t mywebsite:v1 .`
3. Docker Image Created (mywebsite:v1)
4. `docker run -d -p 9090:80 mywebsite:v1`
5. Access in Browser http://<server-ip>:9090 ✅

EXAMPLE PROJECT (NGINX WEBSITE)

Project Structure:

```
mywebsite/
├── Dockerfile
└── index.html
```

index.html:

```
<h1>Welcome
 Cloud Messenger 🚀 </h1>
<p>Docker Training Project</p>
```

Dockerfile:

```
FROM nginx
LABEL maintainer="Cloud Messenger"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx","-g","daemon off;"]
```

Run & Access: Browser → 9090 → Docker Host → 80 → Nginx Container

Access: http://<server-ip>:9090

DOCKERFILE INSTRUCTIONS SUMMARY

Instruction	Purpose
FROM	Base image for building
LABEL	Metadata / Information
COPY	Copy files from local to image
ADD	Copy + Extract + Download
EXPOSE	Declare container port
WORKDIR	Set working directory
RUN	Run commands during build
CMD	Default command on container start
ENTRYPOINT	Main command for container
ENV	Set environment variables
VOLUME	Create mount point for volumes
USER	Set user for running container

KEY TAKEAWAYS

- Dockerfile builds images in layers.
- Better layering = smaller image size.
- Use official base images.
- Keep containers simple and single-purpose.

COMMON DOCKER COMMANDS

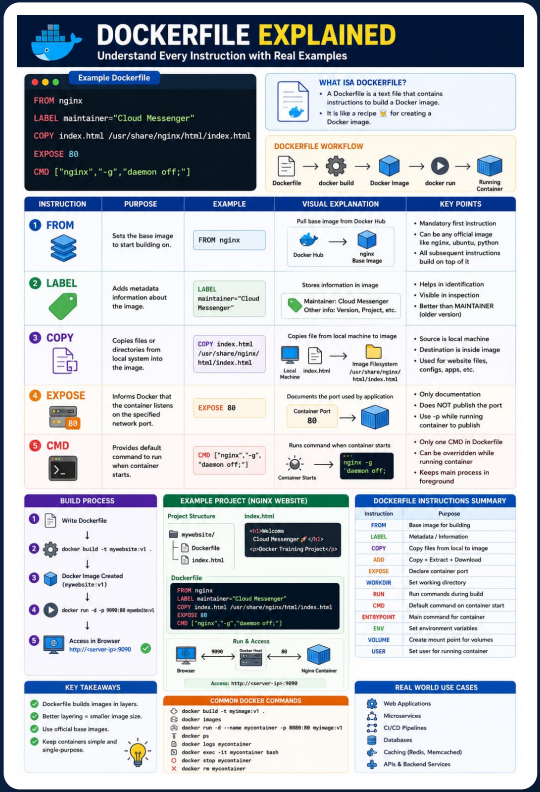
```
docker build -t myimage:v1 .
docker images
docker run -d --name mycontainer -p 8080:80 myimage:v1
docker ps
docker logs mycontainer
docker exec -it mycontainer bash
docker stop mycontainer
docker rm mycontainer
```

REAL WORLD USE CASES

- Web Applications
- Microservices
- CI/CD Pipelines
- Databases
- Caching (Redis, Memcached)
- APIs & Backend Services

© 2026 cloudmessenger

Figure 20: Dockerfile Explained: five core instructions



tap to see the full figure

IN PLAIN WORDS

A Dockerfile is a recipe. **FROM** picks the base image. **LABEL** adds a name tag. **COPY** puts your files into the image. **EXPOSE** documents the port (it does *not* open it). **CMD** says what to run when the container starts. The lower half walks through build, run and browse at port 9090, plus a summary table of other instructions.

KEY POINTS

- FROM must come first
- EXPOSE is documentation; -p publishes the port
- Fewer, simpler layers mean smaller images

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

Docker docs: Building best practices
https://docs.docker.com/build/building/best-practices/

CHAPTER IDEA

Instead of using the stock meal kit, you add your own sauce: your own index.html baked into the nginx image.

© 2026 cloudmessenger

Figure 21: Dockerfile line by line: all 15 instructions

Dockerfile Line-by-Line

① FROM – Starts the Docker image.
– Downloads the specified base image if not available locally.
– Every Dockerfile begins with FROM.

```
FROM ubuntu:22.04
```

② LABEL – Adds metadata to the image.
– Useful for author, version, project, description, etc.

```
LABEL maintainer="anusha@example.com"
```

③ ENV – Defines environment variables.
– Available during build and when the container runs.

```
ENV APP_ENV=production
```

④ WORKDIR – Sets the current working directory.
– Automatically creates it if it doesn't exist.
– All following commands execute from here.

```
WORKDIR /app
```

⑤ COPY – Copies files from your local machine into the image.
– Simple file copy.
– Doesn't support remote URLs.

Example:

```
COPY app.py /app/
```

⑥ ADD – Similar to COPY.
– Can automatically extract local tar archives.
– Can download files from URLs. (less commonly used)

Example:

```
ADD app.tar.gz /app/
```

⑦ RUN – Executes commands while building the image.
– Creates a new image layer.
– Used for installing software and dependencies.

```
RUN apt-get update && apt-get install -y nginx
```

⑧ EXPOSE – Documents which port the application listens on.
– Does NOT publish the port automatically.

```
EXPOSE 8080
```

Run with: `docker run -p 8080:8080 image-name`

⑨ VOLUME – Creates a persistent storage location.
– Data survives container recreation.
– Useful for databases, logs, uploads.

```
VOLUME /data
```

⑩ USER – Runs subsequent commands as a non-root user.
– Improves container security.

```
USER appuser
```

⑪ HEALTHCHECK – Checks if the application is healthy.
– Docker reports: Healthy / Unhealthy / Starting

```
HEALTHCHECK CMD curl --fail http://localhost:8080 || exit 1
```

⑫ CMD – Default command executed when the container starts.
– Can be overridden by docker run.

```
CMD ["python", "app.py"]
```

Example: `docker run image-name bash`

⑬ ENTRYPOINT – Defines the main executable.
– Cannot be easily replaced.
– Command-line arguments are appended.

```
ENTRYPOINT ["python"]
CMD ["app.py"]
```

Running: `docker run myapp test.py`
Executes: `python test.py`

⑭ ARG – Build-time variable.
– Available only while building the image.

```
ARG VERSION=1.0
```

Example: `docker build --build-arg VERSION=2.0 .`

⑮ ONBUILD – Executes only when this image is used as a base image by another Dockerfile.
– Useful for creating reusable base images.

```
ONBUILD COPY . /app
```

Anusha Janjirla

✱ Dockerfile Instruction Summary

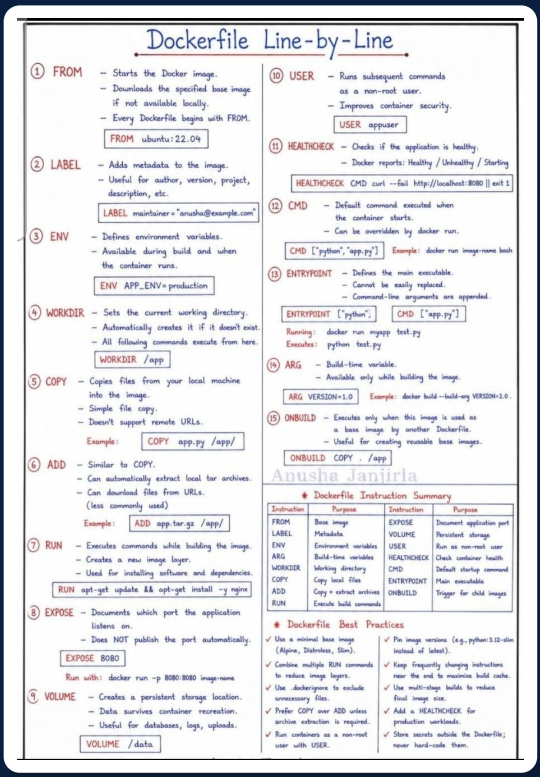
Instruction	Purpose	Instruction	Purpose
FROM	Base image	EXPOSE	Document application port
LABEL	Metadata	VOLUME	Persistent storage
ENV	Environment variables	USER	Run as non-root user
ARG	Build-time variables	HEALTHCHECK	Check container health
WORKDIR	Working directory	CMD	Default startup command
COPY	Copy local files	ENTRYPOINT	Main executable
ADD	Copy + extract archives	ONBUILD	Trigger for child images
RUN	Execute build commands		

✱ Dockerfile Best Practices

- ✓ Use a minimal base image (Alpine, Distroless, Slim).
- ✓ Combine multiple RUN commands to reduce image layers.
- ✓ Use .dockerignore to exclude unnecessary files.
- ✓ Prefer COPY over ADD unless archive extraction is required.
- ✓ Run containers as a non-root user with USER.
- ✓ Pin image versions (e.g., python:3.12-slim instead of latest).
- ✓ Keep frequently changing instructions near the end to maximize build cache.
- ✓ Use multi-stage builds to reduce final image size.
- ✓ Add a HEALTHCHECK for production workloads.
- ✓ Store secrets outside the Dockerfile; never hard-code them.

© 2026 cloudmessenger

Figure 21: Dockerfile line by line: all 15 instructions



tap to see the full figure

IN PLAIN WORDS

Hand-written notes on FROM, LABEL, ENV, WORKDIR, COPY, ADD, RUN, EXPOSE, VOLUME, USER, HEALTHCHECK, CMD, ENTRYPOINT, ARG and ONBUILD, plus best practices: use small base images, combine RUN lines, pin versions and never hard-code secrets.

KEY POINTS

- RUN happens at build time; CMD happens at run time
- Prefer COPY over ADD unless you need extraction
- Never place passwords inside a Dockerfile

GOOD TO KNOW

Original notes by Anusha Janjirla (watermark visible on the image); credited here as a third-party source.

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

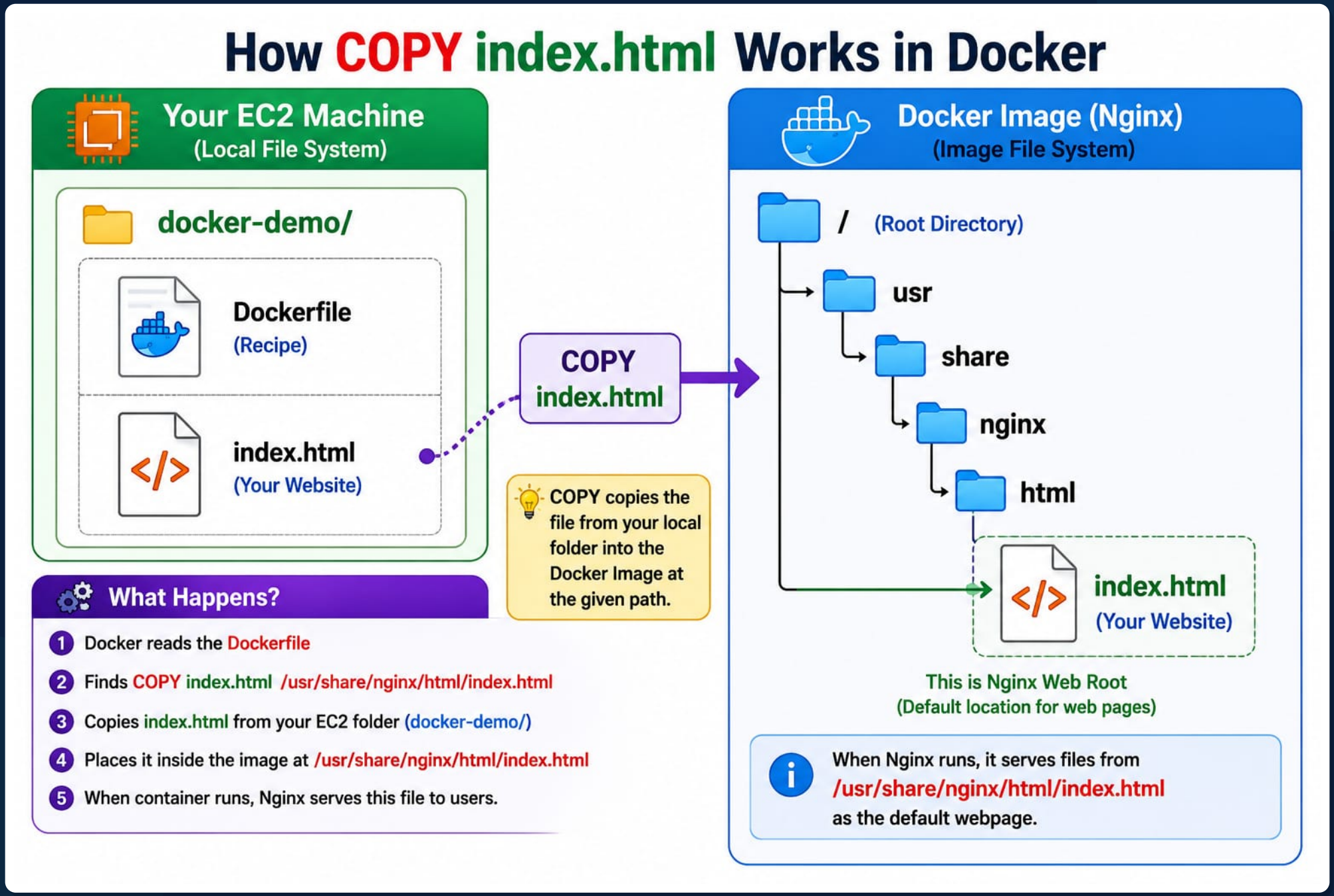
Docker docs: Building best practices
https://docs.docker.com/build/building/best-practices/

CHAPTER IDEA

Instead of using the stock meal kit, you add your own sauce: your own index.html baked into the nginx image.

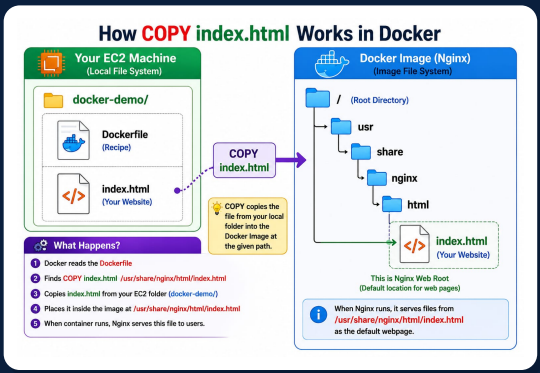
© 2026 cloudmessenger

Figure 22: How COPY index.html works



© 2026 cloudmessenger

Figure 22: How COPY index.html works



tap to see the full figure

IN PLAIN WORDS

On the left is your EC2 folder **docker-demo/** with the Dockerfile and index.html. **COPY** places index.html inside the image at **/usr/share/nginx/html/index.html**, nginx's web root. When the container runs, nginx serves that file as the homepage.

KEY POINTS

- Source path is on your machine; destination is inside the image
- nginx serves files from /usr/share/nginx/html
- Change the HTML, rebuild the image, see the new site

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

Docker Hub: official nginx image
https://hub.docker.com/_/nginx

CHAPTER IDEA

Instead of using the stock meal kit, you add your own sauce: your own index.html baked into the nginx image.

© 2026 cloudmessenger

Figure 23: Cloud Messenger: build your own nginx website image

Dockerfile Explained

Build a simple NGINX web server with your own index.html

From code → to Docker image → to running container → to website

Same code
Same container
Runs Everywhere

Build → Ship → Run → Anywhere

1 Project Structure

Create a folder with two files.

```
nginx-docker/
├── index.html
└── Dockerfile
```

Simple. Just 2 files!

2 index.html

A simple web page.

Index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Cloud Messenger</title>
  </head>
  <body>
    <h1>Welcome to Cloud
      Messenger 🚀</h1>
    <p>This website is running
      inside a Docker container.</p>
  </body>
</html>
```

3 Dockerfile

The recipe for your image.

Dockerfile — dockerfile

```
FROM nginx
LABEL maintainer="Cloud Messenger"
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
CMD ["nginx","-g","daemon off;"]
```

Each line has a purpose (explained below)

4 Line-by-Line Explanation

1	`FROM nginx`	Use the official NGINX image as the base. It already has Linux + NGINX installed.	Like getting a ready-made house and customizing it.
2	`LABEL maintainer="Cloud Messenger"`	Adds metadata to the image. Tells who maintains it.	Just a name tag (doesn't change how it works).
3	`COPY index.html /usr/share/nginx/html/index.html`	Copies your index.html file into the NGINX web directory inside the image.	Puts your website in NGINX's web root.
4	`EXPOSE 80`	Tells Docker that the container will listen on port 80.	Documentation only. Does NOT publish the port.
5	`CMD ["nginx","-g","daemon off;"]`	Starts NGINX in the foreground when the container runs.	Keeps NGINX running (and container alive).

5 Build the Image

Run this command in the same folder.

```
$ docker build -t cloud-messenger-nginx .
```

Creates a Docker image from your Dockerfile.

6 Check the Image

Verify the image is created.

```
$ docker images
REPOSITORY              TAG     IMAGE ID    CREATED
cloud-messenger-nginx   latest  abc123...   ...
nginx                   latest  xyz456...   ...
```

You should see your image in the list.

7 Run the Container

Start a container and map the port.

```
$ docker run -d -p 8080:80 \
  --name cloud-messenger-web \
  cloud-messenger-nginx
```

- `-d` Run in background
- `-p 8080:80` Map host port 8080 to container port 80
- `--name` Give a name to the container

8 Access in Browser

Open in your browser.

http://localhost:8080

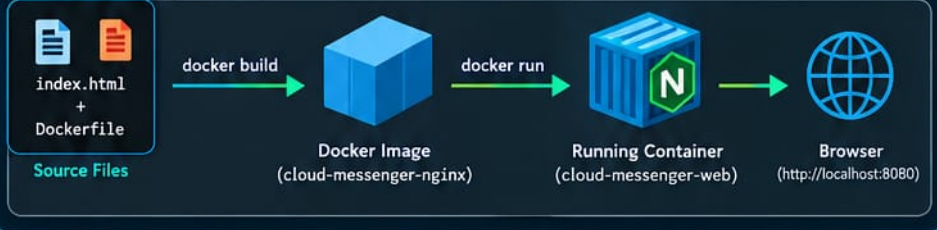
Welcome to Cloud Messenger 🚀

This website is running inside a Docker container.

Your website is live!

9 Complete Flow

From files to website.



Key Concepts

Dockerfile	Recipe (instructions)
Image	Built package (template)
Container	Running instance
EXPOSE	Documentation, not port publishing
Port Mapping	`-p 8080:80` (host:container)
CMD	Runs main process
Base Image	Starting point (e.g., nginx)

Visual Analogy



"Write once, build anywhere, run everywhere."
— The Power of Docker

Why This Matters?

- Same environment everywhere
- No manual installation
- Portable and lightweight
- Works on your laptop, servers, or cloud
- Foundation for DevOps, CI/CD, Kubernetes

Cloud Messenger

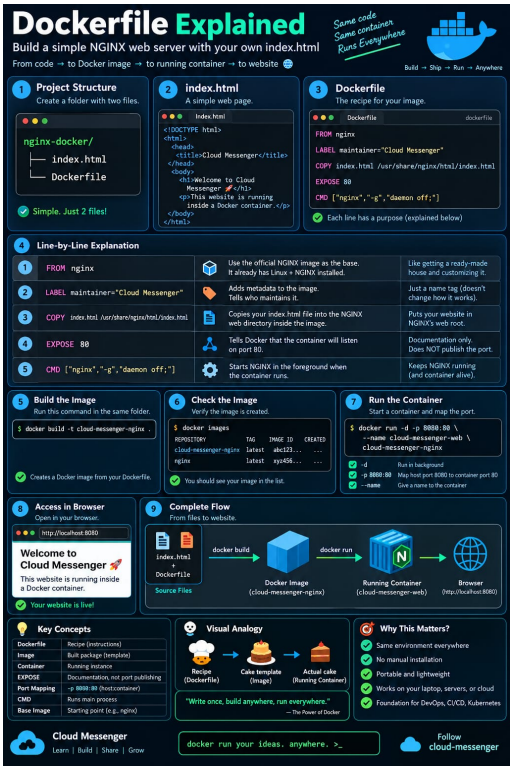
Learn | Build | Share | Grow

`docker run your ideas. anywhere. >_`

Follow cloud-messenger

© 2026 cloudmessenger

Figure 23: Cloud Messenger: build your own nginx website image



tap to see the full figure

IN PLAIN WORDS

Nine panels from folder to browser: a project folder with two files, the index.html, the Dockerfile, a line-by-line explanation, **docker build -t cloud-messenger-nginx .**, **docker images**, **docker run -d -p 8080:80 --name cloud-messenger-web cloud-messenger-nginx**, opening localhost:8080, and the complete flow. The cake analogy: recipe, cake template, actual cake.

KEY POINTS

- Just two files make a website image
- Dockerfile → build → image → run → browser
- Write once, build anywhere, run everywhere

GOOD TO KNOW

This is the original Cloud Messenger design that inspires the style of this handbook.

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

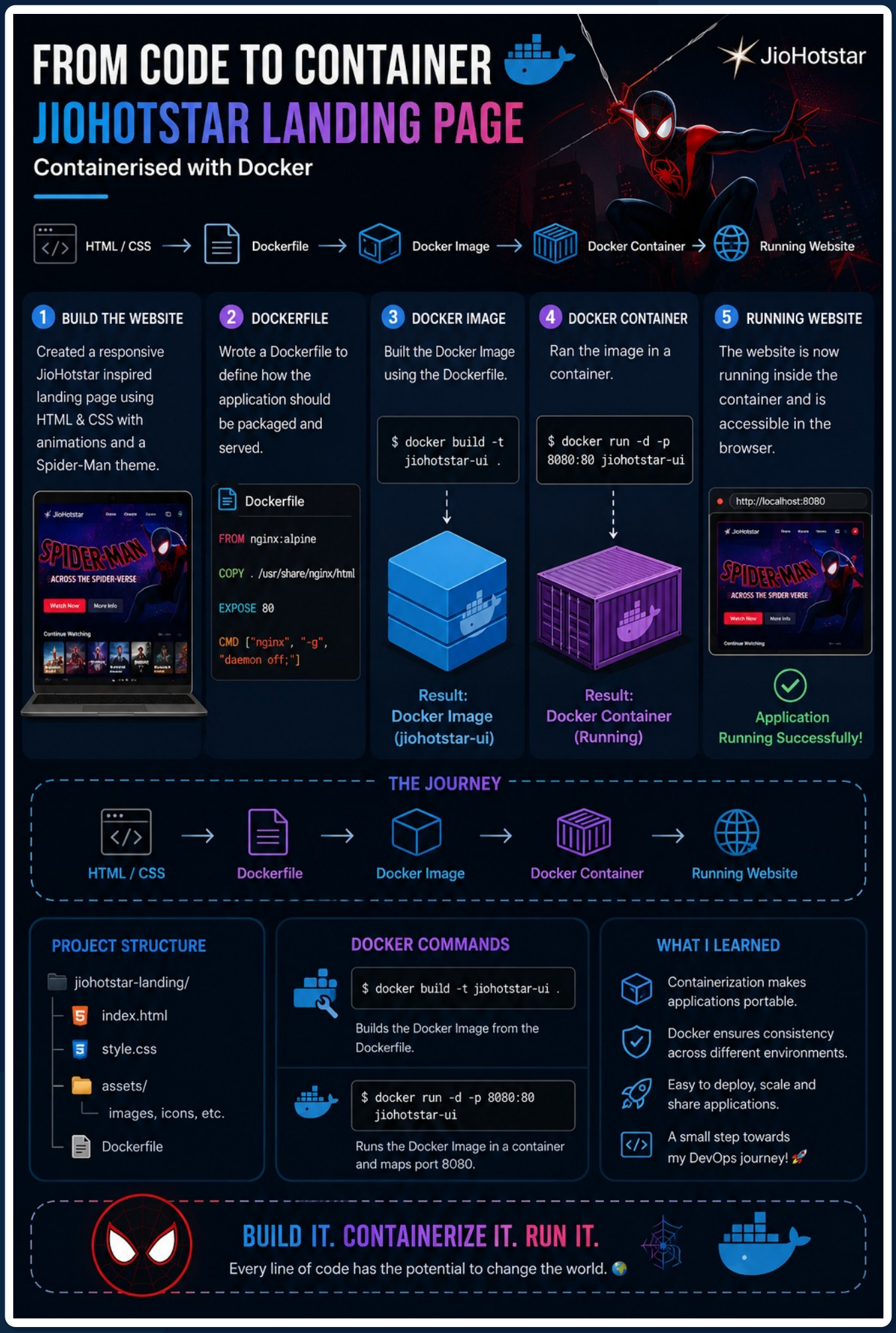
Docker docs: CLI reference
https://docs.docker.com/reference/cli/docker/

CHAPTER IDEA

Instead of using the stock meal kit, you add your own sauce: your own index.html baked into the nginx image.

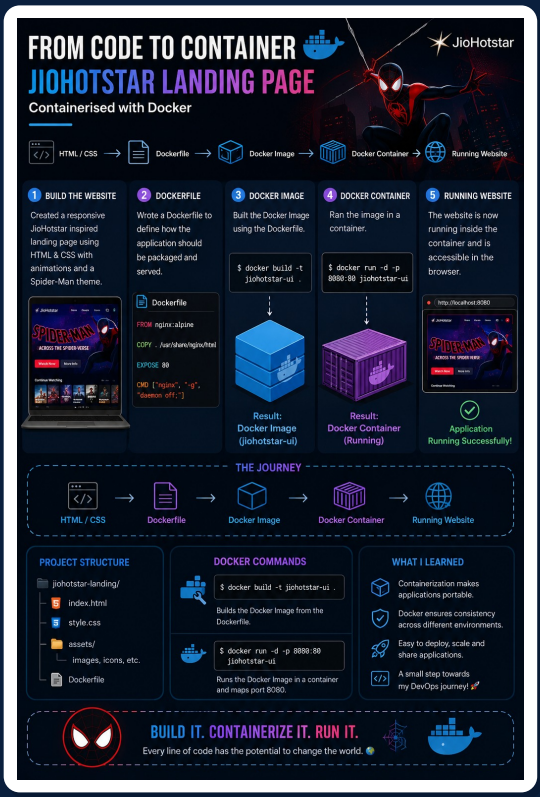
© 2026 cloudmessenger

Figure 24: From code to container: a streaming-site landing page



© 2026 cloudmessenger

Figure 24: From code to container: a streaming-site landing page



tap to see the full figure

IN PLAIN WORDS

The journey is HTML/CSS → Dockerfile → Docker image → container → running website. The page is built, packaged with **docker build -t jiohotstar-ui .** and started with **docker run -d -p 8080:80 jiohotstar-ui**. This version uses **nginx:alpine**, a smaller base image than plain nginx, and the lessons are portability, consistency and easy sharing.

KEY POINTS

- Alpine-based images are small and quick to pull
- Same container, same result on any machine
- Project folder: index.html, style.css, assets/, Dockerfile

GOOD TO KNOW

Brand and character artwork in this demo belongs to its owners and is shown for learning only.

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

Docker Hub: official nginx image
https://hub.docker.com/_/nginx

CHAPTER IDEA

Instead of using the stock meal kit, you add your own sauce: your own index.html baked into the nginx image.

© 2026 cloudmessenger

CHAPTER 8 OF 11

Docker Networking

How containers talk to each other and the world

THINK OF IT LIKE THIS

Bridge is a society road joining all houses, host is a guest on your home Wi-Fi, none is airplane mode, overlay is two offices joined by VPN.

YOU WILL LEARN

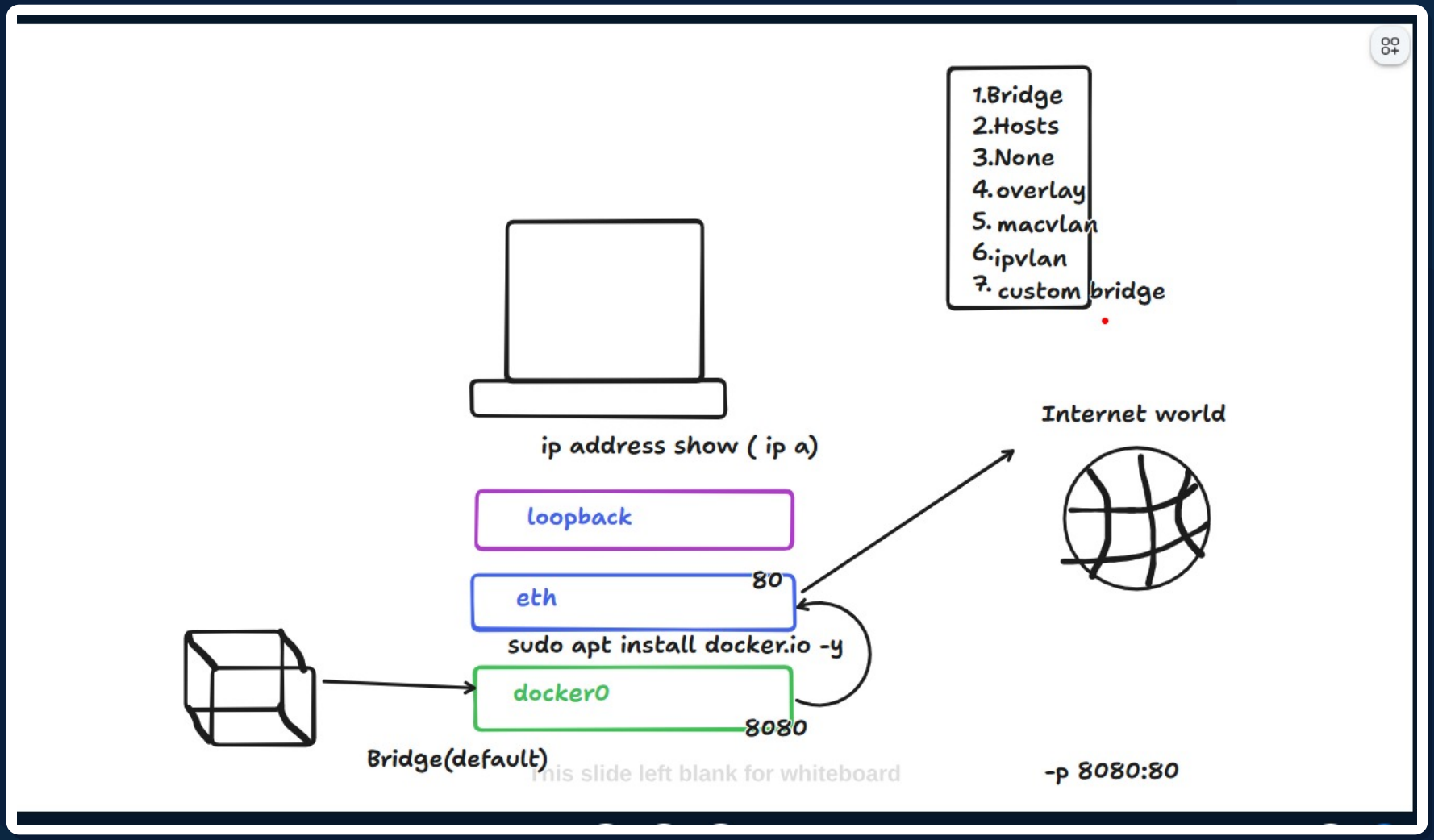
- docker0, eth and port mapping
- Bridge, host, none, overlay, macvlan, internal
- Commands to inspect, create and connect networks

IN THIS CHAPTER

Figure 25 Networking whiteboard, part 1: the machine's network cards (page 51)
Figure 26 Whiteboard, part 2: several containers, several ports (page 52)
Figure 27 Whiteboard, part 3: Bridge, Host, None (page 53)
Figure 28 Docker Networking: six drivers compared (page 54)
Figure 29 Networking in real life: analogies and production scenarios (page 56)
Figure 30 Networking commands you will actually type (page 58)

© 2026 cloudmessenger

Figure 25: Networking whiteboard, part 1: the machine's network cards



IN PLAIN WORDS

Run **ip a** on the Docker host and you see network interfaces: **loopback** (the machine talking to itself), **eth** (the real card facing the internet) and **docker0** (the virtual bridge Docker creates). A container plugs into docker0. **-p 8080:80** forwards host port 8080 to container port 80. The list at the top names seven network types.

KEY POINTS

- docker0 is created automatically when Docker is installed
- eth is how traffic reaches the outside world
- -p HOST:CONTAINER is the door between them

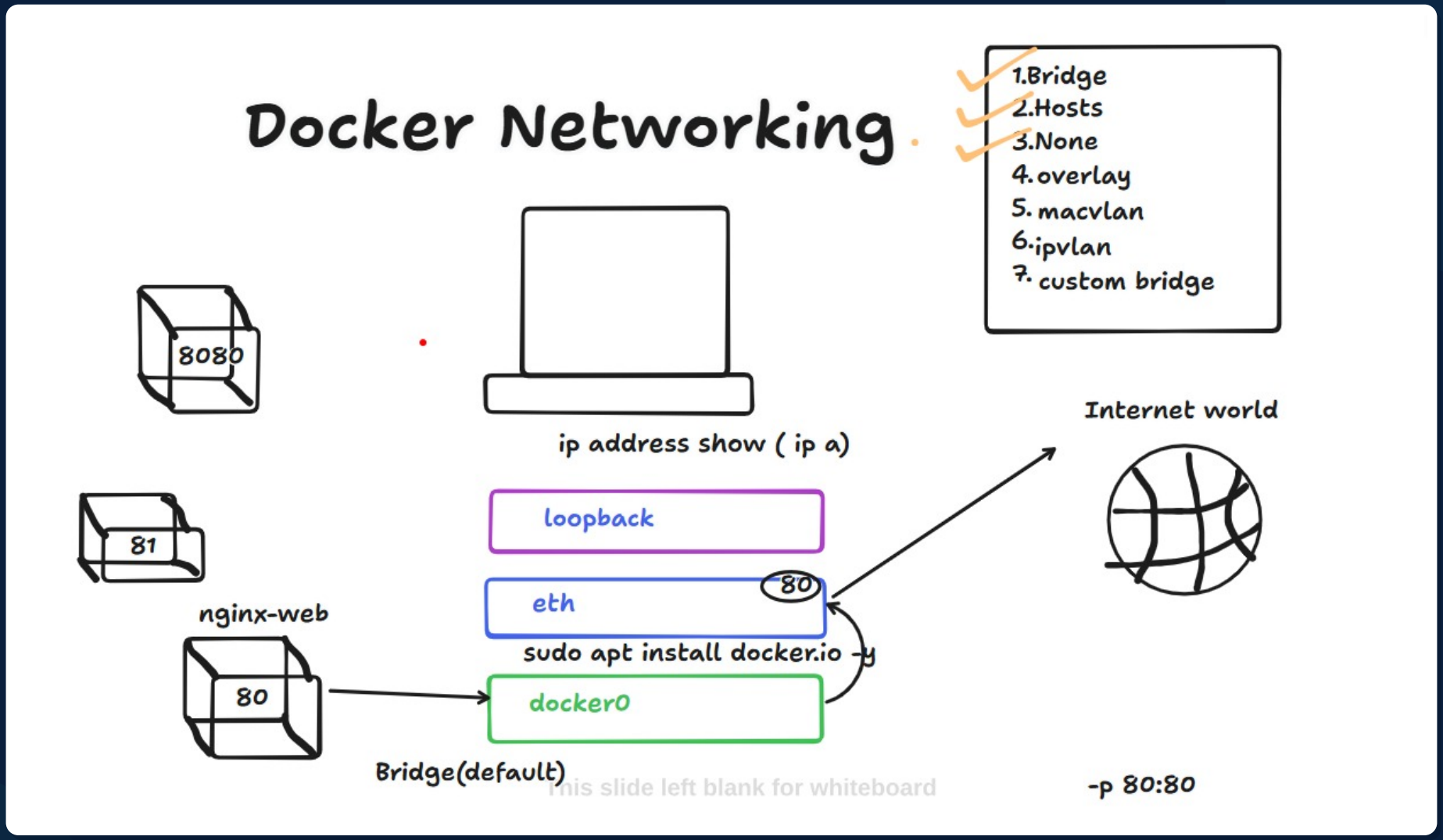
RESOURCES (TAP TO OPEN)

Docker docs: Network drivers
https://docs.docker.com/engine/network/drivers/

Docker docs: Standalone networking tutorial
https://docs.docker.com/engine/network/tutorials/standalone/

© 2026 cloudmessenger

Figure 26: Whiteboard, part 2: several containers, several ports



IN PLAIN WORDS

The same picture with more containers. **nginx-web** listens on 80 inside its container, while others could be published on host ports 81 or 8080. Each published port is a separate door from the host to one container. Bridge, Hosts and None are ticked as the three to master first.

KEY POINTS

- One host port can serve only one container
- Use different host ports to run many sites
- Master Bridge, Host and None first

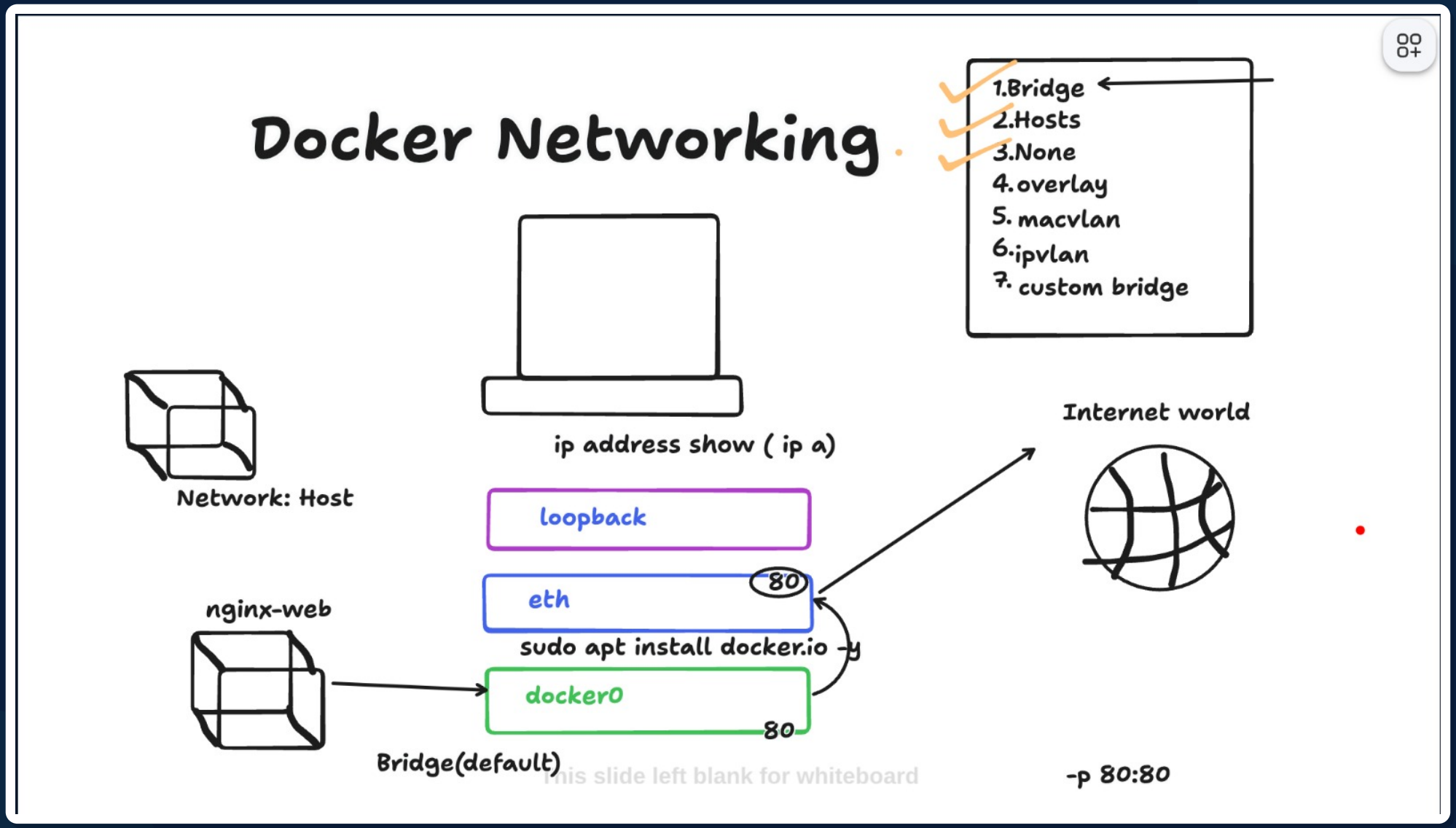
RESOURCES (TAP TO OPEN)

Docker docs: Networking overview
https://docs.docker.com/engine/network/

Docker docs: Standalone networking tutorial
https://docs.docker.com/engine/network/tutorials/standalone/

© 2026 cloudmessenger

Figure 27: Whiteboard, part 3: Bridge, Host, None



IN PLAIN WORDS

Bridge (default): the container plugs into docker0 and reaches the internet through eth.
Host: the container shares the host's network directly, so no port mapping is needed.
None: no network at all. Overlay, macvlan, ipvlan and custom bridge are the advanced four on the list.

KEY POINTS

- Bridge = default and most common
- Host = fastest but least isolated
- None = fully isolated

RESOURCES (TAP TO OPEN)

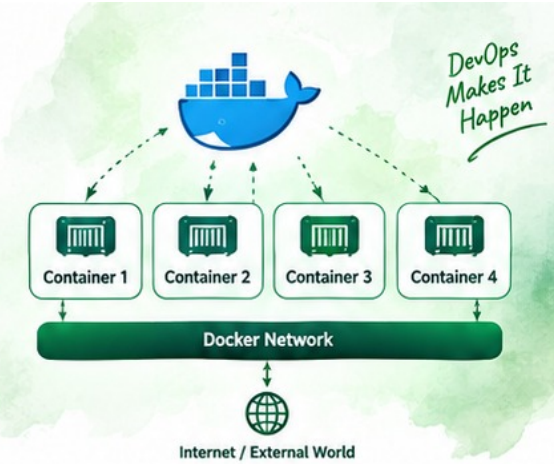
Docker docs: Network drivers
https://docs.docker.com/engine/network/drivers/

© 2026 cloudmessenger

Figure 28: Docker Networking: six drivers compared

Docker Networking

Connect • Communicate • Build Together



What is Docker Networking?

Docker networking allows containers to communicate with each other, with the host system, and with the outside world. It provides different network drivers (types) to suit various use cases, from simple development to complex production environments.

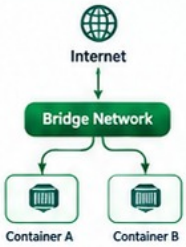
Right Network
Right Use Case
Better Deployments

Types of Docker Networking

Docker provides several network drivers. Here are the most commonly used ones:

1 Bridge Network (Default)

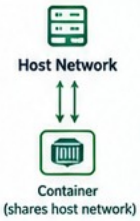
Creates a private network on the host machine. Containers can communicate with each other and access the internet through NAT (Network Address Translation).



Use Case
Local development and single host applications.

2 Host Network

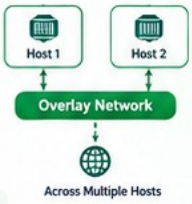
The container shares the host's network namespace. No isolation – it uses the host's IP address and ports.



Use Case
High performance, low latency, network-intensive applications.

3 Overlay Network

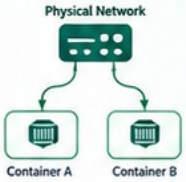
Connects containers across multiple Docker hosts. Uses VXLAN to create a virtual network over the physical network.



Use Case
Multi-host swarm, Kubernetes, production environments.

4 Macvlan Network

Assigns a MAC address to each container, making it appear as a physical device on the network.



Use Case
When you need direct access to the physical network (e.g., IoT, legacy apps).

5 None Network

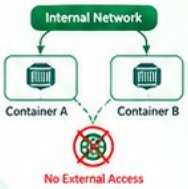
Disables all networking for the container. It is isolated from the outside world and other containers.



Use Case
For security, testing or when networking is not required.

6 Internal Network

Similar to bridge, but blocks external access. Containers can communicate only with each other, not with the internet or host.



Use Case
Secure communication between specific containers.

Quick Comparison

Network Type	Esolation	External Access	Multi-Host	Use Case
Bridge (default)	Yes	Yes (via NAT)	No	Dev & single host apps
Host	No	Yes (host IP)	No	High performance
Overlay	Yes	Yes	Yes	Multi-host / Swarm / K8s
Macvlan	No	Yes (direct)	No	Physical network access
None	Yes	No	No	Security / testing
Internal	Yes	No	No	Private communication

Key Features

- Multiple network drivers for different needs
- Container-to-container communication
- Isolation and security controls
- Support for multi-host environments (Overlay)
- Easy integration with Docker Compose and Swarm
- Customizable IPAM (IP Address Management)

Benefits

- Improved communication between containers
- Better security and isolation
- Flexible deployment options
- Supports scaling and clustering
- Simplifies complex architectures
- Optimized network performance

Best Practices

1. Use bridge for most development scenarios.
2. Use internal network for sensitive services.
3. Use overlay for multi-host or Kubernetes setups.
4. Keep network policies and firewall rules in place.
5. Monitor and log network traffic for security.

Example: Create a Bridge Network

```
# Create a network
docker network create my-network

# Run containers on the network
docker run -d --name app1 --network my-network nginx
docker run -d --name app2 --network my-network alpine
```



> *The power of **Docker** isn't just in **running** containers, but in **connecting** them in the right way.*
>
> — DevOps Engineer

CI/CD | AWS | Kubernetes | Docker | Better Together

© 2026 cloudmessenger

Figure 28: Docker Networking: six drivers compared



tap to see the full figure

IN PLAIN WORDS

Bridge, Host, Overlay, Macvlan, None and Internal side by side, with a comparison table covering isolation, external access, multi-host support and typical use. Best practices: use bridge for development, internal networks for sensitive services, overlay for multi-host or Kubernetes setups. The “Internal” type is a bridge network that blocks outside access.

KEY POINTS

- Overlay links containers across several Docker hosts
- Macvlan gives a container its own MAC address on the LAN
- Example: docker network create my-network

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Network drivers
https://docs.docker.com/engine/network/drivers/

Docker docs: Networking overview
https://docs.docker.com/engine/network/

CHAPTER IDEA

Bridge is a society road joining all houses, host is a guest on your home Wi-Fi, none is airplane mode, overlay is two offices joined by VPN.

© 2026 cloudmessenger

Figure 29: Networking in real life: analogies and production scenarios

DOCKER NETWORKING

Connect Containers. Power Applications.

Simple Definition

Docker Networking is a communication bridge that allows Docker containers to talk to each other and to the outside world securely.

WHY DO WE NEED DOCKER NETWORKING?

Each container runs in its own isolated world. Networking allows them to communicate with each other and with the outside world.

Website (Container) → Backend API (Container) → MySQL DB (Container)

Real-Time Example

In Netflix, every service runs in its own container and communicates over Docker Networking.

Frontend → Login Service → Streaming Service → Database

REAL-LIFE ANALOGY

Bridge Network
Like a road in a society connecting all houses.

Host Network
Like a guest using your home Wi-Fi directly.

None Network
Like airplane mode – no network at all.

Overlay Network
Like two office branches connected over VPN.

Mumbai — VPN VPN — Bangalore

Macvlan Network
Like each person having their own internet connection and IP.

IMPORTANCE OF DOCKER NETWORKING

- Enables containers to communicate securely.
- Keeps applications modular and scalable.
- Improves security with network isolation.
- Allows multi-host / multi-container communication.
- Essential for Microservices & Production applications.
- Helps in load balancing, service discovery and high availability.

> Without Networking, containers are like people in different rooms who cannot talk to each other.

TYPES OF DOCKER NETWORKS

1 BRIDGE NETWORK (Default)

Containers on the same Docker host communicate with each other using a private network.

Docker Bridge (app-network)

Docker Host

Production Use Cases
- Most common network
- Web App (Frontend ↔ Backend ↔ DB)
- Microservices on single host

Example Commands

```
# List networks
docker network ls

# Create bridge network
docker network create app-network

# Run container on network
docker run -d --name web \
  --network app-network nginx

# Run another container
docker run -it --name ubuntu-box \
  --network app-network ubuntu bash

# Inspect network
docker network inspect app-network
```

Key Importance
- Secure & private communication between containers.
- Easy to create and manage.
- Default choice for most applications.

2 HOST NETWORK

Container shares the host machine's network directly.

Host Network (Uses Host IP)

Docker Host

Production Use Cases
- High-performance applications
- Monitoring tools (Prometheus, Node Exporter)
- When low latency is critical

Example Command

```
# Run container with host network
docker run --network host nginx
```

Key Importance
- Highest performance.
- No port mapping required.
- Less network overhead.

⚠ Less isolation – use carefully.

3 NONE NETWORK

Container has no networking capabilities at all.

No Network

Docker Host

Production Use Cases
- Security testing
- Offline / isolated processing
- Sensitive workloads

Example Command

```
docker run --network none ubuntu
```

Key Importance
- Maximum isolation.
- No external or internal communication.
- Enhanced security.

4 OVERLAY NETWORK

Connect containers across multiple Docker hosts.

VPN

Docker Host 1 (Mumbai) — Docker Host 2 (Bangalore)

Production Use Cases
- Docker Swarm
- Multi-host microservices
- Distributed applications

Example Command

```
# Create overlay network
docker network create \
  -d overlay company-network
```

Note: Requires Docker Swarm to be enabled.

Key Importance
- Multi-host communication.
- Highly scalable.
- Used in clustered environments.

5 MACVLAN NETWORK

Gives each container its own MAC address and IP on the physical network.

192.168.1.10 192.168.1.11 192.168.1.12

Physical Network LAN (192.168.1.0/24)

Production Use Cases
- Legacy applications
- Network appliances
- Monitoring systems that need their own IP

Example Command

```
docker network create \
-d macvlan \
--subnet=192.168.1.0/24 \
--gateway=192.168.1.1 \
macvlan-net
```

Key Importance
- Each container gets its own IP.
- Appears like a physical machine on the network.
- Useful for integration with existing infrastructure.

QUICK REFERENCE TABLE

Network Type	Think Like	Best For	Isolation	Performance
Bridge	Society Road	Most apps on single host	Medium	Good
Host	Guest uses Home Wi-Fi	High performance tools	Low	Highest
None	Airplane Mode	Secure / Offline tasks	Highest	N/A
Overlay	Offices connected by VPN	Multi-host applications	Medium	Good
Macvlan	Own Internet Connection	Containers needing own IP	Medium	Good

REAL-TIME PRODUCTION SCENARIOS

1 Website cannot connect to MySQL

First command:

```
docker network inspect app-network
```

Reason: Check if both containers are on the same network.

2 Backend API cannot connect to Redis

Solution:

```
docker network connect app-network redis
```

Now Backend can communicate with Redis.

3 New Microservice Added

Solution:

```
docker network connect app-network payment-service
```

New service can now talk to other services without changing application code.

INTERVIEW QUESTIONS

Q. What is Docker Networking?
👉 It enables containers to communicate with each other and external systems.

Q. Which network is used by default?
👉 Bridge Network

Q. Which network gives the best performance?
👉 Host Network

Q. Which network completely disables networking?
👉 None Network

Q. Which network is used for multiple hosts?
👉 Overlay Network

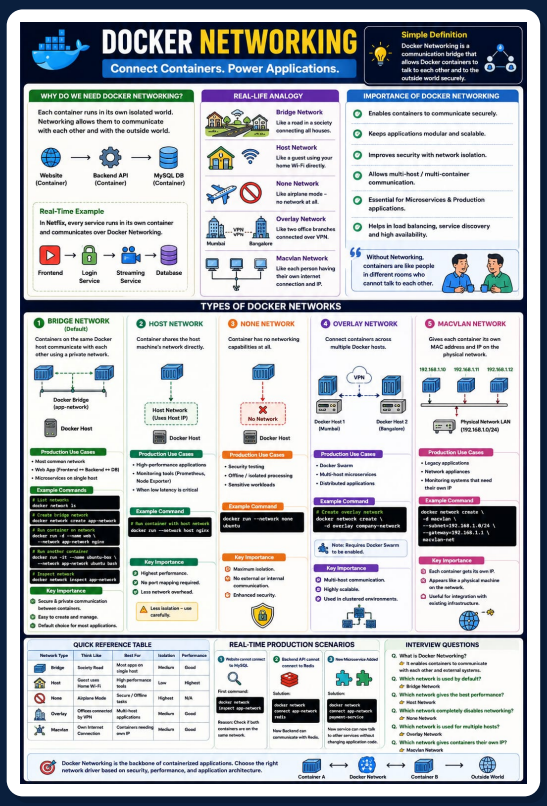
Q. Which network gives containers their own IP?
👉 Macvlan Network

Docker Networking is the backbone of containerized applications. Choose the right network driver based on security, performance, and application architecture.

Container A ↔ Docker Network ↔ Container B → Outside World

© 2026 cloudmessenger

Figure 29: Networking in real life: analogies and production scenarios



tap to see the full figure

IN PLAIN WORDS

Why networking matters: a website container, a backend API and a MySQL database must talk to each other. The analogies: bridge is a road joining houses, host is a guest on your Wi-Fi, none is airplane mode, overlay is two offices on a VPN, macvlan is everyone having their own connection. It ends with troubleshooting scenarios and interview questions.

KEY POINTS

- Website cannot reach MySQL? docker network inspect first
- Backend cannot reach Redis? docker network connect
- Default network: bridge. Fastest: host. Multi-host: overlay

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Networking overview
https://docs.docker.com/engine/network/

Docker docs: Network drivers
https://docs.docker.com/engine/network/drivers/

CHAPTER IDEA

Bridge is a society road joining all houses, host is a guest on your home Wi-Fi, none is airplane mode, overlay is two offices joined by VPN.

© 2026 cloudmessenger

Figure 30: Networking commands you will actually type

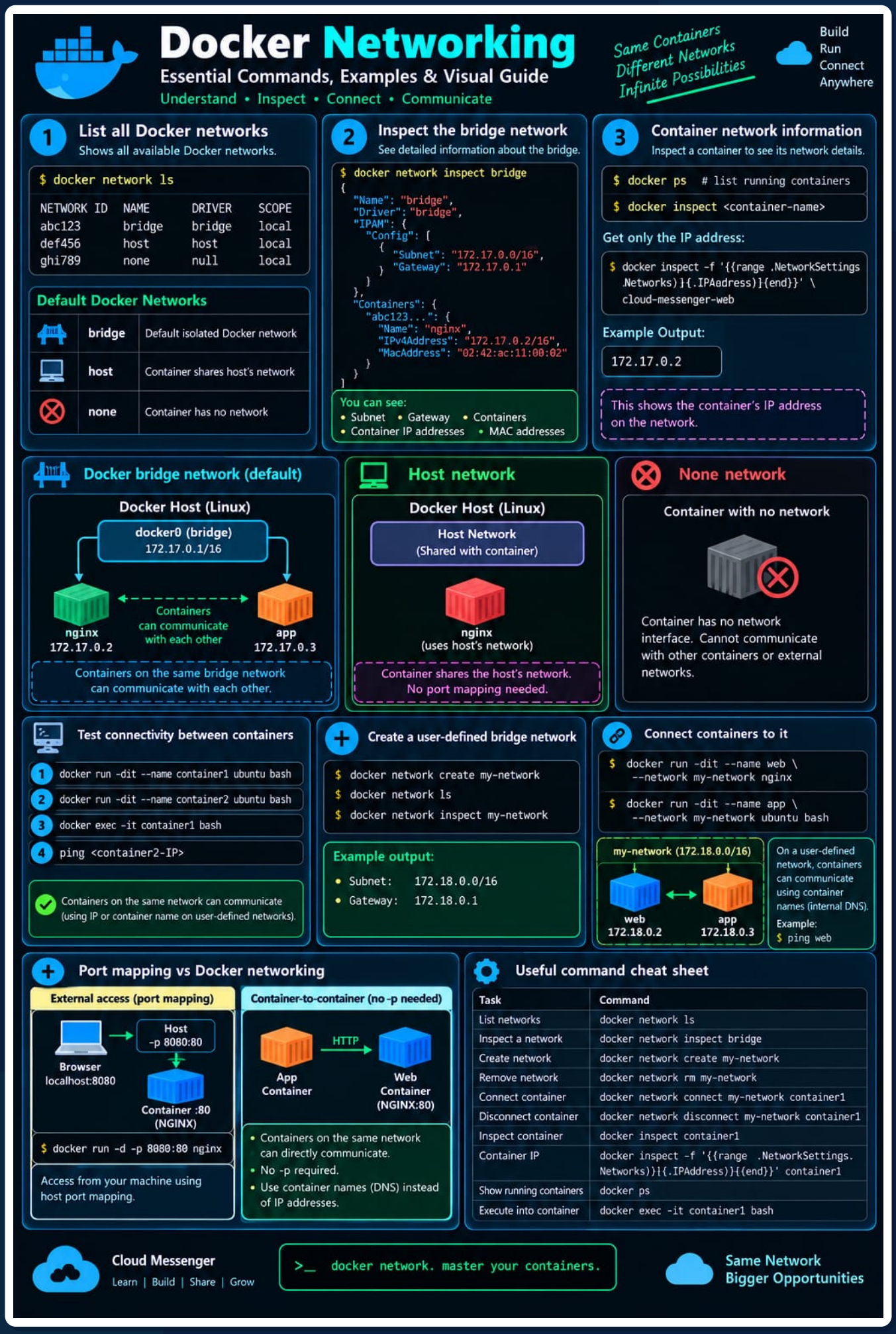
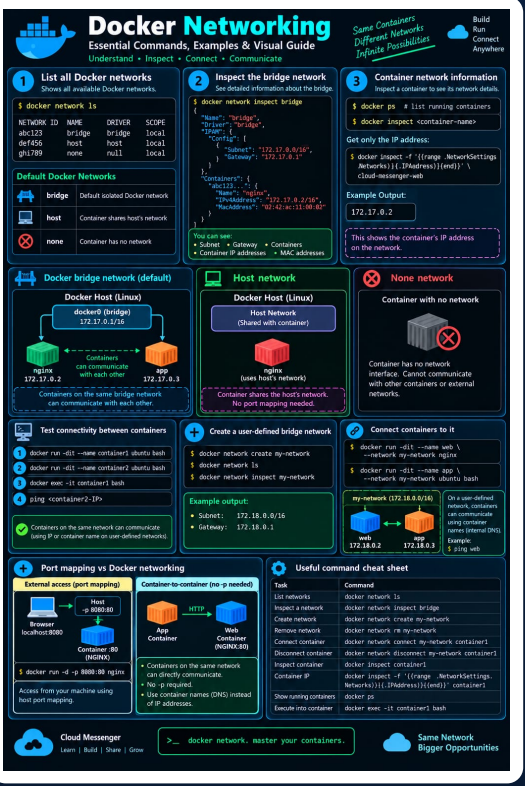


Figure 30: Networking commands you will actually type



tap to see the full figure

IN PLAIN WORDS

A hands-on guide: **docker network ls**, **docker network inspect bridge**, and docker inspect to read a container's IP. Test connectivity by starting two Ubuntu containers and pinging one from the other. Create your own bridge with **docker network create my-network**. On such user-defined networks containers can reach each other by *name*.

KEY POINTS

- Outside access needs -p; container-to-container does not
- User-defined networks give you name-based DNS
- docker exec -it container1 bash lets you step inside to test

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Standalone networking tutorial
https://docs.docker.com/engine/network/tutorials/standalone/

Docker docs: CLI reference
https://docs.docker.com/reference/cli/docker/

CHAPTER IDEA

Bridge is a society road joining all houses, host is a guest on your home Wi-Fi, none is airplane mode, overlay is two offices joined by VPN.

© 2026 cloudmessenger

CHAPTER 9 OF 11

Docker Compose

Many containers, one file

THINK OF IT LIKE THIS

Instead of starting each band member separately, Compose is the conductor: one score, one command, the whole orchestra plays.

YOU WILL LEARN

- Reading a docker-compose YAML line by line
- Why a helper Ubuntu container is useful
- The commands up, ps, logs, down

IN THIS CHAPTER

Figure 31 Why is there an Ubuntu container in Docker Compose? (page 61)
Figure 32 Compose YAML explained line by line (racing_car + ubuntu_admin) (page 63)
Figure 33 Every Compose component with diagrams (flipkart-web + ubuntu-admin) (page 65)

© 2026 cloudmessenger

Figure 31: Why is there an Ubuntu container in Docker Compose?

Why are we pulling the Ubuntu image in Docker Compose?

Ubuntu container is used as a utility / admin container for learning, testing & troubleschooting.

1 Your Docker Compose File

```
version: "3.9"

services:
  racing_car:
    build: .
    container_name: racing_car_container
    ports:
      - "8080:80"
    restart: always

  ubuntu_admin:
    image: ubuntu
    container_name: ubuntu_admin
    tty: true
```

- racing_car : Your application container
- ubuntu_admin : Utility / administration container

2 Why do we use the Ubuntu container?

It is NOT required for the application to run.
It is added as a utility/Admin container for:

Practice Linux Commands
You can enter the container and practice Linux commands.

```
docker exec -it ubuntu_admin bash
```

Test Network Connectivity
Check communication between containers.

```
ping racing_car
curl http://racing_car
```

Troubleshooting
Debug DNS, network, API, or application issues without touching the main container.

Safe Testing Environment
Use it as a sandbox to test tools, scripts, or configurations.



ubuntu_admin
(Utility Container)

3 Why tty: true?

Keeps the Ubuntu container running by allocating a terminal.

Without tty: true ✗	With tty: true ✓
Ubuntu starts	Ubuntu starts
↓	↓
No foreground process	TTY allocated
↓	↓
Container exits immediately	Container stays running
STOP	▶

Now you can access it anytime:
`docker exec -it ubuntu_admin bash`

4 Is Ubuntu mandatory?

NO. ✗

If your project only needs the web application, this is enough:

```
services:
  racing_car:
    build: .
    ports:
      - "8080:80"
```

Note
The Ubuntu service can be removed completely if not required.

5 Is tty: true an alternative to docker exec?

NO. ✗ They are different.

tty: true
Keeps the container alive by providing a terminal.

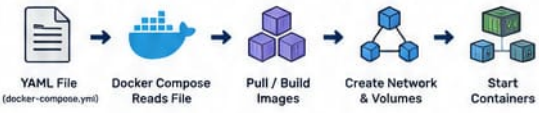
≠

docker exec -it ubuntu_admin bash
Command used to enter a container that is already running.

They work together, not as replacements.

`tty: true` keeps the container running,
`docker exec -it` lets you enter it.

6 How does it work? (High Level Flow)



Both containers are on the same network

racing_car_container (Application) ⟷ Can communicate using service name ⟷ ubuntu_admin (Utility / Admin)

7 Common Mistakes & How to Fix

Mistake	Problem	Solution
Wrong Indentation	YAML error (mapping values not allowed here)	Use 2 spaces for indentation. Never use tabs.
Wrong File Name	Docker Compose can't find the file	Use docker-compose.yml (or compose.yml)
Port Already in Use	Bind for 0.0.0.0:80 failed	Stop other container using the port or use another port (e.g. 8080:80)
Image Not Found	pull access denied or repository not found	Check image name on Docker Hub and fix the spelling
Dockerfile Missing (when using build: .)	failed to read Dockerfile	Create Dockerfile in the specified build directory
Wrong Environment Variable	Application can't read the value	Check variable names expected by the application
Missing Dash in List	YAML doesn't treat it as a list	Use dash (-) for list items (e.g. ports, volumes, etc.)
Using Tabs	YAML parsing error	Use spaces, not tabs

8 Useful Docker Commands

`docker compose up -d`	Start all services in background
`docker compose ps`	List running containers
`docker compose logs`	View logs of all services
`docker compose restart`	Restart all services
`docker compose down`	Stop & remove containers, networks
`docker compose down -v`	Also remove volumes
`docker compose up --build`	Rebuild images & start
`docker exec -it ubuntu_admin bash`	Enter ubuntu container
`docker compose config`	Validate & view final config

9 Production Use Case Example

A typical production stack may look like this:

Nginx (Frontend) · Node.js (Backend) · MySQL (Database) · Redis (Cache) · RabbitMQ (Message) · Prometheus (Monitoring)

Run one command and the entire application stack is ready!

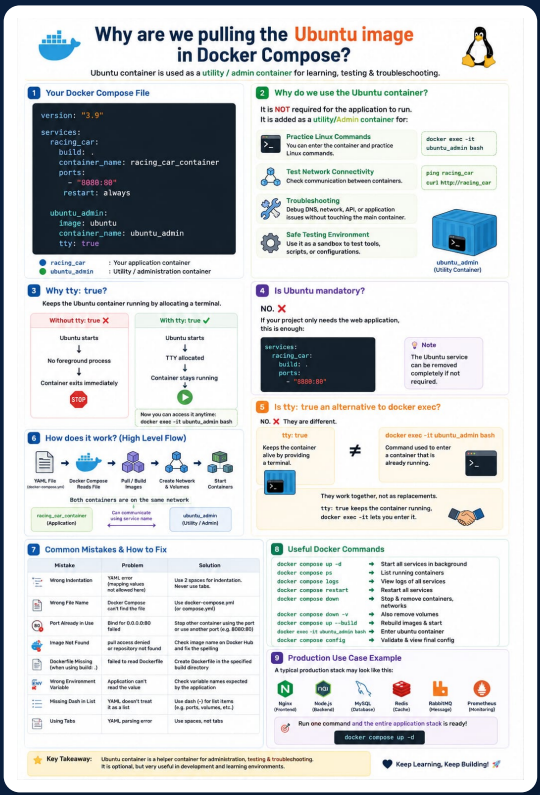
```
docker compose up -d
```

Key Takeaway: Ubuntu container is a helper container for administration, testing & troubleshooting. It is optional, but very useful in development and learning environments.

Keep Learning, Keep Building!

© 2026 cloudmessenger

Figure 31: Why is there an Ubuntu container in Docker Compose?



tap to see the full figure

IN PLAIN WORDS

The compose file defines two services: **racing_car** (your application, built from the Dockerfile) and **ubuntu_admin** (a helper). Ubuntu is optional: it is a utility container for practising Linux commands, pinging racing_car and troubleshooting. **tty: true** keeps it alive and **docker exec -it ubuntu_admin bash** lets you step inside. A table of common YAML mistakes is included.

KEY POINTS

- tty: true keeps the container running; docker exec lets you enter it
- Remove ubuntu_admin if you only need the app
- YAML hates tabs: use two spaces per indent level

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Compose overview
https://docs.docker.com/compose/

Docker docs: Compose services reference
https://docs.docker.com/reference/compose-file/services/

CHAPTER IDEA

Instead of starting each band member separately, Compose is the conductor: one score, one command, the whole orchestra plays.

© 2026 cloudmessenger

Figure 32: Compose YAML explained line by line (racing_car + ubuntu_admin)

DOCKER COMPOSE YAML – EXPLAINED LINE BY LINE

Understand each and every line in your docker-compose.yml

docker-compose.yml

```
1  version: "3.9"
2
3  services:
4    racing_car:
5      build: .
6      container_name: racing_car_container
7      ports:
8        - "8080:80"
9      restart: always
10
11   ubuntu_admin:
12     image: ubuntu
13     container_name: ubuntu_admin
14     tty: true
15
16
18
```

Service 1 — racing_car

Service 2 — ubuntu_admin

LINE-BY-LINE EXPLANATION

1	`version: "3.9"`	Defines the Docker Compose file format version. (Optional in modern Docker Compose V2)
3	`services:`	Parent section that contains all the services (containers).
4	`racing_car:`	Name of the first service. Used to identify the container internally and by other services.
5	`build: .`	Builds a Docker image from the Dockerfile in the current directory (.)
6	`container_name: racing_car_container`	Gives a fixed name to the container. Useful for easy management.
7	`ports:`	Defines port mappings to access the container from outside.
8	`- "8080:80"`	Maps Host Port 8080 to Container Port 80. Access app using http://<host-ip>:8080
9	`restart: always`	If container stops or system restarts, Docker will start it automatically.
11	`ubuntu_admin:`	Name of the second service.
12	`image: ubuntu`	Uses the official Ubuntu image from Docker Hub.
13	`container_name: ubuntu_admin`	Gives a fixed name to the Ubuntu container.
14	`tty: true`	Allocates a pseudo-terminal (TTY) so the container stays running and you can access it interactively.

WHAT HAPPENS WHEN YOU RUN?

Read docker-compose.yml → Read version (3.9) → Read services → Build image for racing_car → Pull ubuntu image

Create default network → Create racing_car_container → Create ubuntu_admin → Apply port mapping 8080 → 80 → Start both containers

CONTAINER ACCESS

Access your application in browser

http://<EC2-IP>:8080

Access Ubuntu container

```
docker exec -it
ubuntu_admin bash
```

APPLICATION FLOW

User / Browser → Host (EC2) Port 8080 → Docker Container racing_car_container (Nginx (Port 80))

NETWORK BETWEEN CONTAINERS

Both containers are connected to the same default Docker Compose network.

Default Network: racing_car (nginx) ⟷ ubuntu_admin (ubuntu)

They can communicate using service names.
Example: From `ubuntu_admin` → `curl http://racing_car`

SUMMARY TABLE

Line No.	YAML	Purpose
1	`version: "3.9"`	Specifies the Compose file format version.
3	`services:`	Defines all services.
4	`racing_car:`	First service name.
5	`build: .`	Builds image from Dockerfile in current directory.
6	`container_name: racing_car_container`	Sets custom container name.
7	`ports:`	Defines port mapping.
8	`- "8080:80"`	Maps host port 8080 to container port 80.
9	`restart: always`	Auto restart container if it stops.
11	`ubuntu_admin:`	Second service name.
12	`image: ubuntu`	Uses Ubuntu image from Docker Hub.
13	`container_name: ubuntu_admin`	Sets custom container name.
14	`tty: true`	Keeps the container running with a terminal.

COMMON DOCKER COMPOSE COMMANDS

`docker compose up -d`	Start all services in detached mode
`docker compose up`	Start services in foreground mode
`docker compose ps`	List running containers
`docker compose logs`	View logs of all services
`docker compose down`	Stop and remove containers, networks
`docker compose down -v`	Also remove volumes
`docker compose restart`	Restart all services
`docker compose exec <service> bash`	Enter into a container

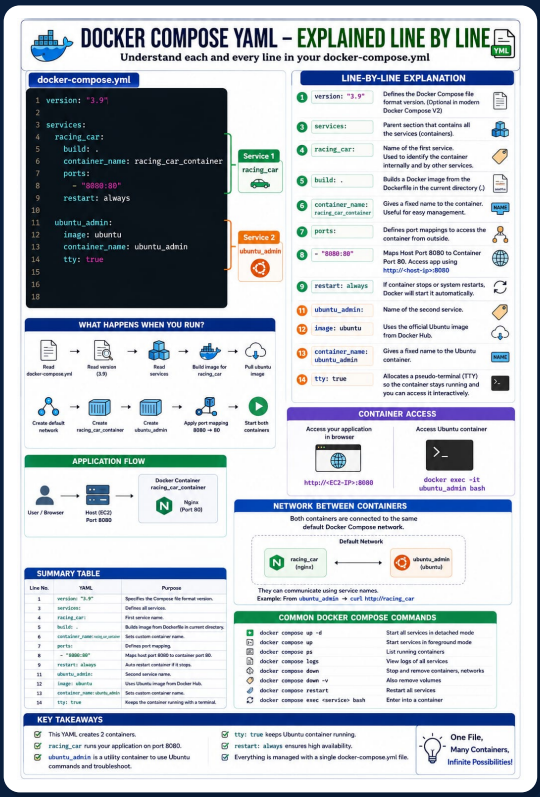
KEY TAKEAWAYS

- This YAML creates 2 containers.
- `racing_car` runs your application on port 8080.
- `ubuntu_admin` is a utility container to use Ubuntu commands and troubleshoot.
- `tty: true` keeps Ubuntu container running.
- `restart: always` ensures high availability.
- Everything is managed with a single docker-compose.yml file.

One File, Many Containers, Infinite Possibilities!

© 2026 cloudmessenger

Figure 32: Compose YAML explained line by line (racing_car + ubuntu_admin)



tap to see the full figure

IN PLAIN WORDS

Every line of the file is numbered and explained: **services**, the service names, **build: .**, **container_name**, **ports: “8080:80”**, **restart: always**, **image: ubuntu** and **tty: true**. The page also shows what happens when you run it and how both containers share one default network and can call each other by service name.

KEY POINTS

- One file creates two containers
- Service names act as hostnames on the shared network
- Browse your app at http://<host-ip>:8080

GOOD TO KNOW

The figure shows version: "3.9". Modern Docker Compose treats the version line as optional and prefers a file named compose.yaml (docker-compose.yml still works).

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: How Compose works
https://docs.docker.com/compose/intro/compose-application-model/

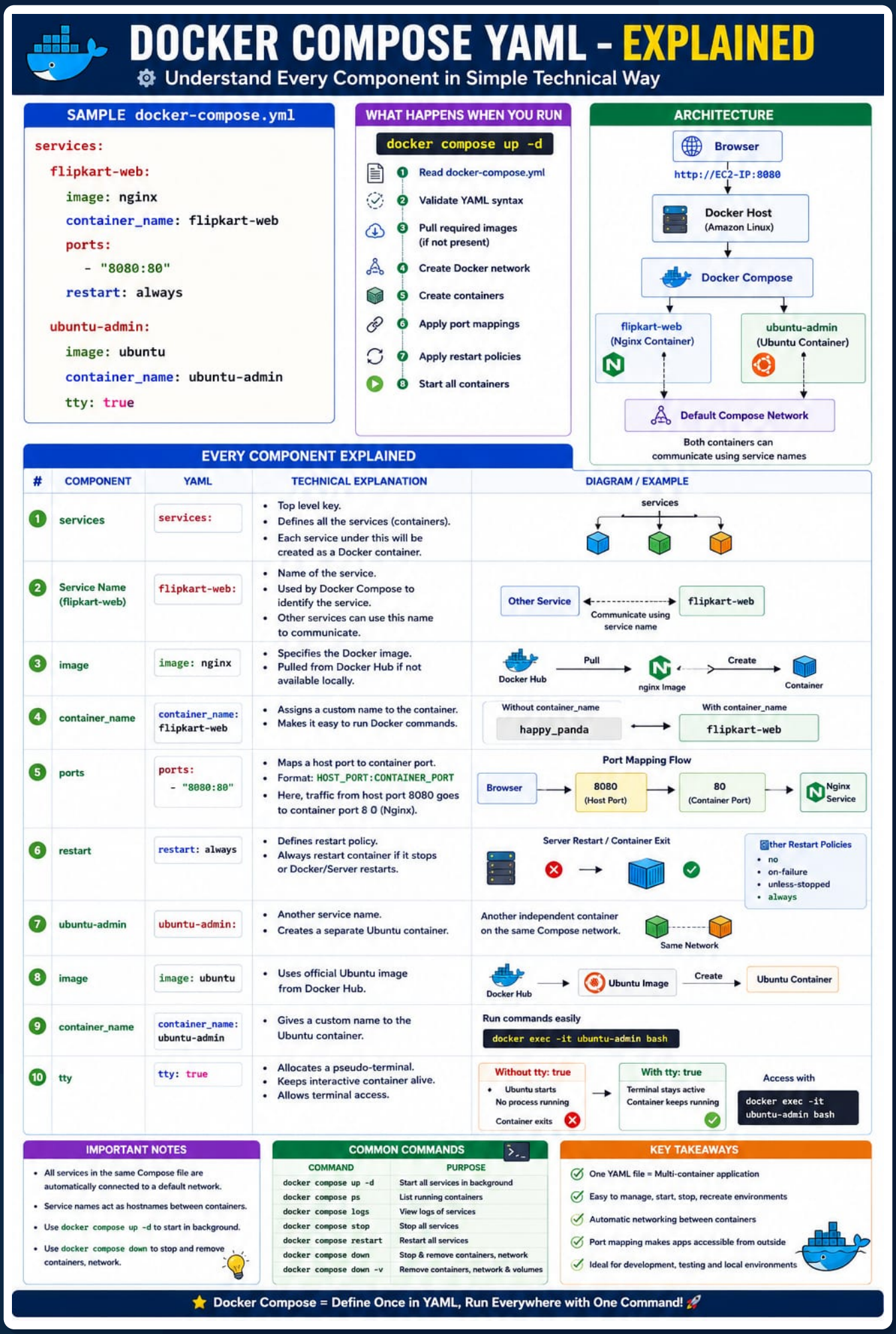
Docker docs: Compose services reference
https://docs.docker.com/reference/compose-file/services/

CHAPTER IDEA

Instead of starting each band member separately, Compose is the conductor: one score, one command, the whole orchestra plays.

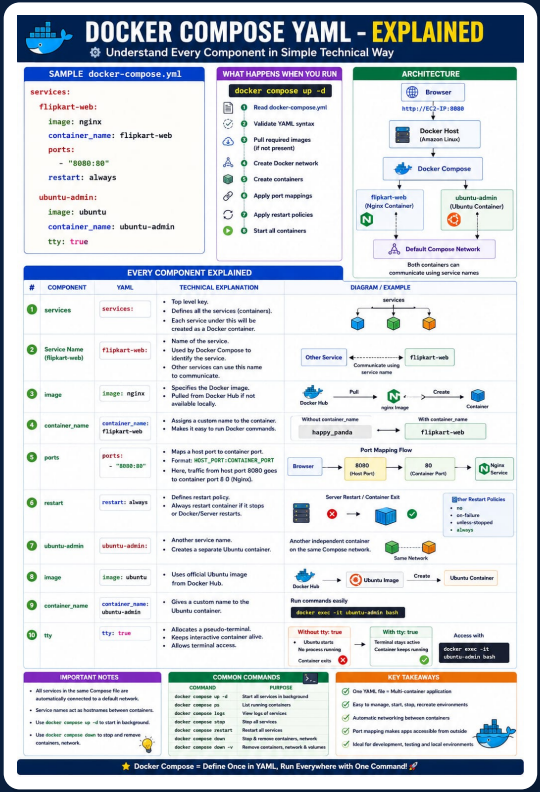
© 2026 cloudmessenger

Figure 33: Every Compose component with diagrams (flipkart-web + ubuntu-admin)



© 2026 cloudmessenger

Figure 33: Every Compose component with diagrams (flipkart-web + ubuntu-admin)



tap to see the full figure

IN PLAIN WORDS

A second walk-through with a different example. It lists the eight steps of **docker compose up -d**, then explains services, image, container_name, ports, **restart policies** (no, on-failure, unless-stopped, always) and tty, each with a small diagram. The architecture panel shows the browser reaching EC2 port 8080, Compose, and two containers on one network.

KEY POINTS

- Without container_name Docker invents names like happy_panda
- restart: always revives the container after a crash or reboot
- docker compose down stops and removes containers and networks

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

Docker docs: Compose overview
https://docs.docker.com/compose/

Docker docs: How Compose works
https://docs.docker.com/compose/intro/compose-application-model/

CHAPTER IDEA

Instead of starting each band member separately, Compose is the conductor: one score, one command, the whole orchestra plays.

© 2026 cloudmessenger

CHAPTER 10 OF 11

ECR & ECS on AWS

Ship containers to the cloud

THINK OF IT LIKE THIS

ECR is the warehouse where finished boxes are stored. ECS is the factory floor that unpacks them, runs them and adds more workers when the crowd grows.

YOU WILL LEARN

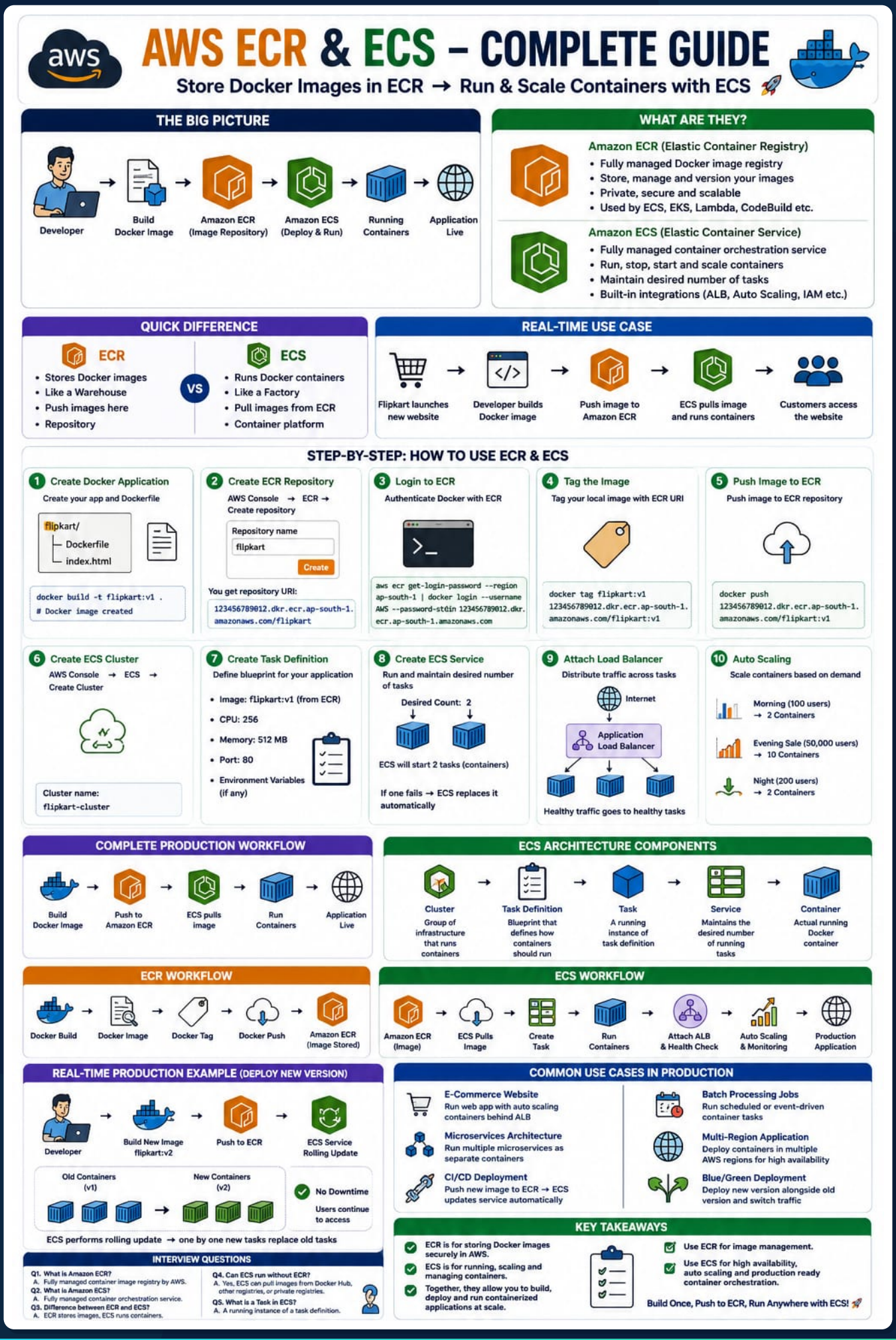
- Tag, log in and push an image to ECR
- Cluster, task definition, service
- Load balancer, auto scaling and zero-downtime updates

IN THIS CHAPTER

Figure 34 AWS ECR & ECS: complete guide (page 68)

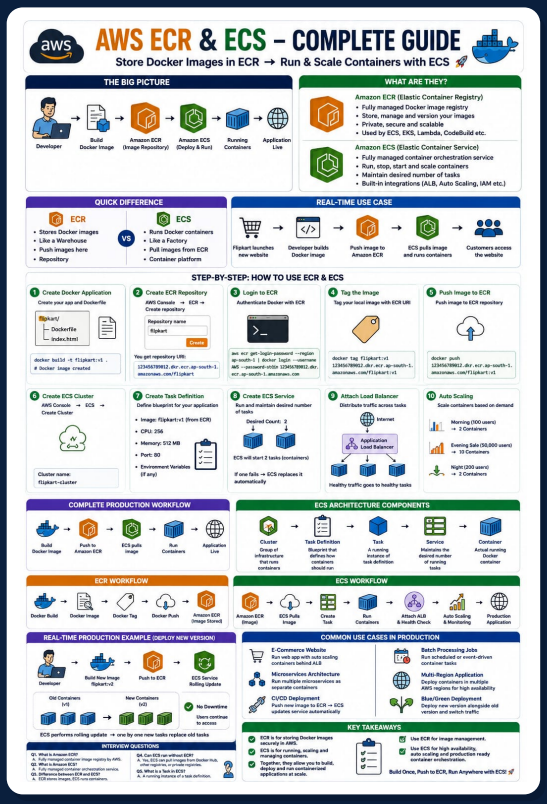
© 2026 cloudmessenger

Figure 34: AWS ECR & ECS: complete guide



© 2026 cloudmessenger

Figure 34: AWS ECR & ECS: complete guide



tap to see the full figure

IN PLAIN WORDS

ECR is the warehouse that stores images; **ECS** is the factory that runs and scales them. Ten steps: build the app, create an ECR repository, log in, tag, push, create an ECS cluster, define a task, create a service, add a load balancer and turn on auto scaling. A new version rolls out one task at a time with no downtime.

KEY POINTS

- Tag and push: docker tag, then docker push to the repository URI
- Task definition = blueprint; service = keep N tasks alive
- Rolling update replaces old containers one by one

RESOURCES: TAP A LINK, OR COPY THE ADDRESS

AWS: What is Amazon ECR?
https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html

AWS: Push an image to Amazon ECR
https://docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli.html

AWS: Amazon ECS Developer Guide
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

AWS: Install Docker on Amazon Linux 2023
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html

CHAPTER IDEA

ECR is the warehouse where finished boxes are stored. ECS is the factory floor that unpacks them, runs them and adds more workers when the crowd grows.

© 2026 cloudmessenger

CHAPTER 11 OF 11

Production Playbook

From code to container, done properly

THINK OF IT LIKE THIS

A lemonade stand and a restaurant chain use the same recipe. The chain adds locked doors, labelled ingredients, health inspections and a plan for when the stove breaks. This chapter upgrades the lemonade stand.

YOU WILL LEARN

- What every command in the JioHotstar-style demo really does
- What to change before a real public launch
- A production checklist and a safer reference build

IN THIS CHAPTER

Executive summary and architecture (page 71)
Visual project showcase (page 72)
Phase 1: Server preparation (8 commands) (page 73)
Phases 2 and 3: Workspace, image and build (page 74)
Phase 4: Container launch (page 75)
Production readiness checklist (page 76)
Safer reference implementation (page 77)
Command summary and conclusion (page 78)
References (page 79)

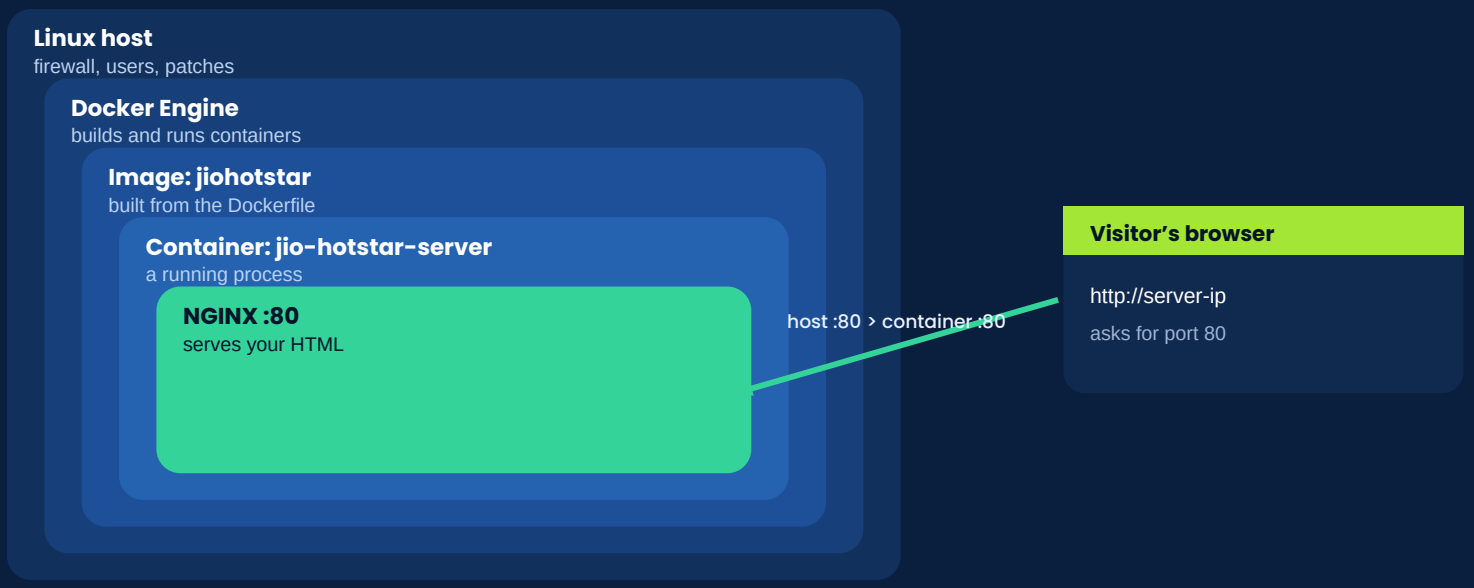
© 2026 cloudmessenger

Executive summary: from one HTML file to a running container

THE IDEA

This project shows the shortest path from a static HTML file to a web server running inside a Docker container. The commands are perfect for learning and for small, controlled deployments. A real public service needs the extra care described in the rest of this chapter.





CORE DISTINCTION

A **Dockerfile** is a recipe. An **image** is the packaged result of that recipe. A **container** is a running process created from an image. `docker build` creates the image; `docker run` creates and starts the container.

HOST

Artifact: Linux server, Docker Engine, firewall, users

Responsibility: Patch, secure, monitor and back up the host

IMAGE

Artifact: jiohotstar:latest built from the Dockerfile

Responsibility: Build reproducibly, scan, sign, store in a registry

CONTAINER

Artifact: jio-hotstar-server running nginx

Responsibility: Publish traffic, observe health, rotate logs, restart safely

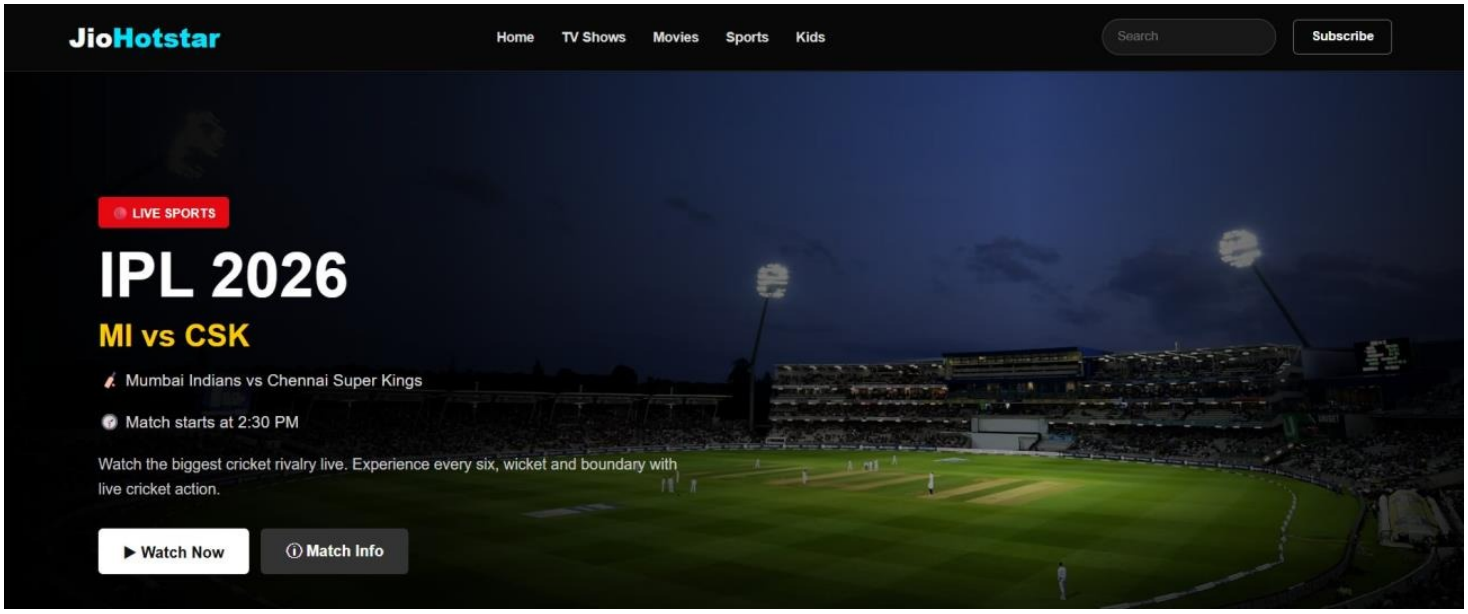
NETWORK

Artifact: Host port 80 mapped to container port 80

Responsibility: Use TLS, ingress, firewall rules and controlled exposure

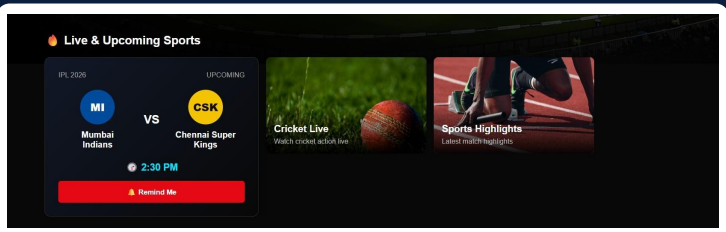
© 2026 cloudmessenger

Visual project showcase: what the container serves



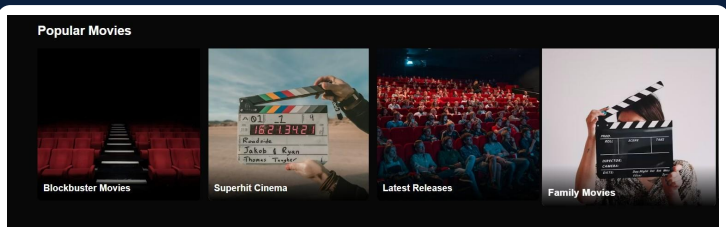
SCREEN 1

Landing-page hero with navigation, live-sports messaging and primary calls to action.



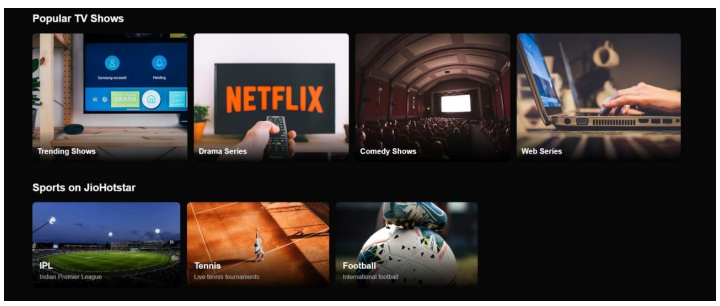
SCREEN 2

Live and upcoming sports: a match card plus supporting content cards.



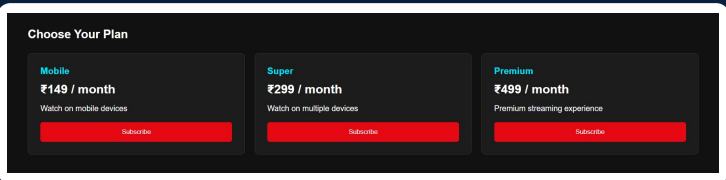
SCREEN 3

Popular Movies row with category cards designed for visual browsing.



SCREEN 4

Popular TV Shows and Sports sections with reusable content-card layouts.



SCREEN 5

Subscription plans: Mobile, Super and Premium cards.

WHY THESE SCREENSHOTS ARE HERE

They show the visible result of what the Docker workflow packages and serves: a static, streaming-style interface made of a hero banner, content rows, sports cards and subscription plans. They are project evidence, not external dependencies. In a production build these assets live in version control or come from a frontend build pipeline, then get copied into the image as a reviewed release artifact.

GOOD TO KNOW

Brand names, logos and artwork in this demo belong to their owners and are shown for learning only. See Figure 24 for the container-journey infographic of the same project.

Phase 1: server preparation

PHASE 1 • Server preparation

STEP 1

`sudo su`

Opens a root shell, so every command after it runs as the all-powerful administrator.

In production: Fine on a throw-away lab server. In production use sudo per command or a tightly controlled admin role, and keep auditing on.

STEP 2

`yum update -y`

Refreshes installed packages to their newest versions; -y answers every prompt with yes. Newer systems may use dnf underneath.

In production: Treat it as a change event: test in staging, take a snapshot, pick a maintenance window, reboot if the kernel changed.

STEP 3

`yum install docker -y`

Installs the Docker package and what it depends on. Package names differ between operating systems.

In production: Follow your distribution's official Docker Engine steps and verify the repository. Docker group membership is effectively root.

STEP 4

`which docker`

Prints the exact program that would run when you type docker. A quick "is it installed?" check.

In production: Great for spotting PATH problems or two installs. It is not a security check.

STEP 5

`docker --version`

Prints the version of the Docker command-line client.

In production: Record versions in your inventory. Use docker version to see both client and server.

STEP 6

`systemctl status docker`

A read-only health check: is the service active, recent log lines, process id, startup setting.

In production: Active does not mean your app is healthy. Dig into the journal, disk space and daemon settings when in doubt.

STEP 7

`systemctl start docker`

Starts the Docker service right now, for this boot only.

In production: Do this in provisioning or configuration management, not as an undocumented manual step.

STEP 8

`systemctl enable docker`

Makes Docker start automatically on future boots. It does not start it right now.

In production: systemctl enable --now docker does both. Test boot recovery so restarts do not cause a traffic surge.

© 2026 cloudmessenger

Phases 2 and 3: workspace, image and build

PHASE 2 • Project workspace

STEP 9

`mkdir jioHotstar`

Creates a dedicated folder for the site. It will become the Docker build context.

In production: Use a predictable layout such as /opt/jiohotstar/releases/VERSION. Linux names are case-sensitive.

STEP 10

`cd jioHotstar`

Moves into that folder. The later docker build uses a dot, meaning “this folder”, as its context.

In production: Check the working directory in automation, and add a .dockerignore to keep .git, secrets and junk out.

PHASE 3 • Image definition and build

STEP 11

`docker pull nginx`

Downloads an image from the registry. With no tag you get latest.

In production: Pin a tag and ideally a digest, for example nginx:1.27-alpine@sha256:DIGEST. Do not treat latest as a version.

STEP 12

`vi index.html`

Opens a terminal text editor to write the landing page.

In production: Quick experiments only. Keep the site in version control, review changes by pull request, build from a clean commit.

STEP 13

`touch Dockerfile`

Creates an empty file named Dockerfile, ready to edit.

In production: Commit it as source code. Identify releases by commit SHA, never by file timestamps.

STEP 14

`FROM nginx:alpine`

Chooses the base image: nginx on the tiny Alpine Linux, usually smaller than most alternatives.

In production: Pin it, scan it and rebuild when it updates. Small is not automatically secure.

STEP 15

`COPY . /usr/share/nginx/html`

Puts your project files into nginx’s web folder inside the image.

In production: Copy only the built artifact, add .dockerignore, and use a multi-stage build if the frontend must be compiled.

STEP 16

`EXPOSE 80`

Documents that the app listens on port 80. It does not open or publish the port.

In production: Enforce exposure at runtime, and put a load balancer or ingress with HTTPS in front.

STEP 17

`CMD ["nginx", "-g", "daemon off;"]`

The default start command. daemon off keeps nginx in the foreground so the container stays alive.

In production: Pair it with graceful shutdown, a health check, structured logs, resource limits and a restart policy.

STEP 18

`docker build -t jiohotstar .`

Reads the Dockerfile and creates the image. -t names it; the dot is the build context.

In production: Build in CI from a clean commit, tag with the commit SHA, scan, make an SBOM and push to a private registry.

© 2026 cloudmessenger

Phase 4: launch the container

the one command that starts everything

```
$ docker run -d --name jio-hotstar-server -p 80:80 jiohotstar
```

WHAT IT DOES

`docker run` creates a container from the **jiohotstar** image and starts it. Each flag has its own job, shown below with what changes for production.

FLAG: -D

Runs in the background (detached).

In production: Detached is not highly available. Use service supervision and collect logs.

FLAG: --NAME

Gives the container a stable, readable name.

In production: Orchestrators track workloads by service and labels, not by name.

FLAG: -P 80:80

Maps host port 80 to container port 80.

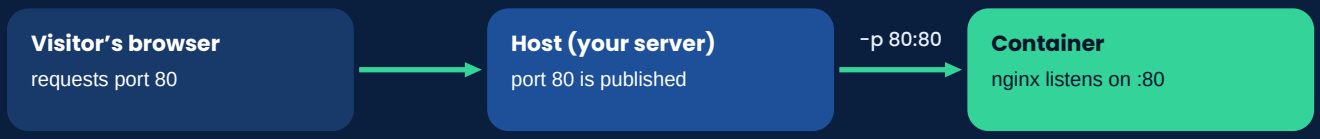
In production: This exposes the service publicly: bind on purpose, add TLS or ingress, match firewall rules.

IMAGE: JIOHOTSTAR

The image the container is created from.

In production: Use an immutable registry reference or digest, not a floating name.

How port mapping carries a visitor to nginx



GOOD FOR LEARNING

A clear, simple demonstration of publishing a container port. Perfect for a lab server or a first deployment.

BEFORE PRODUCTION

- Add a restart policy such as --restart unless-stopped, once you understand failure behaviour.
- Set CPU and memory limits, configure log rotation and define a health check.
- Deploy through Compose, Swarm, Kubernetes or a managed platform when you need service discovery, rolling updates, secrets and replicas.

© 2026 cloudmessenger

Production readiness checklist

4 **CRITICAL controls**
fix these first

3 **HIGH-priority controls**
plan these right after

IDENTITY AND ACCESS — CRITICAL

Use named non-root admin users, least privilege, MFA where available and audited sudo. Avoid routine root shells.

SUPPLY CHAIN — CRITICAL

Pin the OS, nginx base image and dependencies; scan and sign images; record provenance and SBOMs.

BUILD CONTEXT — CRITICAL

Add .dockerignore, build from version control in CI, and never copy secrets into the image.

NETWORK SECURITY — CRITICAL

Use a firewall, TLS, security headers, restricted admin ports and an ingress or load balancer.

AVAILABILITY — HIGH

Use health checks, a restart or replacement policy, capacity limits and a tested rollback path.

OBSERVABILITY — HIGH

Centralise access and error logs, collect metrics, alert on failures and keep deployment identifiers.

OPERATIONS — HIGH

Define patch windows, image rebuild cadence, backup and recovery expectations and incident ownership.

© 2026 cloudmessenger

A safer reference implementation

SAME TEACHING IDEA, BETTER REPEATABILITY

The Dockerfile keeps its four familiar lines, but copies only the reviewed, built site. The exact image digest should be chosen and verified by your release process, not copied blindly.

Dockerfile

```
FROM nginx:alpine
# Copy only the reviewed, built site artifact.
COPY ./dist/ /usr/share/nginx/html/
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

build and run

```
# Build from a clean checkout and an immutable release identifier.
docker build --pull -t registry.example.com/jiohotstar:${GIT_COMMIT_SHA} .
# Run with explicit operational controls; adapt to the platform.
docker run -d --name jio-hotstar-server --restart unless-stopped --memory 256m --cpus 1.0 -p 80:80
  registry.example.com/jiohotstar:${GIT_COMMIT_SHA}
```

Learning version vs production version

TOPIC	LEARNING DEMO	PRODUCTION
Server access	sudo su root shell	Scoped sudo, named admin users, audit trail
Base image	nginx:alpine, unchecked	Pinned tag or digest, scanned, rebuilt on updates
Files copied	COPY . (everything)	COPY ./dist/ (only the built site)
Image name	jiohotstar (latest)	registry/jiohotstar:COMMIT_SHA
Where it is built	By hand on the server	In CI from a clean, reviewed commit
Run settings	docker run -d -p 80:80	Add restart policy, memory and CPU limits
Traffic	Plain HTTP on port 80	HTTPS behind a load balancer or ingress

STILL ONLY A BASELINE

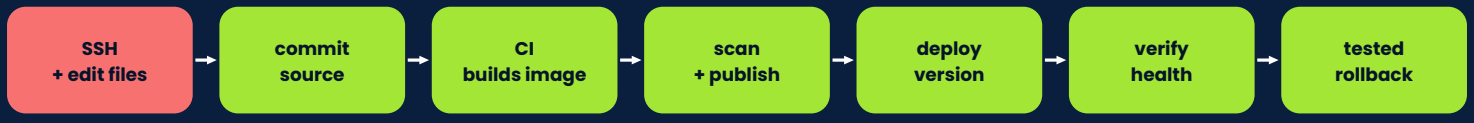
Public production traffic should normally use HTTPS, a reverse proxy or load balancer, a private image registry, automated deployment, and a platform that replaces unhealthy instances without manual SSH sessions.

© 2026 cloudmessenger

Command summary and conclusion

COMMAND	ROLE IN THIS PROJECT	PRODUCTION READING
`sudo su`	Open a root shell	Prefer scoped sudo and audited access
`yum update -y`	Update host packages	Tested, scheduled patch management
`yum install docker -y`	Install Docker	Use supported, verified repositories
`which docker / docker --version`	Verify CLI availability	Record client and daemon versions
`systemctl status/start/enable docker`	Inspect, start and enable the daemon	Automated provisioning and boot recovery
`mkdir / cd`	Create and enter workspace	Controlled release directories, clean contexts
`docker pull nginx`	Download base image	Pin, scan and verify provenance
`vi index.html / touch Dockerfile`	Create or edit source files	Version-controlled, reviewed changes
`FROM / COPY / EXPOSE / CMD`	Define image and runtime contract	Minimal, pinned, observable, secure image
`docker build`	Create the image	CI build, immutable tags, SBOM, scan, sign
`docker run`	Create and start the container	Managed rollout, limits, health, TLS, recovery

The upgrade that matters most



CONCLUSION

The original sequence is valuable because it makes the container lifecycle visible: prepare the host, create a workspace, define the image, build the artifact and run the process. Its main limit is that it is manual, mutable and single-host.

The most important upgrade is to move from “SSH to a server and edit files” to “commit source, build an immutable image in CI, scan and publish it, deploy a versioned release, verify health and keep a tested rollback.” That keeps the project simple while making it auditable and repeatable.

© 2026 cloudmessenger

References, scope and credit

[1]

Dockerfile reference
Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

[2]

docker build
Docker docs: docker build
https://docs.docker.com/reference/cli/docker/build/

[3]

docker run
Docker docs: docker run
https://docs.docker.com/reference/cli/docker/container/run/

[4]

Docker image digest
Docker docs: pull an image by digest
https://docs.docker.com/reference/cli/docker/image/pull/#pull-an-image-by-digest

[5]

Docker security
Docker docs: Docker security
https://docs.docker.com/engine/security/

[6]

NGINX Docker image
Docker Hub: official nginx image
https://hub.docker.com/_/nginx

[7]

systemctl manual
systemd: systemctl manual
https://www.freedesktop.org/software/systemd/man/latest/systemctl.html

[8]

Build checks
Docker docs: Build checks
https://docs.docker.com/build/checks/

SCOPE NOTE

This chapter explains the commands and their operational meaning. It is not a distribution-specific hardening standard. Validate package names, repository instructions, firewall rules, image versions and compliance requirements against your cloud provider, Linux distribution and organisational policy.

CREDIT

Chapter 11 is based on the technical guide **“From Code to Container”** by **Mohit Gadilohar** (dated 24-8-2026), re-presented here in the Cloud Messenger™ visual style. The reference list above is reproduced as given in that guide.

Keep building with Cloud Messenger™
cloudmessenger.in

#buildwithCloud-Messenger

© 2026 cloudmessenger

CHEAT QUICK REFERENCE

Docker command cheat sheet

CHECK DOCKER

```
$ docker --version
$ docker info
$ docker run hello-world
```

CONTAINERS

```
$ docker run -d -p 8080:80 --name web nginx
$ docker ps
$ docker ps -a
$ docker logs web
$ docker exec -it web bash
$ docker stop web
$ docker rm web
```

PUSH TO AWS ECR

```
$ aws ecr get-login-password --region REGION | docker
  login --username AWS --password-stdin
  ACCOUNT.dkr.ecr.REGION.amazonaws.com
$ docker tag flipkart:v1
  ACCOUNT.dkr.ecr.REGION.amazonaws.com/flipkart:v1
$ docker push
  ACCOUNT.dkr.ecr.REGION.amazonaws.com/flipkart:v1
```

IMAGES

```
$ docker pull nginx
$ docker images
$ docker rmi IMAGE
$ docker build -t name:v1 .
```

NETWORKING

```
$ docker network ls
$ docker network inspect bridge
$ docker network create my-network
$ docker run -d --network my-network nginx
$ docker network connect my-network web
```

COMPOSE

```
$ docker compose up -d
$ docker compose ps
$ docker compose logs
$ docker compose down
$ docker compose up --build -d
```


© 2026 cloudmessenger

Glossary

Virtualization
One physical computer pretending to be several.

VM (Virtual Machine)
A software computer with its own guest operating system.

Image
A read-only package with the app and all its dependencies.

Docker Engine / daemon
The background service that builds and runs containers.

Docker Hub
The public registry, the “Play Store” of images.

Bridge network
Default private network joining containers on one host.

Service (Compose)
One container definition inside a compose file.

ECS
AWS Elastic Container Service, which runs and scales containers.

EC2
An AWS virtual server, “a rented computer on the internet”.

Hypervisor
Software that creates and manages virtual machines.

Container
A lightweight, isolated running instance of an image.

Dockerfile
A text recipe of instructions used to build an image.

Registry
A store for images. Docker Hub and Amazon ECR are registries.

Port mapping (-p)
Connecting a host port to a container port, e.g. 8080:80.

Compose
A tool to define and run many containers from one YAML file.

ECR
AWS Elastic Container Registry, a place to store images.

Orchestration
Automating the start, scaling and healing of many containers (Kubernetes, ECS).

Security group
The AWS firewall in front of an EC2 instance.

© 2026 cloudmessenger

Resource index

AWS: What is virtualization?
https://aws.amazon.com/what-is/virtualization/

AWS: What is a hypervisor?
https://aws.amazon.com/what-is/hypervisor

AWS: Type 1 vs Type 2 hypervisors
https://aws.amazon.com/compare/the-difference-between-type-1-and-type-2-hypervisors

AWS Nitro System
https://aws.amazon.com/ec2/nitro/

AWS whitepaper: Traditional virtualization primer
https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/traditional-virtualization-primer.html

AWS: Types of cloud computing
https://aws.amazon.com/types-of-cloud-computing/

Docker docs: Install Docker Engine
https://docs.docker.com/engine/install/

Docker docs: What is Docker?
https://docs.docker.com/get-started/docker-overview/

Docker docs: Linux post-install steps
https://docs.docker.com/engine/install/linux-postinstall/

Docker docs: Install on Ubuntu
https://docs.docker.com/engine/install/ubuntu/

Docker Desktop for Windows
https://docs.docker.com/desktop/setup/install/windows-install/

Docker Desktop documentation
https://docs.docker.com/desktop/

Docker Desktop for Linux
https://docs.docker.com/desktop/setup/install/linux/

Docker Desktop for Mac
https://docs.docker.com/desktop/setup/install/mac-install/

Docker Hub
https://hub.docker.com/

Docker Desktop download page
https://www.docker.com/products/docker-desktop/

Docker Hub: official ubuntu image
https://hub.docker.com/_/ubuntu

Docker Hub: official nginx image
https://hub.docker.com/_/nginx

Docker docs: Dockerfile reference
https://docs.docker.com/reference/dockerfile/

GitHub: official nginx Dockerfiles
https://github.com/nginx/docker-nginx

Docker docs: CLI reference
https://docs.docker.com/reference/cli/docker/

Docker docs: Building best practices
https://docs.docker.com/build/building/best-practices/

Docker docs: How Compose works
https://docs.docker.com/compose/intro/compose-application-model/

Docker docs: Compose overview
https://docs.docker.com/compose/

Docker docs: Networking overview
https://docs.docker.com/engine/network/

Docker docs: Compose services reference
https://docs.docker.com/reference/compose-file/services/

Docker docs: Standalone networking tutorial
https://docs.docker.com/engine/network/tutorials/standalone/

Docker docs: Network drivers
https://docs.docker.com/engine/network/drivers/

AWS: Connect to your EC2 instance
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect.html

AWS: Install Docker on Amazon Linux 2023
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-container-image.html

AWS: Security group rules (open port 80)
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-rules-reference.html

AWS: What is Amazon ECR?
https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html

Links were reviewed in November 2026. Websites move pages from time to time; if a link fails, search the page title shown above it.

© 2026 cloudmessenger

Resource index (continued)

AWS: Push an image to Amazon ECR
https://docs.aws.amazon.com/AmazonECR/latest/userguide/getting-started-cli.html

Terraform documentation
https://developer.hashicorp.com/terraform/docs

GitHub Actions documentation
https://docs.github.com/en/actions

GitHub docs: Cloning a repository
https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

Docker docs: pull an image by digest
https://docs.docker.com/reference/cli/docker/image/pull/#pull-an-image-by-digest

systemd: systemctl manual
https://www.freedesktop.org/software/systemd/man/latest/systemctl.html

AWS: Amazon ECS Developer Guide
https://docs.aws.amazon.com/AmazonECS/latest/developerguide/Welcome.html

Ansible documentation
https://docs.ansible.com/

Kubernetes documentation
https://kubernetes.io/docs/

Docker docs: docker build
https://docs.docker.com/reference/cli/docker/build/

Docker docs: docker run
https://docs.docker.com/reference/cli/docker/container/run/

Docker docs: Docker security
https://docs.docker.com/engine/security/

Docker docs: Build checks
https://docs.docker.com/build/checks/

Links were reviewed in November 2026. Websites move pages from time to time; if a link fails, search the page title shown above it.

© 2026 cloudmessenger

Credits, notes & copyright

Cloud Messenger™

Learn • Build • Visualize • Troubleshoot • Grow

Website cloudmessenger.in

LinkedIn www.linkedin.com/in/cloud-messenger

GitHub github.com/Cloud-Messenger

#buildwithCloud-Messenger

CREATED BY

Created by cloud-messenger™. Learn • Build • Visualize • Troubleshoot • Grow. Made with care by cloud-messenger™.

SOURCE NOTES

- Compiled from 36 supplied images (35 unique diagrams, whiteboard sketches and screenshots; docker-architecture1.jpeg duplicated another file and is used once) and one technical-guide PDF, “From Code to Container” by Mohit Gadilohar (24-8-2026), which forms Chapter 11. Its reference list is reproduced as given.
- The promotional banner of another brand at the foot of the cloud-models image was cropped out. The Dockerfile line-by-line notes carry an “Anusha Janjirla” watermark and are credited to that author.
- The streaming-site demo (Figure 24) shows third-party brand and character artwork that belongs to its owners, included only for learning.
- Docker, Docker Hub, Amazon Web Services, EC2, ECS, ECR, nginx, Kubernetes, Terraform, Ansible, GitHub and other names are trademarks of their respective owners. This is an independent learning guide and is not affiliated with or endorsed by them.
- Commands are for learning environments. Opening 0.0.0.0/0 on a security group exposes a port to the whole internet; restrict it when you can.

COPYRIGHT NOTICE

© 2026 cloudmessenger. The compilation, structure, layout, design and branding of this workbook are the original work of cloudmessenger.